

LAPORAN *PROJECT*
MATA KULIAH BASIS DATA
SISTEM INFORMASI RUMAH SAKIT



Kelompok A

Statistika A

Nama Anggota:

- | | |
|----------------------------|-------------|
| 1. Nabila Alsa Haryanto | (K1D024006) |
| 2. Nayla Safira Aulia | (K1D024012) |
| 3. Meylina Nur Sasmita | (K1D024017) |
| 4. Naila Dwina Azzahra | (K1D024020) |
| 5. Chindy Renita Listiyana | (K1D024039) |

KEMENTERIAN PENDIDIKAN TINGGI, SAINS DAN TEKNOLOGI
UNIVERSITAS JENDERAL SOEDIRMAN
FAKULTAS MATEMATIKA DAN ILMU PENGETAHUAN ALAM
S1 STATISTIKA
2026

DAFTAR ISI

DAFTAR ISI	i
BAB 1	1
PENDAHULUAN.....	1
1.1. Latar Belakang	1
1.2. Rumusan Masalah	2
1.3. Tujuan	3
1.4. Manfaat	3
BAB II	5
ANALISIS KEBUTUHAN.....	5
2.1. Deskripsi Studi Kasus dan Ruang Lingkup Sistem	5
2.2. Identifikasi Aktor atau Pengguna	6
2.3. Daftar Kebutuhan Fungsional Terkait Data	7
2.4. Daftar Business Rules atau Aturan Bisnis Rumah Sakit	8
BAB III	11
PERANCANGAN BASIS DATA	11
3.1. Entity Relationship Diagram (ERD) Konseptual	11
3.2. Pemetaan ERD menjadi Skema Relasional.....	12
3.3. Penjelasan Entitas, Atribut, Primary Key, Foreign Key, dan Kardinalitas	13
3.4. Proses Normalisasi	26
3.5. Data Dictionary Lengkap	40
BAB IV	47
IMPLEMENTASI	47
4.1. Implementasi DDL (Data Definition Language).....	47
4.1.1. Tabel Poli.....	47
4.1.2. Tabel Dokter	47
4.1.3. Tabel Pasien.....	48
4.1.4. Tabel Jadwal.....	48
4.1.5. Tabel Pendaftaran.....	49

4.1.6.	Tabel Rekam Medis.....	49
4.1.7.	Tabel Diagnosa.....	50
4.1.8.	Tabel Obat.....	50
4.1.9.	Tabel Resep.....	51
4.1.10.	Tabel Vital	51
4.1.11.	Tabel Audit Log	52
4.1.12.	Index.....	52
4.2.	Implementasi DML (Data Manipulation Language)	53
4.2.1.	INSERT Data Tabel poli.....	53
4.2.2.	INSERT Data Tabel dokter	53
4.2.3.	INSERT Data Tabel pasien.....	54
4.2.4.	INSERT Data Tabel jadwal	55
4.2.5.	INSERT Data Tabel pendaftaran.....	57
4.2.6.	INSERT Data Tabel rekam_medis	58
4.2.7.	INSERT Data Tabel diagnosa	60
4.2.8.	INSERT Data Tabel obat.....	61
4.2.9.	INSERT Data Tabel resep	62
4.2.10.	INSERT Data Tabel vital.....	64
4.3.	Implementasi Query SELECT.....	65
4.3.1.	Query Sederhana	66
4.3.2.	Query dengan JOIN	67
4.3.3.	Query dengan Subquery dan CTE	70
4.3.4.	Query dengan Fungsi Agregat, GROUP BY, dan HAVING	72
4.4.	Implementasi View	73
4.4.1.	View view_riwayat_pasien	73
4.4.2.	View view_laporan_harian	75
4.5.	Implementasi Stored Procedure	76
4.5.1.	Procedure riwayat_pasien.....	76
4.6.	Implementasi Trigger.....	77
4.6.1.	Trigger tr_audit_pasien_insert	77
4.6.2.	Trigger tr_audit_pasien_update.....	78
4.6.3.	Trigger tr_audit_pasien_delete	80

4.7.	Implementasi Function	81
4.7.1.	Function fn_hitung_usia	81
4.7.2.	Function fn_total_biaya_obat	82
BAB V		84
PENGUJIAN QUERY		84
5.1.	Pengujian DDL dan DML	84
5.2.	Pengujian Query Select.....	85
5.2.1.	Menampilkan Seluruh Data Pasien	86
5.2.2.	Menampilkan Daftar Dokter Spesialis Penyakit Dalam	87
5.2.3.	Menampilkan Obat dengan Harga di Atas 4000	88
5.3.	Pengujian Query dengan JOIN	88
5.3.1.	Menampilkan Data Pendaftaran Pasien Beserta Dokter yang Menanganinya ..	89
5.3.2.	Menampilkan Riwayat Pemeriksaan Pasien	91
5.3.3.	Menampilkan Resep Obat per Pasien	92
5.3.4.	Menampilkan Dokter dan Jadwal Poli	94
5.4.	Pengujian Query Subquery dan CTE.....	96
5.4.1.	Menampilkan Pasien yang Pernah Periksa Lebih dari Satu Kali (Subquery) ...	96
5.4.2.	Menampilkan Jumlah Kunjungan Setiap Dokter (CTE)	97
5.5.	Pengujian Query GROUP BY dan HAVING.....	98
5.5.1.	Menampilkan Dokter yang Menangani Pasien Lebih dari Tiga Pasien	98
5.6.	Pengujian View.....	98
5.6.1.	Menampilkan pengujian view_riwayat_pasien	99
5.6.2.	Menampilkan pengujian view_laporan_harian	100
5.7.	Pengujian Stored Procedure.....	101
5.8.	Pengujian Trigger	101
5.9.	Pengujian Function.....	103
5.9.1.	Menampilkan pengujian fn_hitung_usia	103
5.9.2.	Menampilkan pengujian fn_total_biaya_obat	104
LAMPIRAN		107

DAFTAR TABEL

Tabel 3. 1	Tabel Pemetaan ERD Sistem Informasi Rumah Sakit	12
Tabel 3. 2	Tabel Entitas Pasien.....	14
Tabel 3. 3	Tabel Entitas Poli.....	15
Tabel 3. 4	Tabel Entitas Dokter	16
Tabel 3. 5	Tabel Entitas Jadwal	17
Tabel 3. 6	Tabel Entitas Pendaftaran	18
Tabel 3. 7	Tabel Entitas Rekam Medis.....	19
Tabel 3. 8	Tabel Entitas Vital	21
Tabel 3. 9	Tabel Entitas Diagnosa	22
Tabel 3. 10	Tabel Entitas Obat.....	23
Tabel 3. 11	Tabel Entitas Resep	24
Tabel 3. 12	Tabel Kardinalitas.....	25
Tabel 3. 13	Tabel UNF	27
Tabel 3. 14	Tabel Normalisasi 1NF	32
Tabel 3. 15	Tabel Normalisasi 2NF	37
Tabel 3. 16	Tabel Normalisasi 3NF	39
Tabel 3. 17	Tabel Dictionary Pasien.....	40
Tabel 3. 18	Tabel Dictionary Poli.....	41
Tabel 3. 19	Tabel Dictionary Dokter	41
Tabel 3. 20	Tabel Dictionary Jadwal	42
Tabel 3. 21	Tabel Dictionary Pendaftaran	42
Tabel 3. 22	Tabel Dictionary Rekam Medis.....	43
Tabel 3. 23	Tabel Dictionary Vital	44
Tabel 3. 24	Tabel Dictionary Diganosa	44
Tabel 3. 25	Tabel Dictionary Obat	45
Tabel 3. 26	Tabel Dictionary Resep	45

DAFTAR GAMBAR

Gambar 3.1 Gambar ERD.....	11
Gambar 5. 1 Tampilan DDI dan DML.....	84
Gambar 5. 2 Tampilan DDL dan DML.....	85
Gambar 5. 3 Tampilan Seluruh Data Pasien.....	86
Gambar 5. 4 Tampilan Seluruh Data Pasien.....	86
Gambar 5. 5 Tampilan Daftar Dokter Spesialis.....	87
Gambar 5. 6 Tampilan Obat dengan Harga.....	88
Gambar 5. 7 Tampilan Data Pendaftaran Pasien Beserta.....	89
Gambar 5. 8 Tampilan Data Pendaftaran Pasien Beserta.....	90
Gambar 5. 9 Tampilan Riwayat Pemeriksaan Pasien.....	91
Gambar 5. 10 Tampilan Riwayat Pemeriksaan Pasien.....	91
Gambar 5. 11 Tampilan Resep Obat per Pasien.....	92
Gambar 5. 12 Tampilan Resep Obat per Pasien.....	93
Gambar 5. 13 Tampilan Resep Obat per Pasien.....	93
Gambar 5. 14 Tampilan Dokter dan Jadwal Poli.....	94
Gambar 5. 15 Tampilan Dokter dan Jadwal Poli.....	95
Gambar 5. 16 Tampilan Pasien yang Pernah Periksa Lebih.....	96
Gambar 5. 17 Tampilan Jumlah Kunjungan Setiap Dokter.....	97
Gambar 5. 18 Tampilan Dokter yang Menangani.....	98
Gambar 5. 19 Tampilan Riwayat Pasien.....	99
Gambar 5. 20 Tampilan Laporan Harian.....	100
Gambar 5. 21 Tampilan Laporan Harian.....	100
Gambar 5. 22 Tampilan Procedure.....	101
Gambar 5. 23 Tampilan Trigger.....	102
Gambar 5. 24 Tampilan Function Menghitung Usia.....	103
Gambar 5. 25 Tampilan Function Menghitung Usia.....	103
Gambar 5. 26 Tampilan Function Total Biaya Obat.....	104
Gambar 5. 27 Tampilan Function Total Biaya Obat.....	104

BAB 1

PENDAHULUAN

1.1. Latar Belakang

Perkembangan teknologi informasi telah memberikan dampak yang signifikan terhadap berbagai bidang kehidupan, termasuk sektor kesehatan. Rumah sakit sebagai salah satu institusi pelayanan kesehatan dituntut untuk mampu memberikan pelayanan yang cepat, tepat, dan berkualitas kepada masyarakat. Untuk mendukung pelayanan tersebut, diperlukan sistem pengelolaan data yang baik agar informasi dapat diakses secara akurat, konsisten, dan tepat waktu.

Dalam kegiatan operasional rumah sakit, terdapat berbagai data yang harus dikelola, seperti data pasien, dokter, pendaftaran, poli, jadwal praktik, rekam medis, vital, diagnosa, obat, dan resep. Data-data tersebut saling berkaitan dan terus bertambah seiring meningkatnya jumlah pasien serta aktivitas pelayanan kesehatan yang dilakukan setiap hari. Apabila pengelolaan data masih dilakukan secara manual atau belum terintegrasi dengan baik, maka dapat menimbulkan berbagai permasalahan, seperti duplikasi data, inkonsistensi informasi, kesalahan pencatatan, serta kesulitan dalam proses pencarian dan pengolahan data.

Basis data merupakan salah satu komponen penting dalam pengembangan sistem informasi karena berfungsi sebagai media penyimpanan data yang terorganisasi. Perancangan basis data yang baik dapat menghasilkan struktur data yang terintegrasi, mengurangi redundansi data, serta menjaga integritas dan konsistensi informasi yang tersimpan. Dengan demikian, proses pengelolaan data dapat dilakukan secara lebih efektif dan efisien sehingga mampu mendukung kegiatan operasional rumah sakit.

Sistem Informasi Rumah Sakit (SIRS) merupakan sistem yang dirancang untuk mengelola dan mengintegrasikan berbagai data serta proses pelayanan kesehatan dalam satu kesatuan yang saling terhubung. Sistem ini mencakup pengelolaan data pasien, dokter, pendaftaran, poli, dan jadwal praktik dokter untuk

mendukung proses pelayanan yang terorganisir. Selain itu, sistem juga menangani pencatatan rekam medis, data vital pasien, diagnosa, obat, serta resep yang diberikan kepada pasien. Dengan adanya integrasi antarentitas tersebut, informasi dapat dikelola secara lebih akurat dan efisien sehingga mendukung pengambilan keputusan serta meningkatkan kualitas pelayanan rumah sakit.

Berdasarkan kondisi tersebut, pada proyek ini dilakukan perancangan dan implementasi basis data relasional untuk Sistem Informasi Rumah Sakit yang mencakup pengelolaan data pasien, dokter, pendaftaran, poli, jadwal praktik, rekam medis, vital, diagnosa, obat dan resep. Perancangan dilakukan menggunakan Entity Relationship Diagram (ERD), normalisasi hingga bentuk normal ketiga (3NF), serta implementasi menggunakan Structured Query Language (SQL). Melalui perancangan basis data ini diharapkan dapat dihasilkan sistem yang mampu mendukung pengelolaan data rumah sakit secara efektif, efisien, dan terintegrasi.

1.2. Rumusan Masalah

1. Bagaimana merancang basis data relasional yang mampu mengintegrasikan data operasional pada Sistem Informasi Rumah Sakit?
2. Bagaimana memodelkan hubungan antar entitas dalam Sistem Informasi Rumah Sakit menggunakan Entity Relationship Diagram (ERD)?
3. Bagaimana menerapkan proses normalisasi hingga bentuk normal ketiga (3NF) agar diperoleh struktur basis data yang efisien dan meminimalkan redundansi data?
4. Bagaimana mengimplementasikan rancangan basis data Sistem Informasi Rumah Sakit ke dalam sistem manajemen basis data relasional menggunakan SQL?
5. Bagaimana memanfaatkan query dan objek basis data seperti view, function, stored procedure, dan trigger untuk mendukung pengelolaan data pada Sistem Informasi Rumah Sakit?

1.3. Tujuan

1. Merancang basis data relasional yang mampu mengintegrasikan data pasien, dokter, jadwal praktik, rekam medis, vital, diagnosa, obat, dan resep dalam Sistem Informasi Rumah Sakit.
2. Membuat Entity Relationship Diagram (ERD) yang menggambarkan hubungan antar entitas beserta struktur data yang digunakan dalam sistem.
3. Menerapkan proses normalisasi hingga bentuk normal ketiga (3NF) untuk menghasilkan basis data yang terstruktur, konsisten, dan efisien.
4. Mengimplementasikan rancangan basis data ke dalam sistem manajemen basis data relasional melalui pembuatan tabel, relasi, constraint, indeks, serta pengisian data menggunakan SQL.
5. Mengembangkan *query*, *view*, *function*, *stored procedure*, dan *trigger* yang dapat mendukung pengelolaan data serta kebutuhan operasional pada Sistem Informasi Rumah Sakit.

1.4. Manfaat

1. Bagi Rumah Sakit
 - a. Membantu pengelolaan data rumah sakit secara lebih terstruktur, terintegrasi, dan mudah diakses.
 - b. Meningkatkan efisiensi dalam proses penyimpanan, pencarian, dan pengolahan data yang berkaitan dengan pelayanan kesehatan.
 - c. Mengurangi risiko terjadinya duplikasi data, kesalahan pencatatan, dan inkonsistensi informasi.
 - d. Mendukung penyediaan informasi yang akurat untuk membantu proses pengambilan keputusan serta peningkatan kualitas pelayanan kesehatan.
2. Bagi Pengguna Sistem
 - a. Mempermudah petugas administrasi dalam mengelola data pasien dan pendaftaran.
 - b. Membantu dokter dalam mengakses informasi pasien, rekam medis, vital dan diagnosa secara lebih cepat.

- c. Memudahkan pengelolaan data obat dan resep dalam satu sistem yang terintegrasi.
3. Bagi Mahasiswa
- a. Menambah pemahaman mengenai konsep perancangan dan implementasi basis data relasional.
 - b. Memberikan pengalaman dalam menerapkan Entity Relationship Diagram (ERD), normalisasi, SQL, serta objek basis data pada studi kasus nyata.
 - c. Melatih kemampuan analisis, perancangan sistem, pemanfaatan *version control system* (GitHub) untuk kolaborasi, serta kerja sama tim dalam menyelesaikan proyek basis data.

BAB II

ANALISIS KEBUTUHAN

2.1. Deskripsi Studi Kasus dan Ruang Lingkup Sistem

Sistem Informasi Rumah Sakit (SIRS) merupakan sistem basis data yang dirancang untuk mendukung pengelolaan data dan proses pelayanan kesehatan secara terintegrasi. Rumah sakit sebagai institusi pelayanan kesehatan harus mengelola berbagai jenis data yang saling berkaitan, seperti data pasien, dokter, poli, jadwal praktik, pendaftaran, rekam medis, data vital pasien, diagnosa, obat, dan resep. Pengelolaan data yang baik diperlukan agar informasi dapat diakses secara cepat, akurat, dan konsisten sehingga dapat mendukung kualitas pelayanan kepada pasien.

Dalam proses pelayanan kesehatan, pasien terlebih dahulu melakukan pendaftaran untuk memperoleh layanan pada poli yang dipilih sesuai dengan jadwal praktik dokter yang tersedia. Dokter yang bertugas pada poli tertentu akan melakukan pemeriksaan terhadap pasien dan mencatat hasil pemeriksaan ke dalam rekam medis. Data vital pasien, seperti tekanan darah dan berat badan juga dicatat sebagai bagian dari informasi medis yang mendukung proses diagnosis. Berdasarkan hasil pemeriksaan, dokter dapat menetapkan diagnosa serta memberikan resep yang berisi satu atau lebih jenis obat sesuai dengan kondisi pasien.

Melalui sistem yang terintegrasi, pengelolaan data pasien, dokter, poli, jadwal praktik, pendaftaran, rekam medis, vital, diagnosa, obat, dan resep dapat dilakukan secara lebih efektif dan efisien. Sistem ini membantu mengurangi risiko duplikasi data, mempermudah pencarian informasi, serta mendukung penyediaan data yang akurat untuk kebutuhan operasional rumah sakit. Oleh karena itu, ruang lingkup sistem yang dirancang dalam proyek ini meliputi:

- a. Pengelolaan data pasien.
- b. Pengelolaan data pendaftaran pasien.
- c. Pengelolaan data jadwal praktik.

- d. Pengelolaan data vital.
- e. Pengelolaan data diagnosa penyakit.
- f. Pengelolaan data rekam medis pasien.
- g. Pengelolaan data dokter.
- h. Pengelolaan data obat.
- i. Pengelolaan data resep
- j. Pengelolaan data poli.

2.2. Identifikasi Aktor atau Pengguna

Untuk menjalankan fungsi-fungsi dalam Sistem Informasi Rumah Sakit secara aman dan terstruktur, terdapat beberapa aktor utama yang berinteraksi dengan sistem beserta hak aksesnya masing-masing:

- a. Petugas Administrasi atau Resepsionis
 - 1) Memiliki hak akses untuk mengelola data pasien baru maupun memperbarui data pasien yang sudah terdaftar
 - 2) Menginput data pendaftaran rawat jalan berdasarkan poli dan jadwal praktik dokter yang dipilih
 - 3) Melihat informasi jadwal praktik dokter dan data poli yang tersedia
- b. Dokter
 - 1) Melihat data pasien serta rekam medis pasien yang pernah melakukan pemeriksaan
 - 2) Menginput data vital pasien sebagai hasil pemeriksaan awal
 - 3) Menginput diagnose dan mencatat hasil pemeriksaan ke dalam rekam medis
 - 4) Membuat resep obat yang sesuai dengan hasil diagnosa pasien
- c. Petugas Farmasi atau Apoteker
 - 1) Melihat resep dan detail resep yang diinput oleh dokter untuk menyiapkan obat
 - 2) Melihat dan mengelola informasi obat yang tercantum pada resep pasien dalam sistem

- d. Manajer atau Direktur Rumah Sakit
 - 1) Memiliki hak akses untuk mengelola data master seperti data dokter, poli, jadwal praktik, dan obat.
 - 2) Mengakses seluruh data yang diperlukan untuk keperluan monitoring dan evaluasi pelayanan rumah sakit
 - 3) Melihat laporan terkait pendaftaran pasien, rekam medis, diagnosa, dan penggunaan obat

2.3. Daftar Kebutuhan Fungsional Terkait Data

Untuk menjamin pengelolaan data yang terstruktur, Sistem Informasi Rumah Sakit (SIRS) wajib memenuhi sejumlah kebutuhan fungsional spesifik terkait manajemen data di bawah ini. Komponen-komponen fungsional ini merefleksikan bagaimana sistem seharusnya bekerja dalam menyimpan, memproses, serta mengendalikan aliran data operasional rumah sakit agar meminimalkan redundansi dan meningkatkan efisiensi kerja.

- a. Pengelolaan Data Poli

Sistem harus mampu menyimpan dan mengelola data poli yang meliputi nama poli dan keterangan. Sistem harus dapat menampilkan daftar poli yang tersedia serta mendukung penambahan dan pembaruan data poli.

- b. Pengelolaan Data Dokter

Sistem harus mampu menyimpan data dokter yang mencakup nama, spesialisasi, nomor telepon, dan poli tempat dokter bertugas. Sistem harus mendukung pencarian dokter berdasarkan nama maupun poli, serta menampilkan informasi ketersediaan dokter berdasarkan jadwal praktik.

- c. Pengelolaan Data Jadwal Praktik

Sistem harus mampu menyimpan jadwal praktik dokter yang mencakup hari praktik, jam mulai, dan jam selesai. Sistem harus memvalidasi bahwa tidak ada jadwal yang saling tumpang tindih untuk dokter yang sama dan mendukung pengelolaan jadwal berdasarkan poli.

- d. Pengelolaan Data Pasien

Sistem harus mampu menyimpan dan mengelola data pasien yang mencakup nomor rekam medis, nama lengkap, tanggal lahir, jenis kelamin, alamat, nomor telepon, dan tanggal registrasi. Sistem harus memastikan setiap pasien memiliki identitas yang unik dan mendukung pencarian data pasien berdasarkan nama maupun nomor rekam medis.

e. Pengelolaan Data Pendaftaran

Sistem harus mampu mencatat data pendaftaran pasien yang mencakup tanggal pendaftaran, keluhan, status pendaftaran, serta referensi ke dokter dan jadwal praktik yang dipilih. Sistem harus memastikan satu pendaftaran hanya dapat dilakukan oleh satu pasien pada satu jadwal praktik tertentu dan mendukung pembaruan status pendaftaran.

f. Pengelolaan Data Rekam Medis dan Diagnosa

Sistem harus mampu mencatat rekam medis hasil pemeriksaan dokter yang mencakup tanggal periksa, anamnesis, hasil pemeriksaan fisik, dan catatan dokter. Setiap rekam medis harus dapat memuat satu atau lebih data diagnosa, beserta keterangan tambahan dari dokter.

g. Pengelolaan Obat dan Detail Resep

Sistem harus mampu mencatat data resep yang diberikan oleh dokter kepada pasien berdasarkan rekam medis tertentu. Setiap resep dapat memuat satu atau lebih detail resep yang mengacu pada data obat, termasuk dosis dan aturan pakai. Sistem harus memastikan obat yang diresepkan tersedia dalam basis data obat.

2.4. Daftar Business Rules atau Aturan Bisnis Rumah Sakit

Business rules merupakan aturan-aturan yang mengatur hubungan antar entitas dan proses bisnis dalam Sistem Informasi Rumah Sakit. Aturan ini dibuat berdasarkan kebutuhan sistem serta relasi yang terdapat pada Entity Relationship Diagram (ERD) sehingga dapat memastikan bahwa data yang tersimpan dalam basis data tetap konsisten, valid, dan sesuai dengan alur pelayanan rumah sakit.

Dalam sistem yang dirancang, proses pelayanan dimulai dari pasien yang melakukan pendaftaran pada jadwal praktik dokter tertentu. Dokter yang bertugas berada pada suatu poli dan memiliki jadwal praktik yang telah ditentukan. Setelah pasien menjalani pemeriksaan, sistem akan menghasilkan rekam medis yang memuat informasi keluhan, hasil pemeriksaan, data vital, diagnosa, serta resep obat yang diberikan. Setiap entitas dalam sistem memiliki hubungan dan batasan tertentu yang harus dipatuhi untuk menjaga integritas data dan mendukung kelancaran proses pelayanan kesehatan.

Berdasarkan proses bisnis tersebut, disusun business rules yang menjadi pedoman dalam perancangan dan implementasi basis data Sistem Informasi Rumah Sakit sebagai berikut.

1. Setiap poli memiliki identitas unik berupa id_poli.
2. Satu poli dapat memiliki banyak dokter, tetapi setiap dokter hanya terdaftar pada satu poli.
3. Setiap dokter memiliki identitas unik berupa id_dokter.
4. Satu dokter dapat memiliki lebih dari satu jadwal praktik.
5. Setiap jadwal praktik hanya dimiliki oleh satu dokter.
6. Setiap pasien memiliki identitas unik berupa id_pasien.
7. Satu pasien dapat melakukan pendaftaran berkali-kali untuk kunjungan yang berbeda.
8. Setiap pendaftaran hanya dilakukan oleh satu pasien.
9. Setiap pendaftaran harus terhubung dengan satu jadwal praktik dokter yang dipilih.
10. Satu jadwal praktik dapat melayani banyak pendaftaran pasien.
11. Setiap pendaftaran memiliki nomor antrian yang digunakan untuk mengatur urutan pelayanan.
12. Setiap pendaftaran memiliki status kunjungan, seperti "Menunggu" atau "Selesai".

13. Setiap pendaftaran menghasilkan tepat satu rekam medis setelah pasien diperiksa.
14. Setiap rekam medis hanya terkait dengan satu pendaftaran.
15. Setiap rekam medis dibuat oleh satu dokter yang melakukan pemeriksaan.
16. Satu dokter dapat membuat banyak rekam medis untuk pasien yang berbeda maupun kunjungan yang berbeda.
17. Setiap rekam medis memuat informasi keluhan, hasil pemeriksaan, dan tanggal pemeriksaan pasien.
18. Setiap rekam medis memiliki tepat satu data tanda vital (vital sign).
19. Data vital hanya dapat dicatat untuk satu rekam medis tertentu.
20. Setiap rekam medis dapat memiliki satu atau lebih diagnosa.
21. Setiap diagnosa hanya terkait dengan satu rekam medis.
22. Setiap rekam medis dapat memiliki satu atau lebih resep obat.
23. Setiap resep hanya terkait dengan satu rekam medis dan satu jenis obat.
24. Satu jenis obat dapat diresepkan pada banyak rekam medis yang berbeda.
25. Informasi resep harus memuat dosis, jumlah obat, dan aturan pakai untuk pasien.
26. Setiap obat memiliki identitas unik dan informasi harga yang digunakan dalam pelayanan farmasi.

BAB III

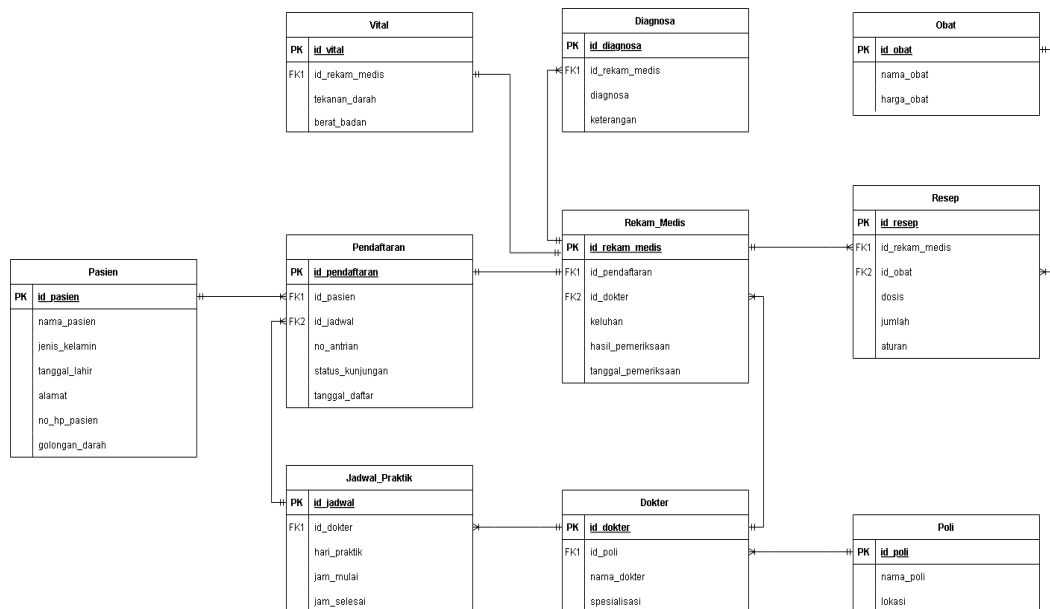
PERANCANGAN BASIS DATA

3.1. Entity Relationship Diagram (ERD) Konseptual

Entity Relationship Diagram (ERD) konseptual merupakan representasi visual dari hubungan antar entitas yang terdapat dalam Sistem Informasi Rumah Sakit (SIRS). ERD ini menggambarkan struktur data secara abstrak sebelum diimplementasikan ke dalam sistem manajemen basis data relasional. Diagram ini memuat seluruh entitas utama beserta atribut dan relasi yang menghubungkan satu entitas dengan entitas lainnya.

Sistem Informasi Rumah Sakit yang dirancang mencakup sepuluh entitas utama, yaitu: Pasien, Pendaftaran, Jadwal_Praktik, Dokter, Poli, Rekam_Medis, Vital, Diagnosa, Resep, dan Obat. Setiap entitas memiliki atribut-atribut yang merepresentasikan informasi yang perlu disimpan, serta memiliki kunci utama (Primary Key) yang berfungsi sebagai identitas unik setiap record dalam tabel.

Adapun ERD konseptual dari Sistem Informasi Rumah Sakit ini dapat dilihat pada gambar berikut.



Gambar 3.1 Gambar ERD

3.2. Pemetaan ERD menjadi Skema Relasional

Pemetaan ERD ke dalam skema relasional dilakukan berdasarkan kaidah-kaidah berikut.

1. Setiap entitas diubah menjadi sebuah tabel dengan atribut-atributnya, dan primary key entitas menjadi primary key tabel.
2. Relasi 1:N diimplementasikan dengan menambahkan foreign key pada tabel di sisi banyak (N), yang mengacu pada primary key tabel di sisi satu (1).
3. Relasi 1:1 diimplementasikan dengan menambahkan foreign key pada salah satu tabel, umumnya pada entitas yang bergantung.
4. Atribut yang dapat bernilai lebih dari satu (multivalued) dipisahkan menjadi tabel tersendiri dan dihubungkan melalui foreign key.

Berdasarkan kaidah-kaidah tersebut, hasil pemetaan ERD Sistem Informasi Rumah Sakit adalah sebagai berikut.

Tabel 3. 1 Tabel Pemetaan ERD Sistem Informasi Rumah Sakit

Entitas	Primary Key	Foreign Key	Atribut Lainnya
Pasien	id_pasien	-	nama_pasien, jenis_kelamin, tanggal_lahir, alamat, no_hp_pasien, golongan_darah
Dokter	id_dokter	id_poli	nama_dokter, spesialisasi
Pendaftaran	id_pendaftaran	Id_pasien, id_jadwal	no_antrian, status_kunjungan, tanggal_daftar
Poli	id_poli	-	nama_poli, lokasi

Entitas	Primary Key	Foreign Key	Atribut Lainnya
Jadwal_Praktik	id_jadwal	id_dokter	hari_praktik, jam_mulai, jam_selesai
Rekam_Medis	id_rekam_medis	id_pendaftaran, id_dokter	keluhan, hasil_pemeriksaan, tanggal_pemeriksaan
Vital	id_vital	id_rekam_medis	tekanan_darah, berat_badan
Diagnosa	id_diagnosa	id_rekam_medis	diagnosa, keterangan
Obat	id_obat	-	nama_obat, harga_obat
Resep	id_resep	id_rekam_medis, id_obat	dosis, jumlah, aturan

Setiap tabel hasil pemetaan di atas memiliki minimal satu atribut sebagai Primary Key yang bersifat unik dan tidak boleh bernilai NULL. Hubungan antar tabel diimplementasikan melalui Foreign Key yang memastikan setiap nilai yang dirujuk selalu memiliki data yang valid pada tabel asalnya.

3.3. Penjelasan Entitas, Atribut, Primary Key, Foreign Key, dan Kardinalitas

a. Entitas, Atribut, *Primary Key*, dan *Foreign Key*

1. Pasien

Entitas pasien digunakan untuk menyimpan data identitas pasien yang menerima pelayanan kesehatan di rumah sakit. Data pada entitas ini menjadi dasar dalam berbagai proses pelayanan, seperti pendaftaran, pemeriksaan, pencatatan rekam medis, dan pemberian obat. Setiap pasien memiliki identitas unik yang digunakan untuk membedakan satu pasien dengan pasien lainnya dalam sistem.

Tabel 3. 2 Tabel Entitas Pasien

Atribut	Penjelasan
id_pasien (PK)	Kode unik yang digunakan sebagai identitas utama setiap pasien dan menjadi pembeda antar data pasien dalam sistem.
nama_pasien	Menyimpan nama lengkap pasien sesuai identitas yang digunakan dalam proses pelayanan dan administrasi rumah sakit.
jenis_kelamin	Menyimpan informasi jenis kelamin pasien yang terdiri dari laki-laki (L) atau perempuan (P) dan digunakan untuk kebutuhan identifikasi serta pelayanan medis.
tgl_lahir	Menyimpan informasi tanggal lahir pasien yang terdiri dari tanggal, bulan, dan tahun untuk mengetahui usia pasien.
alamat	Menyimpan alamat tempat tinggal pasien yang mencakup nama jalan, nomor rumah, dan kabupaten/kota.
no_hp_pasien	Menyimpan nomor telepon pasien yang digunakan untuk keperluan komunikasi, konfirmasi jadwal, atau penyampaian informasi dari rumah

Atribut	Penjelasan
	sakit.
golongan_darah	Menyimpan informasi golongan darah pasien yang terdiri dari A, B, AB, atau O dan digunakan untuk mendukung tindakan medis tertentu.

2. Poli

Entitas poli digunakan untuk menyimpan informasi mengenai unit pelayanan kesehatan yang tersedia di rumah sakit. Setiap poli melayani bidang kesehatan tertentu dan menjadi tempat bertugas bagi dokter sesuai spesialisasinya. Data poli digunakan untuk mengelompokkan layanan kesehatan yang tersedia dalam sistem.

Tabel 3. 3 Tabel Entitas Poli

Atribut	Penjelasan
id_poli (PK)	Kode unik yang digunakan sebagai identitas utama setiap poli dan menjadi pembeda antar poli dalam sistem rumah sakit.
nama_poli	Menyimpan nama poli berdasarkan bidang pelayanan kesehatan yang tersedia, seperti Poli Umum, Poli Penyakit Dalam, Poli Bedah, Poli Gigi dan Mulut.
lokasi	Menyimpan informasi lokasi poli yang dapat terdiri atas gedung dan

	lantai sebagai petunjuk tempat pelayanan kesehatan berlangsung.
--	---

3. Dokter

Entitas dokter digunakan untuk menyimpan informasi tenaga medis yang memberikan pelayanan kesehatan kepada pasien. Setiap dokter terhubung dengan satu poli sesuai bidang spesialisasinya dan dapat memiliki beberapa jadwal praktik. Data dokter digunakan dalam proses pemeriksaan pasien dan pencatatan rekam medis.

Tabel 3. 4 Tabel Entitas Dokter

Atribut	Penjelasan
id_dokter (PK)	Kode unik yang digunakan sebagai identitas utama setiap dokter dan menjadi pembeda antar dokter dalam sistem rumah sakit.
id_poli (FK)	Kode poli tempat dokter bertugas yang digunakan untuk menghubungkan data dokter dengan data poli dalam sistem.
nama_dokter	Menyimpan nama lengkap dokter yang memberikan pelayanan kesehatan kepada pasien di rumah sakit.
spesialis	Menyimpan informasi bidang keahlian atau kompetensi dokter sesuai pendidikan dan sertifikasi

	yang dimiliki, seperti Dokter Umum, Spesialis Penyakit Dalam, Spesialis Bedah, Spesialis Konservasi Gigi.
--	---

4. Jadwal

Entitas jadwal digunakan untuk menyimpan informasi jadwal pelayanan dokter di rumah sakit. Data ini membantu pasien dalam mengetahui waktu praktik dokter yang tersedia serta mendukung pengelolaan jadwal pelayanan oleh rumah sakit. Setiap jadwal praktik terkait dengan satu dokter tertentu.

Tabel 3. 5 Tabel Entitas Jadwal

Atribut	Penjelasan
id_jadwal (PK)	Kode unik yang digunakan sebagai identitas utama setiap jadwal praktik dokter dalam sistem rumah sakit.
id_dokter (FK)	Kode dokter yang digunakan untuk menghubungkan data jadwal praktik dengan dokter yang bertugas.
hari_praktik	Menyimpan hari pelaksanaan praktik dokter yang digunakan untuk mengatur ketersediaan layanan kesehatan.
jam_mulai	Menyimpan waktu mulai praktik dokter yang menjadi acuan awal pelayanan pasien pada jadwal

	tertentu.
jam_selesai	Menyimpan waktu berakhirnya praktik dokter yang menandai berakhirnya pelayanan pada jadwal tersebut.

5. Pendaftaran

Entitas pendaftaran digunakan untuk mencatat proses registrasi pasien sebelum memperoleh pelayanan kesehatan. Entitas ini menghubungkan data pasien dengan jadwal praktik dokter yang dipilih. Selain itu, data pendaftaran digunakan untuk mengelola antrean dan status kunjungan pasien.

Tabel 3. 6 Tabel Entitas Pendaftaran

Atribut	Penjelasan
id_pendaftaran (PK)	Kode unik yang digunakan sebagai identitas utama setiap data pendaftaran pasien dalam sistem rumah sakit.
id_pasien (FK)	Kode pasien yang mengacu pada entitas Pasien dan digunakan untuk menunjukkan pasien yang melakukan pendaftaran.
id_jadwal (FK)	Kode jadwal praktik yang mengacu pada entitas Jadwal Praktik dan digunakan untuk menunjukkan jadwal yang dipilih pasien.
tgl_daftar	Menyimpan tanggal pendaftaran

Atribut	Penjelasan
	pasien yang terdiri atas tanggal, bulan, dan tahun sebagai informasi waktu dilakukannya registrasi pelayanan kesehatan.
no_antrian	Menyimpan nomor urut pelayanan pasien pada jadwal praktik tertentu yang digunakan untuk mengatur urutan pemeriksaan pasien.
status_kunjungan	Menyimpan status pelayanan pasien seperti selesai untuk menunjukkan tahapan kunjungan pasien dalam proses pelayanan kesehatan.

6. Rekam Medis

Entitas rekam medis digunakan untuk menyimpan hasil pemeriksaan dan riwayat pelayanan kesehatan pasien. Data pada entitas ini menjadi sumber informasi utama mengenai kondisi kesehatan pasien berdasarkan pemeriksaan yang dilakukan oleh dokter. Setiap rekam medis dibuat berdasarkan data kunjungan pasien yang telah terdaftar.

Tabel 3. 7 Tabel Entitas Rekam Medis

Atribut	Penjelasan
id_rekam_media (PK)	Kode unik yang digunakan sebagai identitas utama setiap rekam medis pasien dalam sistem rumah sakit.
id_pendaftaran (FK)	Kode pendaftaran yang mengacu

Atribut	Penjelasan
	pada entitas Pendaftaran dan digunakan untuk menghubungkan rekam medis dengan data kunjungan pasien.
id_dokter (FK)	Kode dokter yang mengacu pada entitas Dokter dan digunakan untuk menunjukkan dokter yang melakukan pemeriksaan pasien
keluhan	Menyimpan informasi mengenai keluhan atau gejala yang disampaikan pasien saat melakukan pemeriksaan, seperti demam, batuk, nyeri, atau keluhan kesehatan lainnya.
hasil_pemeriksaan	Menyimpan hasil pemeriksaan yang dilakukan dokter sebagai dasar dalam penentuan diagnosis dan tindakan medis yang diberikan kepada pasien.
tgl_pemeriksaan	Menyimpan tanggal pemeriksaan pasien yang terdiri atas tanggal, bulan, dan tahun sebagai informasi waktu dilakukannya pemeriksaan medis

7. Vital

Entitas vital digunakan untuk menyimpan data tanda-tanda vital pasien yang diperoleh saat pemeriksaan. Data ini digunakan sebagai informasi pendukung dalam proses diagnosis dan evaluasi kondisi kesehatan pasien. Setiap data vital terkait dengan satu rekam medis tertentu.

Tabel 3. 8 Tabel Entitas Vital

Atribut	Penjelasan
id_vital (PK)	Kode unik yang digunakan sebagai identitas utama setiap data vital pasien dalam sistem rumah sakit.
id_rekam_medis (FK)	Kode rekam medis yang mengacu pada entitas Rekam Medis dan digunakan untuk menghubungkan data vital dengan hasil pemeriksaan pasien.
tekanan_darah	Menyimpan hasil pengukuran tekanan darah pasien dalam satuan mmHg yang digunakan sebagai salah satu indikator kondisi kesehatan pasien.
berat_badan	Menyimpan berat badan pasien dalam satuan kilogram (kg) yang digunakan sebagai salah satu indikator kondisi kesehatan pasien.

8. Diagnosa

Entitas diagnosa digunakan untuk menyimpan hasil diagnosis yang ditetapkan dokter berdasarkan pemeriksaan pasien. Data diagnosa digunakan untuk mendokumentasikan kondisi kesehatan pasien dan menjadi dasar dalam penentuan tindakan medis maupun pemberian obat.

Tabel 3. 9 Tabel Entitas Diagnosa

Atribut	Penjelasan
id_diagnosa (PK)	Kode unik yang digunakan sebagai identitas utama setiap data diagnosa dalam sistem rumah sakit.
id_rekam_medis (FK)	Kode rekam medis yang mengacu pada entitas Rekam Medis dan digunakan untuk menghubungkan data diagnosa dengan hasil pemeriksaan pasien.
diagnosa	Menyimpan nama penyakit atau kondisi kesehatan yang didiagnosis oleh dokter berdasarkan hasil pemeriksaan dan evaluasi kondisi pasien.
keterangan	Menyimpan informasi tambahan yang berkaitan dengan hasil diagnosis, seperti tingkat keparahan penyakit, kondisi pasien, rekomendasi tindak lanjut, atau catatan medis lainnya dari dokter.

9. Obat

Entitas obat digunakan untuk menyimpan informasi mengenai obat yang tersedia dan digunakan dalam pelayanan kesehatan rumah sakit. Data obat diperlukan untuk mendukung proses pemberian resep serta pengelolaan informasi obat dalam sistem.

Tabel 3. 10 Tabel Entitas Obat

Atribut	Penjelasan
id_obat (PK)	Kode unik yang digunakan sebagai identitas utama setiap obat dalam sistem rumah sakit.
nama_obat	Menyimpan nama obat yang diresepkan atau diberikan kepada pasien sesuai kebutuhan pengobatan, seperti Paracetamol untuk menurunkan demam atau Amoksisilin sebagai antibiotik.
harga_obat	Menyimpan harga per satuan obat dalam satuan rupiah yang digunakan sebagai dasar perhitungan biaya obat pada proses administrasi dan pembayaran pasien.

10. Resep

Entitas resep digunakan untuk mencatat obat yang diberikan kepada pasien berdasarkan hasil pemeriksaan dan diagnosis dokter. Entitas ini menghubungkan data rekam medis dengan data obat yang diresepkan sehingga riwayat pemberian obat dapat terdokumentasi dengan baik.

Tabel 3. 11 Tabel Entitas Resep

Atribut	Penjelasan
id_resep (PK)	Kode unik yang digunakan sebagai identitas utama setiap data resep dalam sistem rumah sakit.
id_rekam_medis (FK)	Kode rekam medis yang mengacu pada entitas Rekam Medis dan digunakan untuk menghubungkan resep dengan hasil pemeriksaan pasien.
id_obat (FK)	Kode obat yang mengacu pada entitas Obat dan digunakan untuk menunjukkan obat yang diresepkan kepada pasien.
dosis	Menyimpan informasi dosis atau takaran obat yang diberikan kepada pasien sesuai hasil diagnosis dan resep dokter.
jumlah	Menyimpan banyaknya obat yang diresepkan kepada pasien untuk memenuhi kebutuhan pengobatan

Atribut	Penjelasan
	selama periode tertentu.
aturan	Menyimpan petunjuk penggunaan obat yang harus diikuti pasien sesuai anjuran dokter.

b. Kardinalitas

Tabel 3. 12 Tabel Kardinalitas

Entitas 1	Entitas 2	Kardinalitas	Keterangan
Poli	Dokter	1:N	Satu poli dapat memiliki banyak dokter, sedangkan satu dokter hanya bertugas pada satu poli.
Dokter	Jadwal Praktik	1:N	Satu dokter dapat memiliki beberapa jadwal praktik pada hari atau jam yang berbeda.
Pasien	Pendaftaran	1:N	Satu pasien dapat melakukan pendaftaran berkali-kali untuk kunjungan yang berbeda.
Jadwal Praktik	Pendaftaran	1:N	Satu jadwal praktik dapat dipilih oleh banyak pasien.
Pendaftaran	Rekam Medis	1:1	Satu kunjungan (pendaftaran) menghasilkan

Entitas 1	Entitas 2	Kardinalitas	Keterangan
			satu rekam medis.
Dokter	Rekam Medis	1:N	Satu dokter dapat membuat banyak rekam medis dari berbagai pasien.
Rekam Medis	Vital	1:1	Satu rekam medis memiliki satu data tanda vital yang dicatat saat pemeriksaan.
Rekam Medis	Diagnosa	1:N	Dalam satu pemeriksaan pasien bisa ditemukan lebih dari satu diagnosis.
Rekam Medis	Resep	1:N	Satu rekam medis dapat menghasilkan beberapa item resep obat.
Obat	Resep	1:N	Satu jenis obat dapat muncul pada banyak resep yang berbeda.

3.4. Proses Normalisasi

Normalisasi merupakan proses pengelolaan struktur tabel dalam basis data relasional agar memenuhi kriteria tertentu sehingga terhindar dari redundansi data, anomali, atau tidak konsistennya informasi. Perancangan yang baik sangat bergantung pada proses normalisasi agar data operasional yang tersimpan tetap valid saat terjadi adanya pembaruan. Pada sistem informasi rumah sakit, proses normalisasi dilakukan secara bertahap dimulai dari bentuk tidak normal (Unnormalized

Form/UNF), first normal form (1NF) kemudian second normal form (2NF) dan terakhir third normal form (3NF).

a. Unnormalized Form (UNF)

Unnormalized Form merupakan kondisi awal dimana kumpulan data mentah yang diperoleh dari seluruh aktivitas operasional rumah sakit tanpa ada aturan pengisian data tertentu. Pada tahap ini, seluruh data dari proses operasional rumah sakit digabungkan ke dalam satu tabel tanpa memperhatikan aturan relasional.

Tabel 3. 13 Tabel UNF

id_pasi en	Nama_ Pasien	Diagnosa 1	Ket_Diag nosa1	Diagnosa 2	Ket_Diag nosa 2	...	Aturan 2
PA01	Ahmad Fauzi	ISPA	—	—	—		—
PA02	Sari Wuland ari	Bronkitis Akut	Perlu antibiotik	—	—		—
PA03	Rina Wuland ari	Diabetes Mellitus Tipe 2	GDS 180 mg/dL, baru terdiagno sis	Hiperten si Grade I	TD 150/95, dikombin asi dengan DM		Malam sebelum tidur
PA04	Lilis Suryani	Suspect Appendis itis Akut	Perlu pemeriks aan penunjan g segera	—	—		—

id_pasien	Nama_Pasien	Diagnosa 1	Ket_Diagnosa 1	Diagnosa 2	Ket_Diagnosa 2	...	Aturan 2
PA05	Eko Nugroho	Karies Dentis Profunda Gigi 46	Pulpa terbuka, perlu PSA	—	—		—
PA06	Hasan Basri	Diabetes Mellitus Tipe 2	Kontrol rutin, GDA 210 mg/dL	—	—		Sesudah makan
PA07	Sri Handayani	Suspect Tuberkulosis Paru	Gejala klinis dan foto thorax mendukung	—	—		—
PA08	Dian Permata	Hipertensi Grade II	TD 160/100, perlu terapi kombinasi	—	—		Malam sesudah makan
PA09	Andi Saputro	Diabetes Mellitus Tipe 2	Terkontrol, lanjut obat rutin	Hipertensi Grade I	TD 140/90, terkontrol		Pagi sebelum makan
PA03	Rina	Diabetes	Kontrol	Hiperten	TD		Malam

id_pasien	Nama_Pasien	Diagnosa 1	Ket_Diagnosa 1	Diagnosa 2	Ket_Diagnosa 2	...	Aturan 2
	Wulandari	Mellitus Tipe 2	ulang – evaluasi terapi	si Grade I	membaik, lanjut Amlodipine		sebelum tidur
PA11	Nita Anggraeni	Rhinitis Alergi	Diduga dipicu debu rumah	–	–		–
PA12	Bambang Supriadi	Tonsilitis Akut	Infeksi bakteri ringan	–	–		Sesudah makan
PA13	Lis Setyawati	Perikoronitis Gigi 38	Perlu pembersihan area gigi	–	–		Sesudah makan
PA14	Muhammad Arif	Diabetes Mellitus Tipe 2	GDS meningkat	Hipertensi Grade I	TD sedikit meningkat		Pagi
PA15	Ratna Dewi	ISPA	Infeksi saluran napas atas	–	–		Pagi

id_pasien	Nama_Pasien	Diagnosa 1	Ket_Diagnosa 1	Diagnosa 2	Ket_Diagnosa 2	...	Aturan 2
PA16	Hendra Saputra	Anemia Ringan	Diduga kurang asupan zat besi	—	—		—
PA17	Yusuf Maulana	Lipoma	Jinak, observasi	—	—		—
PA18	Indah Permatasari	Gastritis Akut	Keluhan kambuh	—	—		Sebelum makan
PA19	Siti Nurjannah	Hipertensi Grade I	Perlu kontrol rutin	—	—		Sesudah makan
PA20	Fajar Ramadhan	ISPA	Infeksi ringan	—	—		Pagi
PA21	Desi Kartika	Asma Bronkial Ringan	Kondisi stabil	—	—		—
PA22	Rudi Hartono	Vulnus Laceratum	Perlu perawatan luka	—	—		Sesudah makan
PA23	Wulan	Diabetes	Kondisi	—	—		—

id_pasi en	Nama_ Pasien	Diagnosa 1	Ket_Diag nosa1	Diagnosa 2	Ket_Diag nosa 2	...	Aturan 2
	Kartika	Mellitus Tipe 2	membaik				
PA24	Agung Prasetyo	Influenza	Gejala ringan	–	–		Pagi
PA25	Maya Lestari	Gastritis Akut	Perlu pengatura n pola makan	–	–		Sesudah makan

Dari data mentah pada sistem informasi rumah sakit yang ada, struktur data dalam bentuk yang tidak normal, hal ini menjadi kelemahan dan ketidaksesuaian dengan prinsip basis data relasional.

Berikut adalah permasalahan utama yang dapat diidentifikasi dari data mentah dalam tahap UNF :

1. Adanya Kelompok Data Berulang (Repeating Groups)

Di dalam tabel UNF, terdapat atribut multi nilai yang diakomodasi melalui pembuatan kolom yang sama dan berulang, yaitu pada atribut Diagnosa1 dan Diagnosa2 beserta keterangannya dan atribut Obat1 dan Obat2. Kondisi ini melanggar prinsip keatomikan data dalam basis data relasional, dimana setiap perpotongan baris dan kolom seharusnya menyimpan satu nilai tunggal. Penyusunan seperti ini membatasi kapasitas penyimpanan data secara aktual serta menyulitkan proses pencarian data.

2. Redudansi Data Masif akibat Penggabungan Multi-Entitas

Seluruh data atribut yang merepresentasikan entitas-entitas independen termasuk data master yaitu Pasien, Dokter, Poli, dan Obat, serta data transaksi dipaksakan berada dalam satu tabel tunggal berukuran besar. Penggabungan ini

menyebabkan terjadinya duplikasi atau perulangan data yang besar pada setiap baris transaksi.

3. Tidak Ada Kunci Utama yang Unik

Karena adanya penggabungan data dari berbagai entitas dan adanya kelompok data berulang, tabel pada bentuk UNF tidak memiliki satupun atribut yang dapat bertindak sebagai *primary key* atau kunci utama yang valid untuk mengidentifikasi setiap baris secara unik. Dampaknya, Atribut seperti *id_pasien* tidak dapat dijalankan sebagai *primary key* tunggal karena nilainya akan terus berulang pada baris yang berbeda akibat adanya pencatatan transaksi yang jamak. Ketiadaan identitas unik yang mutlak ini membuat sistem tidak memiliki integritas entitas (*entity integrity*), sehingga menyulitkan proses relasi data dan memicu terjadinya kegagalan pembaruan (*update anomalies*) atau penghapusan data secara tidak sengaja (*delete anomalies*).

b. First Normal Form (1NF)

First Normal Form memiliki syarat bahwa setiap kolom harus bernilai atomik dan setiap baris harus dapat diidentifikasi secara unik untuk mencapai 1NF, kelompok berulang pada atribut diagnosa dan obat dipecah menjadi baris-baris terpisah sehingga setiap sel hanya memuat satu nilai tunggal. Perubahan dari UNF ke 1NF yaitu:

1. Kolom *diagnosa1*, *diagnosa2*, *obat1*, dan *obat2* dipecah menjadi satu kolom *diagnosa* dan satu kolom *obat*, dengan setiap baris mewakili satu pasangan *diagnosa* dan satu resep *obat*.
2. Data yang semula berada dalam satu barisan dengan dua *diagnosa* atau dua *obat* kini dipecah menjadi dua baris atau lebih.
3. Data tidak lagi mengandung data berulang atau *repeating groups*.

Tabel 3. 14 Tabel Normalisasi 1NF

id_pasien	Nama_Pasien	id_pendaftaran	diagnosa	keterangan	obat	aturan
PA01	Ahmad Fauzi	PE01	ISPA	–	Paracetamol 500mg	Sesudah makan

id_pasien	Nama_Pasien	id_pendaftaran	diagnosa	keterangan	obat	aturan
PA02	Sari Indah Lestari	PE02	Bronkitis Akut	Perlu antibiotik	Vitamin C 500mg	Sesudah makan pagi
PA03	Rina Wulandari	PE03	Diabetes Mellitus Tipe 2	GDS 180 mg/dL, baru terdiagnosis	Paracetamol 500mg	Sesudah makan
PA03	Rina Wulandari	PE03	Hipertensi Grade I	TD 150/95, dikombinasikan dengan DM	Hipertensi Grade I	Malam sebelum tidur
PA03	Rina Wulandari	PE10	Diabetes Mellitus Tipe 2	Kontrol ulang – evaluasi terapi	Metformin 500mg	Sesudah makan
PA03	Rina Wulandari	PE10	Hipertensi Grade I	TD membaik, lanjut Amlodipine	Amlodipine 5mg	Malam sebelum tidur
PA04	Lilis Suryani	PE04	Suspect Appendisitis Akut	Perlu pemeriksaan penunjang segera	Cetirizine 10mg	Malam sebelum tidur
PA05	Eko Nugroho	PE05	Karies Dentis	Pulpa terbuka,	Metformin 500mg	Sesudah makan

id_pasien	Nama_Pasien	id_pendaftaran	diagnosa	keterangan	obat	aturan
			Profunda Gigi 46	perlu PSA		
PA06	Hasan Basri	PE06	Diabetes Mellitus Tipe 2	Kontrol rutin, GDA 210 mg/dL	Omeprazole 20mg	Sebelum makan
PA06	Hasan Basri	PE06	-	-	Metformin 500mg	Sesudah makan
PA07	Sri Handayani	PE07	Suspect Tuberkulosis Paru	Gejala klinis dan foto thorax mendukung	Paracetamol 500mg	Sesudah makan
PA08	Dian Permata	PE08	Hipertensi Grade II	TD 160/100, perlu terapi kombinasi	Paracetamol 500mg	Sesudah makan
PA08	Dian Permata	PE08	-	-	Amlodipine 5mg	Malam sesudah makan
PA09	Andi Saputro	PE09	Diabetes Mellitus Tipe 2	Terkontrol, lanjut obat rutin	Amoxicillin 500mg	Sesudah makan
PA09	Andi Saputro	PE09	Hipertensi Grade I	TD 140/90, terkontrol	Amlodipine 5mg	Pagi sebelum makan
PA11	Nita	PE11	Rhinitis	Diduga	Vitamin C	Sesudah

id_pasien	Nama_Pasien	id_pendaftaran	diagnosa	keterangan	obat	aturan
	Anggraeni		Alergi	dipicu debu rumah	500mg	makan
PA12	Bambang Supriadi	PE12	Tonsilitis Akut	Infeksi bakteri ringan	Amoxicillin 500mg	Sesudah makan
PA12	Bambang Supriadi	PE12	—	—	Paracetamol 500mg	Sesudah makan
PA13	Lis Setyawati	PE13	Perikoronitis Gigi 38	Perlu pembersihan area gigi	Paracetamol 500mg	Sesudah makan
PA13	Lis Setyawati	PE13	—	—	Amoxicillin 500mg	Sesudah makan
PA14	Muhammad Arif	PE14	Diabetes Mellitus Tipe 2	GDS meningkat	Metformin 500mg	Sesudah makan
PA14	Muhammad Arif	PE14	Hipertensi Grade I	TD sedikit meningkat	Amlodipine 5mg	Pagi
PA15	Ratna Dewi	PE15	ISPA	Infeksi saluran napas atas	Paracetamol 500mg	Sesudah makan
PA15	Ratna Dewi	PE15	—	—	Vitamin C 500mg	Pagi
PA16	Hendra Saputra	PE16	Anemia Ringan	Diduga kurang	Vitamin C 500mg	Sesudah makan

id_pasien	Nama_Pasien	id_pendaftaran	diagnosa	keterangan	obat	aturan
				asupan zat besi		
PA17	Yusuf Maulana	PE17	Lipoma	Jinak, observasi	Paracetamol 500mg	Sesudah makan
PA18	Indah Permatasari	PE18	Gastritis Akut	Keluhan kambuh	Paracetamol 500mg	Sesudah makan
PA18	Indah Permatasari	PE18	–	–	Omeprazole 20mg	Sebelum makan
PA19	Siti Nurjanah	PE19	Hipertensi Grade I	Perlu kontrol rutin	Amlodipine 5mg	Pagi
PA19	Siti Nurjanah	PE19	–	–	Paracetamol 500mg	Sesudah makan
PA20	Fajar Ramadhan	PE20	ISPA	Infeksi ringan	Paracetamol 500mg	Sesudah makan
PA20	Fajar Ramadhan	PE20	–	–	Vitamin C 500mg	Pagi
PA21	Desi Kartika	PE21	Asma Bronkial Ringan	Kondisi stabil	Vitamin C 500mg	Sesudah makan
PA22	Rudi Hartono	PE22	Vulnus Laceratum	Perlu perawatan luka	Paracetamol 500mg	Sesudah makan
PA22	Rudi Hartono	PE22	–	–	Amoxicilli	Sesudah

id_pasien	Nama_Pasien	id_pendaftaran	diagnosa	keterangan	obat	aturan
					n 500mg	makan
PA23	Wulan Kartika	PE23	Diabetes Mellitus Tipe 2	Kondisi membaik	Metformin 500mg	Sesudah makan
PA24	Agung Prasetyo	PE24	Influenza	Gejala ringan	Paracetamol 500mg	Sesudah makan
PA24	Agung Prasetyo	PE24	–	–	Vitamin C 500mg	Pagi
PA25	Maya Lestari	PE25	Gastritis Akut	Perlu pengaturan pola makan	Omeprazole 20mg	Sebelum makan
PA25	Maya Lestari	PE25	–	–	Paracetamol 500mg	Sesudah makan

c. Second Normal Form (2NF)

2NF memiliki syarat bahwa tabel sudah memenuhi 1NF dan tidak terdapat ketergantungan parsial, yaitu setiap atribut non-kunci harus bergantung penuh pada semua primary key. Pada tahap ini tabel yang masih memiliki ketergantungan parsial dipisah menjadi tabel-tabel yang lebih kecil. Perubahan dari 1NF ke 2NF yaitu memecah tabel menjadi beberapa entitas terpisah sebagai berikut.

Tabel 3. 15 Tabel Normalisasi 2NF

Tabel	Primary Key	Atribut
Pasien	id_pasien	nama_pasien, jenis_kelamin,

		tgl_lahir, alamat, no_hp_pasien, gol_darah
Dokter	id_dokter	nama_dokter, spesialisasi, id_poli (FK)
Pendaftaran	id_pendaftaran	id_pasien (FK), id_jadwal (FK), tgl_daftar, no_antrian, status_kunjungan
Rekam_Medis	id_rekam_medis	id_pendaftaran (FK), id_dokter (FK), keluhan, hasil_pemeriksaan, tgl_pemeriksaan
Diagnosa	id_diagnosa	id_rekam_medis (FK), diagnosa, keterangan
Resep/Obat	id_resep	id_rekam_medis (FK), id_obat (FK), dosis, jumlah, aturan

d. Third Normal Form (3NF)

3NF memiliki syarat bahwa tabel sudah memenuhi 2NF dan tidak terdapat ketergantungan transitif, yaitu setiap atribut non-kunci harus bergantung langsung pada primary key. Pada tahap ini, diidentifikasi bahwa atribut nama_poli dan lokasi_poli bergantung pada id_poli, bukan pada id_dokter, sehingga menimbulkan ketergantungan transitif. Perubahan 2NF ke 3NF yaitu:

1. Tabel Dokter sebelumnya memuat id_poli, nama_poli, dan lokasi_poli. Atribut nama_poli dan lokasi_poli tidak bergantung langsung pada id_dokter melainkan pada id_poli, sehingga terjadi ketergantungan transitif: $id_dokter \rightarrow id_poli \rightarrow nama_poli, lokasi$. Atribut yang mengalami ketergantungan transitif dipisahkan dengan membentuk tabel

poli dimana id_poli sebagai primary key yang mengidentifikasi atribut nama_poli dan lokasi poli secara unik, sementara tabel dokter tetap mempertahankan atribut id_poli yang sekarang sebagai foreign key untuk menghubungkan kedua tabel.

2. Untuk menghindari penumpukan data operasional dan menghilangkan ketergantungan transitif antara data penugasan waktu dengan data personal dokter, dilakukan pemisahan entitas menjadi tabel jadwal. Tabel ini menggunakan id_jadwal sebagai Primary Key dan menyimpan id_dokter sebagai Foreign Key.
3. Data tanda-tanda klinik fisik pasien dipisahkan dari Tabel Rekam Medis menjadi Tabel Vital dengan id_vital sebagai Primary Key dan id_rekam_medis sebagai Foreign Key (FK). Pemisahan ini dilakukan karena parameter tanda vital memiliki karakteristik data klinis yang bersifat spesifik, dinamis, serta memiliki pola ketergantungan tersendiri.

Tabel 3. 16 Tabel Normalisasi 3NF

Tabel	Primary Key	Atribut	Forein Key
Pasien	id_pasien	nama_pasien, jenis_kelamin, tgl_lahir, alamat, no_hp_pasien, gol_darah	–
Poli	id_poli	nama_poli, lokasi	–
Dokter	id_dokter	nama_dokter, spesialisasi	id_poli
Jadwal_Pra ktik	id_jadwal	hari_praktik, jam_mulai, jam_selesai	id_dokter

Tabel	Primary Key	Atribut	Forein Key
Pendaftaran	id_pendaftaran	tgl_daftar, no_antrian, status_kunjungan	id_pasien, id_jadwal
Rekam_Medis	id_rekam_medis	keluhan, hasil_pemeriksaan, tgl_pemeriksaan	id_pendaftaran, id_dokter
Vital	id_vital	tekanan_darah, berat_badan	id_rekam_medis
Diagnosa	id_diagnosa	diagnosa, keterangan	id_rekam_medis
Obat	id_obat	nama_obat, harga_satuan	—
Resep	id_resep	dosis, jumlah, aturan	id_rekam_medis, id_obat

3.5. Data Dictionary Lengkap

1. Pasien

Tabel 3. 17 Tabel Dictionary Pasien

Atribut	Tipe Data	Panjang	Constraint
id_pasien	VARCHAR	10	PRIMARY KEY
nama_pasien	VARCHAR	100	NOT NULL, INDEX (idx_pasien_nama)
jenis_kelamin	CHAR	1	NOT NULL

tgl_lahir	DATE	-	NOT NULL
alamat	TEXT	-	-
no_hp_pasien	VARCHAR	15	-
gol_darah	VARCHAR	3	-

2. Poli

Tabel 3. 18 Tabel Dictionary Poli

Atribut	Tipe Data	Panjang	Constraint
id_poli	VARCHAR	10	PRIMARY KEY
nama_poli	VARCHAR	100	NOT NULL
lokasi	VARCHAR	100	NOT NULL

3. Dokter

Tabel 3. 19 Tabel Dictionary Dokter

Atribut	Tipe Data	Panjang	Constraint
id_dokter	VARCHAR	10	PRIMARY KEY
id_poli	VARCHAR	10	FOREIGN KEY REFERENCES poli (id_poli), NOT NULL
nama_dokter	VARCHAR	100	NOT NULL, INDEX (idx_dokter_nama)

Atribut	Tipe Data	Panjang	Constraint
spesialis	VARCHAR	100	NOT NULL

4. Jadwal

Tabel 3. 20 Tabel Dictionary Jadwal

Atribut	Tipe Data	Panjang	Constraint
id_jadwal	VARCHAR	10	PRIMARY KEY
id_dokter	VARCHAR	10	FOREIGN KEY REFERENCES dokter (id_dokter), NOT NULL
hari_praktik	VARCHAR	20	NOT NULL
jam_mulai	TIME	-	NOT NULL
jam_selesai	TIME	-	NOT NULL

5. Pendaftaran

Tabel 3. 21 Tabel Dictionary Pendaftaran

Atribut	Tipe Data	Panjang	Constraint
id_pendaftaran	VARCHAR	10	PRIMARY KEY
id_pasien	VARCHAR	10	FOREIGN KEY, REFERENCES pasien(id_pasien), NOT NULL
id_jadwal	VARCHAR	10	FOREIGN KEY

Atribut	Tipe Data	Panjang	Constraint
			REFERENCES jadwal(id_jadwal), NOT NULL
tgl_daftar	DATE	-	NOT NULL
no_antrian	INT	-	NOT NULL
status_kunjungan	VARCHAR	30	NOT NULL

6. Rekam Medis

Tabel 3. 22 Tabel Dictionary Rekam Medis

Atribut	Tipe Data	Panjang	Constraint
id_rekam_medis	VARCHAR	10	PRIMARY KEY
id_pendaftaran	VARCHAR	10	FOREIGN KEY REFERENCES pendaftaran(id_pendaftaran), NOT NULL
id_dokter	VARCHAR	10	FOREIGN KEY REFERENCES dokter(id_dokter), NOT NULL
keluhan	TEXT	-	-

Atribut	Tipe Data	Panjang	Constraint
hasil_pemeriksaan	TEXT	-	-
tgl_pemeriksaan	DATE	-	NOT NULL

7. Vital

Tabel 3. 23 Tabel Dictionary Vital

Atribut	Tipe Data	Panjang	Constraint
id_vital	VARCHAR	10	PRIMARY KEY
id_rekam_medis	VARCHAR	10	FOREIGN KEY REFERENCES rekam medis(id_rekam_medis), NOT NULL
tekanan_darah	VARCHAR	20	-
berat_badan	DECIMAL	5,2	-

8. Diagnosa

Tabel 3. 24 Tabel Dictionary Diganosa

Atribut	Tipe Data	Panjang	Constraint
id_diagnosa	VARCHAR	10	PRIMARY KEY
id_rekam_medis	VARCHAR	10	FOREIGN KEY REFERENCES rekam

Atribut	Tipe Data	Panjang	Constraint
			medis(id_rekam_medis), NOT NULL
diagnosa	VARCHAR	255	NOT NULL
keterangan	TEXT	-	-

9. Obat

Tabel 3. 25 Tabel Dictionary Obat

Atribut	Tipe Data	Panjang	Constraint
id_obat	VARCHAR	10	PRIMARY KEY
nama_obat	VARCHAR	100	NOT NULL, INDEX (idx_nama_obat)
harga_obat	DECIMAL	12,2	NOT NULL

10. Resep

Tabel 3. 26 Tabel Dictionary Resep

Atribut	Tipe Data	Panjang	Constraint
id_resep	VARCHAR	10	PRIMARY KEY
id_rekam_medis	VARCHAR	10	FOREIGN KEY REFERENCES rekam_medis(id_rekam_medis), NOT NULL
id_obat	VARCHAR	10	FOREIGN KEY

Atribut	Tipe Data	Panjang	Constraint
			REFERENCES obat(id_obat), NOT NULL
dosis	VARCHAR	100	-
jumlah	INT	-	-
aturan	VARCHAR	255	-

BAB IV

IMPLEMENTASI

4.1. Implementasi DDL (Data Definition Language)

Implementasi DDL dilakukan dengan menerapkan rancangan basis data ke dalam MySQL Workbench menggunakan perintah SQL. Pada tahap ini dibuat struktur tabel, constraint, dan index sesuai dengan kebutuhan sistem agar data dapat dikelola dengan baik serta mendukung proses pengolahan data yang lebih efisien.

Membuat database dengan nama `db_rumah_sakit` menggunakan perintah berikut:

```
DROP DATABASE IF EXISTS db_rumah_sakit;  
CREATE DATABASE IF NOT EXISTS db_rumah_sakit;  
USE db_rumah_sakit;
```

Sistem Informasi Rumah sakit ini terdiri dari 10 tabel utama beserta 1 tabel audit log.

4.1.1. Tabel Poli

Tabel poli menyimpan data poliklinik yang tersedia di rumah sakit, terdiri dari 3 atribut dengan `id_poli` sebagai primary key.

```
CREATE TABLE poli (  
    id_poli VARCHAR(10) PRIMARY KEY,  
    nama_poli VARCHAR(100) NOT NULL,  
    lokasi VARCHAR(100) NOT NULL  
);
```

4.1.2. Tabel Dokter

Tabel dokter menyimpan data dokter yang bertugas di rumah sakit. Atribut `id_poli` merupakan foreign key yang merujuk ke tabel poli, sehingga setiap dokter terhubung dengan poliklinik tempat ia bertugas.

```
CREATE TABLE dokter (  
    id_dokter VARCHAR(10) PRIMARY KEY,  
    id_poli VARCHAR(10) NOT NULL,
```

```

        nama_dokter VARCHAR(100) NOT NULL,
        spesialis VARCHAR(100) NOT NULL,

        CONSTRAINT fk_dokter_poli
        FOREIGN KEY (id_poli)
        REFERENCES poli(id_poli)
    );

```

4.1.3. Tabel Pasien

Tabel pasien menyimpan data identitas pasien meliputi nama, jenis kelamin, tanggal lahir, alamat, nomor telepon, dan golongan darah. Constraint NOT NULL diterapkan pada atribut-atribut esensial.

```

CREATE TABLE pasien (
    id_pasien VARCHAR(10) PRIMARY KEY,
    nama_pasien VARCHAR(100) NOT NULL,
    jenis_kelamin CHAR(1) NOT NULL,
    tgl_lahir DATE NOT NULL,
    alamat TEXT,
    no_hp_pasien VARCHAR(15),
    gol_darah VARCHAR(3)
);

```

4.1.4. Tabel Jadwal

Tabel jadwal menyimpan jadwal praktik dokter, mencakup hari dan jam praktik. Foreign key id_dokter merujuk ke tabel dokter.

```

CREATE TABLE jadwal (
    id_jadwal VARCHAR(10) PRIMARY KEY,
    id_dokter VARCHAR(10) NOT NULL,
    hari_praktik VARCHAR(20) NOT NULL,
    jam_mulai TIME NOT NULL,
    jam_selesai TIME NOT NULL,

    CONSTRAINT fk_jadwal_dokter
    FOREIGN KEY (id_dokter)

```

```
REFERENCES dokter(id_dokter)
);
```

4.1.5. Tabel Pendaftaran

Tabel pendaftaran mencatat setiap kunjungan pasien ke rumah sakit, termasuk tanggal daftar, nomor antrian, dan status kunjungan. Terdapat dua Foreign Key yang menghubungkan tabel ini dengan tabel pasien dan jadwal.

```
CREATE TABLE pendaftaran (
    id_pendaftaran VARCHAR(10) PRIMARY KEY,
    id_pasien VARCHAR(10) NOT NULL,
    id_jadwal VARCHAR(10) NOT NULL,
    tgl_daftar DATE NOT NULL,
    no_antrian INT NOT NULL,
    status_kunjungan VARCHAR(30) NOT NULL,

    CONSTRAINT fk_pendaftaran_pasien
    FOREIGN KEY (id_pasien)
    REFERENCES pasien(id_pasien),

    CONSTRAINT fk_pendaftaran_jadwal
    FOREIGN KEY (id_jadwal)
    REFERENCES jadwal(id_jadwal)
);
```

4.1.6. Tabel Rekam Medis

Tabel rekam_medis menyimpan catatan pemeriksaan medis pasien, termasuk keluhan dan hasil pemeriksaan. Terdapat dua foreign Key yang menghubungkan dengan tabel pendaftaran dan dokter.

```
CREATE TABLE rekam_medis (
    id_rekam_medis VARCHAR(10) PRIMARY KEY,
    id_pendaftaran VARCHAR(10) NOT NULL,
    id_dokter VARCHAR(10) NOT NULL,
    keluhan TEXT,
    hasil_pemeriksaan TEXT,
```

```

        tgl_pemeriksaan DATE NOT NULL,

        CONSTRAINT fk_rekam_pendaftaran
        FOREIGN KEY (id_pendaftaran)
        REFERENCES pendaftaran(id_pendaftaran),

        CONSTRAINT fk_rekam_dokter
        FOREIGN KEY (id_dokter)
        REFERENCES dokter(id_dokter)
    );

```

4.1.7. Tabel Diagnosa

Tabel diagnosa menyimpan hasil diagnosa dokter untuk setiap rekam medis. Satu rekam medis dapat memiliki lebih dari satu diagnosa.

```

CREATE TABLE diagnosa (
    id_diagnosa VARCHAR(10) PRIMARY KEY,
    id_rekam_medis VARCHAR(10) NOT NULL,
    diagnosa VARCHAR(255) NOT NULL,
    keterangan TEXT,

    CONSTRAINT fk_diagnosa_rekam
    FOREIGN KEY (id_rekam_medis)
    REFERENCES rekam_medis(id_rekam_medis)
);

```

4.1.8. Tabel Obat

Tabel obat menyimpan data obat yang tersedia di apotek rumah sakit, meliputi nama obat dan harga satuan.

```

CREATE TABLE obat (
    id_obat VARCHAR(10) PRIMARY KEY,
    nama_obat VARCHAR(100) NOT NULL,
    harga_obat DECIMAL(12,2) NOT NULL
);

```

4.1.9. Tabel Resep

Tabel resep menyimpan data resep obat yang diberikan dokter kepada pasien pada setiap rekam medis, mencakup dosis, jumlah, dan aturan pemakaian.

```
CREATE TABLE resep (  
    id_resep VARCHAR(10) PRIMARY KEY,  
    id_rekam_medis VARCHAR(10) NOT NULL,  
    id_obat VARCHAR(10) NOT NULL,  
    dosis VARCHAR(100),  
    jumlah INT,  
    aturan VARCHAR(255),  
  
    CONSTRAINT fk_resep_rekam  
    FOREIGN KEY (id_rekam_medis)  
    REFERENCES rekam_medis(id_rekam_medis),  
  
    CONSTRAINT fk_resep_obat  
    FOREIGN KEY (id_obat)  
    REFERENCES obat(id_obat)  
);
```

4.1.10. Tabel Vital

Tabel vital menyimpan data tanda-tanda vital pasien yang diukur saat pemeriksaan, yaitu tekanan darah dan berat badan.

```
CREATE TABLE vital (  
    id_vital VARCHAR(10) PRIMARY KEY,  
    id_rekam_medis VARCHAR(10) NOT NULL,  
    tekanan_darah VARCHAR(20),  
    berat_badan DECIMAL(5,2),  
  
    CONSTRAINT fk_vital_rekam  
    FOREIGN KEY (id_rekam_medis)  
    REFERENCES rekam_medis(id_rekam_medis)  
);
```

4.1.11. Tabel Audit Log

Tabel `audit_log` merupakan tabel khusus yang digunakan untuk mencatat setiap perubahan data (INSERT, UPDATE, DELETE) pada tabel pasien secara otomatis melalui trigger.

```
CREATE TABLE IF NOT EXISTS audit_log (  
    id_audit INT AUTO_INCREMENT PRIMARY KEY,  
    nama_tabel VARCHAR(50) NOT NULL,  
    aksi VARCHAR(20) NOT NULL, -- INSERT, UPDATE, DELETE  
    id_data VARCHAR(50) NOT NULL, -- ID dari data yang  
    berubah  
    data_lama TEXT,  
    data_baru TEXT,  
    username VARCHAR(100) NOT NULL,  
    waktu TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

4.1.12. Index

Untuk mengoptimalkan performa query pencarian berdasarkan nama, dibuat tiga buah index pada kolom `nama_pasien`, `nama_dokter`, dan `nama_obat`.

```
CREATE INDEX idx_pasien_nama  
ON pasien(nama_pasien);  
  
CREATE INDEX idx_dokter_nama  
ON dokter(nama_dokter);  
  
CREATE INDEX idx_obat_nama  
ON obat(nama_obat);
```

4.2. Implementasi DML (Data Manipulation Language)

Setelah struktur tabel berhasil dibuat, tahap selanjutnya adalah mengisi tabel dengan data dummy yang realistis menggunakan perintah INSERT INTO. Data yang dimasukkan mencerminkan kondisi operasional nyata sebuah rumah sakit.

4.2.1. INSERT Data Tabel poli

Tabel poli diisi dengan 4 data poliklinik yang tersedia di rumah sakit.

```
INSERT INTO poli (id_poli, nama_poli, lokasi)
VALUES
('POL01', 'Umum', 'Gedung A Lantai 1'),
('POL02', 'Penyakit Dalam', 'Gedung C Lantai 2'),
('POL03', 'Bedah', 'Gedung D Lantai 3'),
('POL04', 'Gigi dan Mulut', 'Gedung E Lantai 1');
```

Empat poliklinik berhasil dimasukkan meliputi Poli Umum, Penyakit Dalam, Bedah, dan Gigi dan Mulut. Masing-masing poli berlokasi di gedung yang berbeda untuk memudahkan navigasi pasien.

4.2.2. INSERT Data Tabel dokter

Tabel dokter diisi dengan 6 data dokter.

```
INSERT INTO dokter (id_dokter, id_poli, nama_dokter,
spesialis)
VALUES
('D01', 'POL01', 'dr. Andi Wirawan', 'Dokter Umum'),
('D02', 'POL01', 'dr. Siti Rahayu', 'Dokter Umum'),
('D03', 'POL02', 'dr. Hendra Prasetyo, Sp.PD', 'Spesialis
Penyakit Dalam'),
('D04', 'POL04', 'dr. Rizky Firmansyah, Sp.B', 'Spesialis
Bedah'),
('D05', 'POL03', 'drg. Fajar Nugroho, Sp.KG', 'Spesialis
Konservasi Gigi'),
('D06', 'POL02', 'dr. Lina Marlina, Sp.PD', 'Spesialis
Penyakit Dalam');
```

Enam dokter berhasil dimasukkan. Poli Umum (POL01) memiliki 2 dokter umum, Poli Penyakit Dalam (POL02) memiliki 2 spesialis penyakit dalam, serta Poli Bedah (POL03) dan Poli Gigi (POL04) masing-masing memiliki 1 dokter spesialis.

4.2.3. INSERT Data Tabel pasien

Tabel pasien diisi dengan 24 data pasien yang mencerminkan keberagaman demografis. Data mencakup nama, jenis kelamin, tanggal lahir, alamat di wilayah Banyumas dan sekitarnya, nomor telepon, serta golongan darah.

```
INSERT INTO pasien (id_pasien, nama_pasien, jenis_kelamin,
tgl_lahir, alamat, no_hp_pasien, gol_darah)
VALUES
('PA01', 'Ahmad Fauzi', 'L', '1990-01-01', 'Jl. Merdeka No.
12, Purwokerto', '82100000001', 'A'),
('PA02', 'Sari Indah Lestari', 'P', '1985-03-02', 'Jl.
Sudirman No. 5, Purwokerto', '82100000002', 'B'),
('PA03', 'Rina Wulandari', 'P', '1978-07-04', 'Jl. Ahmad
Yani No. 7, Cilacap', '82100000004', 'AB'),
('PA04', 'Lilis Suryani', 'P', '1980-11-06', 'Jl. Pemuda No.
19, Purwokerto', '82100000006', 'B'),
('PA05', 'Eko Nugroho', 'L', '1988-01-13', 'Jl. Veteran No.
23, Banyumas', '82100000007', 'O'),
('PA06', 'Hasan Basri', 'L', '1975-03-17', 'Jl. Pahlawan No.
15, Cilacap', '82100000009', 'A'),
('PA07', 'Sri Handayani', 'P', '1970-08-27', 'Jl. Kalimantan
No. 21, Cilacap', '82100000014', 'O'),
('PA08', 'Dian Permata', 'P', '1986-06-01', 'Jl. Cempaka No.
22, Purwokerto', '82100000019', 'B'),
('PA09', 'Andi Saputro', 'L', '1990-12-04', 'Jl. Elang No.
8, Purwokerto', '82100000022', 'B'),
('PA11', 'Nita Anggraeni', 'P', '1984-06-14', 'Jl. Flamingo
No. 3, Banjarnegara', '82100000023', 'A'),
('PA12', 'Bambang Supriadi', 'L', '1992-08-16', 'Jl. Garuda
```

```

No. 12, Purbalingga', '82100000024', 'B'),
('PA13', 'Lis Setyawati', 'P', '1972-10-18', 'Jl. Hasanudin
No. 9, Banyumas', '82100000025', 'O'),
('PA14', 'Muhammad Arif', 'L', '1976-05-03', 'Jl. Anggrek
No. 4, Purwokerto', '82100000026', 'B'),
('PA15', 'Ratna Dewi', 'P', '1988-07-22', 'Jl. Melati No.
10, Cilacap', '82100000027', 'A'),
('PA16', 'Hendra Saputra', 'L', '1989-07-30', 'Jl. Kamboja
No. 7, Banyumas', '82100000028', 'O'),
('PA17', 'Yusuf Maulana', 'L', '1963-07-03', 'Jl. Cemara No.
15, Purwokerto', '82100000029', 'B'),
('PA18', 'Indah Permatasari', 'P', '1986-05-03', 'Jl.
Teratai No. 5, Purbalingga', '82100000030', 'AB'),
('PA19', 'Siti Nurjanah', 'P', '1987-12-15', 'Jl. Rajawali
No. 8, Cilacap', '82100000031', 'A'),
('PA20', 'Fajar Ramadhan', 'L', '1993-01-27', 'Jl. Kenari
No. 11, Banyumas', '82100000032', 'B'),
('PA21', 'Desi Kartika', 'P', '1994-09-03', 'Jl. Mangga No.
18, Purwokerto', '82100000033', 'O'),
('PA22', 'Rudi Hartono', 'L', '1996-07-29', 'Jl. Merpati No.
6, Banjarnegara', '82100000034', 'A'),
('PA23', 'Wulan Kartika', 'P', '1990-03-19', 'Jl. Dahlia No.
14, Cilacap', '82100000035', 'B'),
('PA24', 'Agung Prasetyo', 'L', '1985-02-27', 'Jl. Flamboyan
No. 9, Banyumas', '82100000036', 'O'),
('PA25', 'Maya Lestari', 'P', '1992-04-14', 'Jl. Sakura No.
7, Purwokerto', '82100000037', 'A');

```

Total 24 data pasien berhasil dimasukkan. Data berisi jenis kelamin (laki-laki dan perempuan), empat golongan darah (A, B, AB, O), rentang usia dari 30 hingga 63 tahun, serta asal wilayah yang mencakup Purwokerto, Cilacap, Banyumas, Purbalingga, dan Banjarnegara.

4.2.4. INSERT Data Tabel jadwal

Tabel jadwal diisi dengan 25 data jadwal praktik dokter. Satu dokter dapat memiliki beberapa sesi jadwal pada hari yang sama maupun berbeda.

```

INSERT INTO jadwal (id_jadwal, id_dokter, hari_praktik,
jam_mulai, jam_selesai)
VALUES
('JAD01', 'D01', 'Senin', '08:00:00', '12:00:00'),
('JAD02', 'D02', 'Selasa', '08:00:00', '12:00:00'),
('JAD03', 'D04', 'Rabu', '07:00:00', '12:00:00'),
('JAD04', 'D03', 'Senin', '10:00:00', '15:00:00'),
('JAD05', 'D05', 'Selasa', '08:00:00', '12:00:00'),
('JAD06', 'D02', 'Kamis', '08:00:00', '12:00:00'),
('JAD07', 'D03', 'Senin', '10:00:00', '15:00:00'),
('JAD08', 'D06', 'Selasa', '08:00:00', '13:00:00'),
('JAD09', 'D03', 'Senin', '10:00:00', '15:00:00'),
('JAD10', 'D03', 'Senin', '10:00:00', '15:00:00'),
('JAD11', 'D02', 'Rabu', '11:00:00', '12:00:00'),
('JAD12', 'D01', 'Kamis', '10:00:00', '12:00:00'),
('JAD13', 'D05', 'Selasa', '13:00:00', '14:00:00'),
('JAD14', 'D03', 'Senin', '08:00:00', '12:00:00'),
('JAD15', 'D03', 'Rabu', '07:00:00', '09:00:00'),
('JAD16', 'D01', 'Kamis', '08:00:00', '11:00:00'),
('JAD17', 'D04', 'Rabu', '09:00:00', '10:00:00'),
('JAD18', 'D06', 'Selasa', '07:00:00', '10:00:00'),
('JAD19', 'D03', 'Senin', '09:00:00', '11:00:00'),
('JAD20', 'D01', 'Kamis', '08:00:00', '13:00:00'),
('JAD21', 'D02', 'Selasa', '09:00:00', '11:00:00'),
('JAD22', 'D04', 'Rabu', '08:00:00', '12:00:00'),
('JAD23', 'D03', 'Senin', '09:00:00', '11:00:00'),
('JAD24', 'D01', 'Kamis', '08:00:00', '12:00:00'),
('JAD25', 'D06', 'Selasa', '09:00:00', '14:00:00');

```

Total 25 jadwal praktik berhasil dimasukkan mencakup hari Senin hingga Kamis dengan variasi jam mulai antara pukul 07.00 hingga 13.00. Jadwal terbanyak dimiliki oleh dr. Hendra Prasetyo, Sp.PD (D03) dengan 8 sesi praktik, mencerminkan tingginya permintaan layanan poli penyakit dalam.

4.2.5. INSERT Data Tabel pendaftaran

Tabel pendaftaran diisi dengan 25 data kunjungan pasien yang seluruhnya berlangsung pada bulan Januari 2026. Setiap baris mencatat pasien, jadwal yang dipilih, tanggal daftar, nomor antrian, dan status kunjungan.

```
INSERT INTO pendaftaran (id_pendaftaran, id_pasien,
id_jadwal, tgl_daftar, no_antrian, status_kunjungan)
VALUES
('PE01', 'PA01', 'JAD01', '2026-01-06', 1, 'Selesai'),
('PE02', 'PA02', 'JAD02', '2026-01-07', 1, 'Selesai'),
('PE03', 'PA03', 'JAD03', '2026-01-05', 1, 'Selesai'),
('PE04', 'PA04', 'JAD04', '2026-01-07', 1, 'Selesai'),
('PE05', 'PA05', 'JAD05', '2026-01-06', 1, 'Selesai'),
('PE06', 'PA06', 'JAD06', '2026-01-08', 1, 'Selesai'),
('PE07', 'PA07', 'JAD07', '2026-01-12', 2, 'Selesai'),
('PE08', 'PA08', 'JAD08', '2026-01-13', 3, 'Selesai'),
('PE09', 'PA09', 'JAD09', '2026-01-19', 3, 'Selesai'),
('PE10', 'PA03', 'JAD10', '2026-01-05', 1, 'Selesai'),
('PE11', 'PA11', 'JAD11', '2026-01-20', 3, 'Selesai'),
('PE12', 'PA12', 'JAD12', '2026-01-20', 2, 'Selesai'),
('PE13', 'PA13', 'JAD13', '2026-01-20', 2, 'Selesai'),
('PE14', 'PA14', 'JAD14', '2026-01-21', 1, 'Selesai'),
('PE15', 'PA15', 'JAD15', '2026-01-21', 2, 'Selesai'),
('PE16', 'PA16', 'JAD16', '2026-01-22', 1, 'Selesai'),
('PE17', 'PA17', 'JAD17', '2026-01-22', 2, 'Selesai'),
('PE18', 'PA18', 'JAD18', '2026-01-23', 1, 'Selesai'),
('PE19', 'PA19', 'JAD19', '2026-01-23', 2, 'Selesai'),
('PE20', 'PA20', 'JAD20', '2026-01-24', 1, 'Selesai'),
('PE21', 'PA21', 'JAD21', '2026-01-24', 2, 'Selesai'),
('PE22', 'PA22', 'JAD22', '2026-01-25', 1, 'Selesai'),
('PE23', 'PA23', 'JAD23', '2026-01-25', 2, 'Selesai'),
('PE24', 'PA24', 'JAD24', '2026-01-26', 1, 'Selesai'),
('PE25', 'PA25', 'JAD25', '2026-01-26', 2, 'Selesai');
```

Total 25 data pendaftaran berhasil dimasukkan dengan rentang tanggal 5 Januari hingga 26 Januari 2026. Seluruh status kunjungan berstatus 'Selesai'. Perlu diperhatikan bahwa PE03 dan PE10 sama-sama mencatat PA03 (Rina

Wulandari), yang menunjukkan adanya pasien yang melakukan kunjungan lebih dari satu kali.

4.2.6. INSERT Data Tabel `rekam_medis`

Tabel `rekam_medis` diisi dengan 24 data catatan pemeriksaan. Setiap rekam medis mencatat keluhan pasien secara deskriptif, hasil pemeriksaan klinis oleh dokter, dan tanggal pemeriksaan.

```
INSERT INTO rekam_medis (id_rekam_medis, id_pendaftaran,
id_dokter, keluhan, hasil_pemeriksaan, tgl_pemeriksaan)
VALUES
('RM01', 'PE01', 'D01', 'Demam tinggi sejak 3 hari, sakit
kepala', 'Suhu 38.9°C, tenggorokan merah', '2026-01-06'),
('RM02', 'PE02', 'D02', 'Batuk berdahak 2 minggu, sesak
ringan', 'Ronki paru kiri, suhu 37.5°C', '2026-01-07'),
('RM04', 'PE03', 'D03', 'Nyeri dada, mudah lelah, sering
pusing', 'TD 150/95, GDS 180 mg/dL', '2026-01-05'),
('RM06', 'PE04', 'D04', 'Nyeri perut kanan bawah, mual, demam
ringan', 'Nyeri tekan McBurney positif, leukositosis', '2026-
01-07'),
('RM07', 'PE05', 'D05', 'Gigi berlubang bawah kanan, nyeri
berdenyut', 'Karies profunda gigi 46, pulpa terbuka', '2026-
01-06'),
('RM09', 'PE06', 'D02', 'Kontrol gula darah, obat habis',
'GDA 210 mg/dL, TD normal, kontrol DM', '2026-01-08'),
('RM14', 'PE07', 'D03', 'Batuk kronis 3 minggu, keringat
malam, turun BB', 'Suhu subfebris, ronki halus puncak paru',
'2026-01-12'),
('RM19', 'PE08', 'D06', 'Sering pusing, tegang di leher,
sulit tidur', 'TD 160/100, diagnosis hipertensi grade II',
'2026-01-13'),
('RM22', 'PE09', 'D03', 'Kontrol rutin DM dan hipertensi,
obat hampir habis', 'GDA 185 mg/dL, TD 140/90', '2026-01-
19'),
('RM23', 'PE11', 'D02', 'Pilek berulang sejak sebulan,
sering bersin pagi', 'Mukosa hidung edema, sekret jernih',
```

```

'2026-01-20'),
('RM24', 'PE12', 'D01', 'Sakit tenggorokan dan demam',
'Tonsil membesar dan hiperemis', '2026-01-20'),
('RM25', 'PE13', 'D05', 'Sakit gigi geraham bawah, gusi
bengkak', 'Perikoronitis gigi 38', '2026-01-20'),
('RM26', 'PE14', 'D03', 'Sering haus dan sering BAK', 'GDS
220 mg/dL', '2026-01-21'),
('RM27', 'PE15', 'D03', 'Batuk dan pilek sejak 5 hari',
'Faring hiperemis', '2026-01-21'),
('RM28', 'PE16', 'D01', 'Pusing dan lemas', 'Hb sedikit
rendah', '2026-01-22'),
('RM29', 'PE17', 'D04', 'Benjolan kecil di punggung',
'Lipoma ukuran 2 cm', '2026-01-22'),
('RM30', 'PE18', 'D06', 'Nyeri ulu hati', 'Mukosa lambung
iritasi', '2026-01-23'),
('RM31', 'PE19', 'D03', 'Sakit kepala dan tengkuk tegang',
'TD 155/95', '2026-01-23'),
('RM32', 'PE20', 'D01', 'Demam dan batuk kering', 'Suhu
38°C', '2026-01-24'),
('RM33', 'PE21', 'D02', 'Sesak ringan dan batuk malam',
'Hiperreaktivitas bronkus', '2026-01-24'),
('RM34', 'PE22', 'D04', 'Luka robek di tangan', 'Luka bersih
3 cm', '2026-01-25'),
('RM35', 'PE23', 'D03', 'Kontrol diabetes', 'GDS 170 mg/dL',
'2026-01-25'),
('RM36', 'PE24', 'D01', 'Flu dan nyeri otot', 'Suhu 37.8°C',
'2026-01-26'),
('RM37', 'PE25', 'D06', 'Nyeri lambung dan mual',
'Tenderness epigastrium', '2026-01-26');

```

Total 24 rekam medis berhasil dimasukkan. Keluhan bervariasi mulai dari penyakit akut ringan (demam, batuk, ISPA) hingga kondisi kronis (diabetes mellitus, hipertensi), serta kasus bedah (suspect appendisitis) dan gigi. Data keluhan dan hasil pemeriksaan dibuat realistis sesuai terminologi medis yang digunakan di fasilitas kesehatan.

4.2.7. INSERT Data Tabel diagnosa

Tabel diagnosa diisi dengan 28 data hasil diagnosa dokter. Satu rekam medis dapat memiliki lebih dari satu diagnosa — misalnya RM04 memiliki tiga diagnosa sekaligus (Diabetes Mellitus Tipe 2, Hipertensi Grade I, dan evaluasi terapi).

```
INSERT INTO diagnosa (id_diagnosa, id_rekam_medis, diagnosa,
keterangan)
VALUES
('DI01', 'RM01', 'ISPA', '-'),
('DI02', 'RM02', 'Bronkitis Akut', 'Perlu antibiotik'),
('DI03', 'RM04', 'Diabetes Mellitus Tipe 2', 'GDS 180 mg/dL,
baru terdiagnosis'),
('DI05', 'RM06', 'Suspect Appendisitis Akut', 'Perlu
pemeriksaan penunjang segera'),
('DI06', 'RM07', 'Karies Dentis Profunda Gigi 46', 'Pulpa
terbuka, perlu PSA'),
('DI07', 'RM09', 'Diabetes Mellitus Tipe 2', 'Kontrol rutin,
GDA 210 mg/dL'),
('DI09', 'RM14', 'Suspect Tuberkulosis Paru', 'Gejala klinis
dan foto thorax mendukung'),
('DI10', 'RM19', 'Hipertensi Grade II', 'TD 160/100, perlu
terapi kombinasi'),
('DI12', 'RM04', 'Diabetes Mellitus Tipe 2', 'Kontrol ulang
- evaluasi terapi'),
('DI13', 'RM04', 'Hipertensi Grade I', 'TD membaik, lanjut
Amlodipine'),
('DI14', 'RM23', 'Rhinitis Alergi', 'Diduga dipicu debu
rumah'),
('DI15', 'RM24', 'Tonsilitis Akut', 'Infeksi bakteri
ringan'),
('DI17', 'RM25', 'Perikoronitis Gigi 38', 'Perlu pembersihan
area gigi'),
('DI19', 'RM26', 'Diabetes Mellitus Tipe 2', 'GDS
meningkat'),
('DI20', 'RM26', 'Hipertensi Grade I', 'TD sedikit
```

```

meningkat'),
('DI21', 'RM27', 'ISPA', 'Infeksi saluran napas atas'),
('DI23', 'RM28', 'Anemia Ringan', 'Diduga kurang asupan zat
besi'),
('DI24', 'RM29', 'Lipoma', 'Jinak, observasi'),
('DI25', 'RM30', 'Gastritis Akut', 'Keluhan kambuh'),
('DI27', 'RM31', 'Hipertensi Grade I', 'Perlu kontrol
rutin'),
('DI29', 'RM32', 'ISPA', 'Infeksi ringan'),
('DI31', 'RM33', 'Asma Bronkial Ringan', 'Kondisi stabil'),
('DI32', 'RM34', 'Vulnus Laceratum', 'Perlu perawatan
luka'),
('DI34', 'RM35', 'Diabetes Mellitus Tipe 2', 'Kondisi
membaik'),
('DI35', 'RM36', 'Influenza', 'Gejala ringan'),
('DI37', 'RM37', 'Gastritis Akut', 'Perlu pengaturan pola
makan'),
('DI39', 'RM22', 'Diabetes Mellitus Tipe 2', 'Terkontrol,
lanjut obat rutin'),
('DI40', 'RM22', 'Hipertensi Grade I', 'TD 140/90,
terkontrol');

```

Total 28 diagnosa berhasil dimasukkan untuk 24 rekam medis. Diagnosa yang paling sering muncul adalah Diabetes Mellitus Tipe 2, ISPA, dan Hipertensi Grade I, mencerminkan pola penyakit yang umum ditemukan di fasilitas kesehatan primer Indonesia.

4.2.8. INSERT Data Tabel obat

Tabel obat diisi dengan 7 jenis obat yang umum digunakan di rumah sakit.

```

INSERT INTO obat (id_obat, nama_obat, harga_obat)
VALUES
('OB01', 'Paracetamol 500mg', 2000),
('OB02', 'Vitamin C 500mg', 1000),
('OB03', 'Cetirizine 10mg', 4000),
('OB05', 'Omeprazole 20mg', 7500),

```

```

('OB06', 'Amlodipine 5mg', 5500),
('OB07', 'Amoxicillin 500mg', 5000),
('OB08', 'Metformin 500mg', 3000);

```

Tujuh jenis obat berhasil dimasukkan dengan rentang harga Rp1.000 hingga Rp7.500 per satuan. Obat yang tersedia mencakup analgesik (Paracetamol), vitamin (Vitamin C), antihistamin (Cetirizine), antasid/PPI (Omeprazole), antihipertensi (Amlodipine), antibiotik (Amoxicillin), dan antidiabetik (Metformin).

4.2.9. INSERT Data Tabel resep

Tabel resep diisi dengan 40 data resep obat. Satu rekam medis dapat memiliki lebih dari satu resep apabila dokter meresepkan beberapa jenis obat sekaligus.

```

INSERT INTO resep (id_resep, id_rekam_medis, id_obat, dosis,
jumlah, aturan)
VALUES
('RES01', 'RM01', 'OB01', '500mg 3x sehari', 10, 'Sesudah
makan'),
('RES02', 'RM02', 'OB02', '500mg 1x sehari', 7, 'Sesudah
makan pagi'),
('RES03', 'RM04', 'OB01', '500mg 3x sehari', 15, 'Sesudah
makan'),
('RES04', 'RM04', 'OB01', '5mg 1x sehari malam', 5, 'Malam
sebelum tidur'),
('RES05', 'RM06', 'OB03', '5mg 1x sehari malam', 5, 'Malam
sebelum tidur'),
('RES06', 'RM07', 'OB08', '500mg 3x sehari', 30, 'Sesudah
makan'),
('RES07', 'RM09', 'OB05', '20mg 2x sehari', 14, 'Sebelum
makan'),
('RES08', 'RM09', 'OB08', '500mg 3x sehari', 15, 'Sesudah
makan'),
('RES09', 'RM14', 'OB01', '500mg 3x sehari', 10, 'Sesudah
makan'),

```

```

('RES10', 'RM19', 'OB01', '500mg 3x sehari', 15, 'Sesudah
makan'),
('RES11', 'RM19', 'OB06', '10mg 1x sehari malam', 10, 'Malam
sesudah makan'),
('RES12', 'RM04', 'OB08', '500mg 3x sehari', 30, 'Sesudah
makan'),
('RES13', 'RM04', 'OB06', '5mg 1x sehari malam', 10, 'Malam
sebelum tidur'),
('RES14', 'RM23', 'OB02', '500mg 1x sehari', 10, 'Sesudah
makan'),
('RES15', 'RM24', 'OB07', '500mg 3x sehari', 15, 'Sesudah
makan'),
('RES16', 'RM24', 'OB01', '500mg 3x sehari', 10, 'Sesudah
makan'),
('RES17', 'RM25', 'OB01', '500mg 3x sehari', 10, 'Sesudah
makan'),
('RES18', 'RM25', 'OB07', '500mg 3x sehari', 10, 'Sesudah
makan'),
('RES19', 'RM26', 'OB08', '500mg 3x sehari', 30, 'Sesudah
makan'),
('RES20', 'RM26', 'OB06', '5mg 1x sehari', 30, 'Pagi'),
('RES21', 'RM27', 'OB01', '500mg 3x sehari', 10, 'Sesudah
makan'),
('RES22', 'RM27', 'OB02', '500mg 1x sehari', 10, 'Pagi'),
('RES23', 'RM28', 'OB02', '500mg 1x sehari', 14, 'Sesudah
makan'),
('RES24', 'RM29', 'OB01', '500mg 3x sehari', 10, 'Sesudah
makan'),
('RES25', 'RM30', 'OB01', '500mg 3x sehari', 10, 'Sesudah
makan'),
('RES26', 'RM30', 'OB05', '20mg 2x sehari', 14, 'Sebelum
makan'),
('RES27', 'RM31', 'OB06', '5mg 1x sehari', 30, 'Pagi'),
('RES28', 'RM31', 'OB01', '500mg 3x sehari', 10, 'Sesudah
makan'),
('RES29', 'RM32', 'OB01', '500mg 3x sehari', 10, 'Sesudah

```

```

makan'),
('RES30', 'RM32', 'OB02', '500mg 1x sehari', 10, 'Pagi'),
('RES31', 'RM33', 'OB02', '500mg 1x sehari', 10, 'Sesudah
makan'),
('RES32', 'RM34', 'OB01', '500mg 3x sehari', 10, 'Sesudah
makan'),
('RES33', 'RM34', 'OB07', '500mg 3x sehari', 10, 'Sesudah
makan'),
('RES34', 'RM35', 'OB08', '500mg 3x sehari', 30, 'Sesudah
makan'),
('RES35', 'RM36', 'OB01', '500mg 3x sehari', 10, 'Sesudah
makan'),
('RES36', 'RM36', 'OB02', '500mg 1x sehari', 10, 'Pagi'),
('RES37', 'RM37', 'OB05', '20mg 2x sehari', 14, 'Sebelum
makan'),
('RES38', 'RM37', 'OB01', '500mg 3x sehari', 10, 'Sesudah
makan'),
('RES39', 'RM22', 'OB07', '500mg 3x sehari', 15, 'Sesudah
makan'),
('RES40', 'RM22', 'OB06', '5mg 1x sehari', 30, 'Pagi sebelum
makan');

```

Total 40 resep berhasil dimasukkan. Paracetamol 500mg (OB01) merupakan obat yang paling banyak diresepkan karena digunakan untuk berbagai kondisi. Metformin 500mg (OB08) banyak diresepkan untuk pasien diabetes, sedangkan Amlodipine 5mg (OB06) untuk pasien hipertensi. Aturan pakai bervariasi: sesudah makan, sebelum makan, dan malam hari sesuai farmakokinetik masing-masing obat.

4.2.10. INSERT Data Tabel vital

Tabel vital diisi dengan 23 data tanda-tanda vital yang diukur saat pemeriksaan. Tidak semua rekam medis memiliki data vital, RM06 (suspect appendisitis) tidak memiliki data vital karena pasien langsung dirujuk untuk tindakan.

```

INSERT INTO vital (id_vital, id_rekam_medis, tekanan_darah,
berat_badan)
VALUES
('VIT01', 'RM01', '120/80', 70.0),
('VIT02', 'RM02', '118/78', 58.5),
('VIT03', 'RM04', '150/95', 82.0),
('VIT05', 'RM07', '115/75', 68.0),
('VIT06', 'RM09', '125/82', 79.0),
('VIT07', 'RM14', '118/76', 52.0),
('VIT08', 'RM19', '160/100', 64.0),
('VIT09', 'RM22', '140/90', 83.0),
('VIT10', 'RM23', '120/80', 56.0),
('VIT11', 'RM24', '118/78', 72.0),
('VIT12', 'RM25', '130/80', 60.0),
('VIT13', 'RM26', '145/90', 85.0),
('VIT14', 'RM27', '120/80', 58.0),
('VIT15', 'RM28', '118/76', 68.0),
('VIT16', 'RM29', '120/80', 75.0),
('VIT17', 'RM30', '120/80', 61.0),
('VIT18', 'RM31', '155/95', 67.0),
('VIT19', 'RM32', '120/80', 70.0),
('VIT20', 'RM33', '120/78', 59.0),
('VIT21', 'RM34', '120/80', 78.0),
('VIT22', 'RM35', '135/85', 73.0),
('VIT23', 'RM36', '118/78', 76.0),
('VIT24', 'RM37', '120/80', 57.0);

```

Total 23 data vital berhasil dimasukkan. Tekanan darah tertinggi tercatat pada RM19 (160/100 mmHg — hipertensi grade II) dan RM18/RM31 (155/95 mmHg). Berat badan pasien bervariasi antara 52 kg (RM14) hingga 85 kg (RM13), dengan mayoritas berada pada kisaran berat badan normal hingga overweight.

4.3. Implementasi Query SELECT

Disajikan seluruh query SELECT yang dibuat untuk mengekstrak informasi dari db_rumah_sakit.

4.3.1. Query Sederhana

Query 1: Menampilkan Seluruh Data Pasien

Studi Kasus: Petugas bagian administrasi pendaftaran perlu melihat daftar lengkap seluruh pasien yang sudah terdaftar di sistem, misalnya untuk keperluan verifikasi data atau pembuatan laporan. Daripada membuka tabel satu per satu, dibuatlah query yang langsung menampilkan semua data pasien sekaligus.

```
SELECT *  
FROM pasien;
```

Query menghasilkan 24 baris data yang memuat seluruh atribut pada tabel pasien, yaitu id_pasien, nama_pasien, jenis_kelamin, tgl_lahir, alamat, no_hp_pasien, dan gol_darah. Output ini dapat digunakan langsung oleh petugas administrasi untuk kebutuhan pengecekan maupun rekap data pasien.

Query 2: Menampilkan Daftar Dokter Spesialis Penyakit Dalam

Studi Kasus: Seorang pasien datang ke loket pendaftaran dan ingin berobat ke poli penyakit dalam, namun ia tidak tahu siapa saja dokter yang bertugas di sana. Petugas membutuhkan cara cepat untuk menampilkan daftar dokter berdasarkan spesialisasi tertentu, sehingga pasien bisa memilih dokter yang diinginkan sebelum mendaftar.

```
SELECT id_dokter, nama_dokter  
FROM dokter  
WHERE spesialis LIKE '%Penyakit Dalam%';
```

Query menggunakan operator LIKE dengan wildcard % agar pencarian bersifat fleksibel dan dapat menangkap variasi penulisan nama spesialisasi. Hasilnya menampilkan 2 dokter, yaitu dr. Hendra Prasetyo, Sp.PD (D03) dan dr. Lina Marlina, Sp.PD (D06), yang keduanya bertugas di Poli Penyakit Dalam (POL02).

Query 3: Menampilkan Obat dengan Harga di Atas Rp4000

Studi Kasus: Petugas apotek rumah sakit ingin mengetahui daftar obat yang termasuk dalam kategori harga menengah ke atas, misalnya untuk keperluan evaluasi harga atau penentuan subsidi obat bagi pasien tertentu. Dengan query ini, petugas dapat langsung melihat obat-obat yang harganya melebihi batas tertentu tanpa harus memeriksa seluruh daftar obat secara manual.

```
SELECT nama_obat, harga_obat
FROM obat
WHERE harga_obat > 4000;
```

Query menggunakan operator > untuk menyaring obat dengan harga satuan di atas Rp4.000. Hasilnya menampilkan 3 obat, yaitu Omeprazole 20mg (Rp7.500), Amlodipine 5mg (Rp5.500), dan Amoxicillin 500mg (Rp5.000). Cetirizine 10mg tidak masuk dalam hasil karena harganya tepat Rp4.000, bukan di atas Rp4.000.

4.3.2. Query dengan JOIN

Query dengan JOIN digunakan ketika informasi yang dibutuhkan tersebar di beberapa tabel yang saling berelasi. Hampir semua informasi penting memerlukan penggabungan data dari lebih dari satu tabel karena struktur basis datanya yang dinormalisasi.

Query 4: Data Pendaftaran Pasien Beserta Dokter yang Menangani

Studi Kasus: Kepala bagian pelayanan ingin memantau aktivitas pendaftaran harian dan ingin tahu pasien mana saja yang sudah mendaftar beserta dokter yang akan menangani mereka. Informasi ini tidak bisa didapat dari satu tabel saja karena nama pasien ada di tabel pasien, nama dokter ada di tabel dokter, dan hubungannya melalui tabel jadwal dan pendaftaran.

```
SELECT
p.nama_pasien,
d.nama_dokter,
```

```

pd.tgl_daftar,
pd.no_antrian
FROM pendaftaran pd
JOIN pasien p
ON pd.id_pasien = p.id_pasien
JOIN jadwal j
ON pd.id_jadwal = j.id_jadwal
JOIN dokter d
ON j.id_dokter = d.id_dokter;

```

Query menggabungkan 4 tabel sekaligus menggunakan INNER JOIN berantai. Tabel pendaftaran menjadi titik tengah yang menghubungkan data pasien (melalui id_pasien) dan data jadwal (melalui id_jadwal), yang kemudian diteruskan ke tabel dokter. Query menghasilkan 25 baris data yang memudahkan petugas memantau seluruh antrian dalam satu tampilan.

Query 5: Riwayat Pemeriksaan Pasien

Studi Kasus: Seorang dokter akan memeriksa pasien yang pernah berobat sebelumnya dan ingin mengetahui riwayat keluhan serta hasil pemeriksaan terdahulu sebelum melakukan konsultasi. Dengan adanya query ini, dokter atau petugas medis dapat dengan cepat menelusuri catatan pemeriksaan pasien tanpa harus mencari-cari secara manual di banyak tabel.

```

SELECT
p.nama_pasien,
d.nama_dokter,
rm.keluhan,
rm.hasil_pemeriksaan,
rm.tgl_pemeriksaan
FROM rekam_medis rm
JOIN pendaftaran pd
ON rm.id_pendaftaran = pd.id_pendaftaran
JOIN pasien p
ON pd.id_pasien = p.id_pasien

```

```
JOIN dokter d
ON rm.id_dokter = d.id_dokter;
```

Query bertolak dari tabel `rekam_medis` sebagai tabel utama, kemudian menghubungkannya ke tabel pendaftaran untuk mendapatkan `id_pasien`, lalu ke tabel pasien untuk nama pasien, dan langsung ke tabel dokter untuk nama dokter yang memeriksa. Hasil query berupa 24 baris riwayat pemeriksaan yang menampilkan seluruh catatan medis pasien secara kronologis.

Query 6: Resep Obat yang Diberikan kepada Pasien

Studi Kasus: Petugas apotek menerima resep dari dokter dan perlu memverifikasi kesesuaian nama obat, dosis, dan jumlah yang tertera dengan data di sistem sebelum menyerahkan obat kepada pasien. Query ini membantu apotek menelusuri resep berdasarkan nama pasien, sehingga proses verifikasi lebih cepat dan mengurangi risiko kesalahan pemberian obat.

```
SELECT
p.nama_pasien,
o.nama_obat,
r.jumlah,
r.dosis
FROM resep r
JOIN obat o
ON r.id_obat = o.id_obat
JOIN rekam_medis rm
ON r.id_rekam_medis = rm.id_rekam_medis
JOIN pendaftaran pd
ON rm.id_pendaftaran = pd.id_pendaftaran
JOIN pasien p
ON pd.id_pasien = p.id_pasien;
```

Query menelusuri data melalui 5 tabel secara berurutan dari resep ke obat, kemudian naik ke `rekam_medis`, pendaftaran, dan terakhir pasien. Jalur JOIN yang panjang ini diperlukan karena tabel resep tidak menyimpan

id_pasien secara langsung, informasi tersebut hanya bisa didapat dengan menelusuri relasi antar tabel. Hasilnya berupa 40 baris data resep lengkap beserta nama pasien penerimanya.

Query 7: Jadwal Praktik Dokter per Poli

Studi Kasus: Pasien atau keluarga pasien menghubungi rumah sakit untuk menanyakan jadwal praktik dokter. Resepsionis membutuhkan informasi lengkap yang menampilkan nama dokter, spesialisasi, nama poli, dan hari praktiknya sekaligus agar bisa menjawab pertanyaan pasien dengan cepat dan akurat. Informasi ini berasal dari tiga tabel berbeda yang harus digabungkan.

```
SELECT
d.nama_dokter,
d.spesialis,
p.nama_poli,
j.hari_praktik
FROM dokter d
JOIN poli p
ON d.id_poli = p.id_poli
JOIN jadwal j
ON d.id_dokter = j.id_dokter;
```

Query menggabungkan tabel dokter dengan tabel poli melalui id_poli, dan dengan tabel jadwal melalui id_dokter. Hasilnya berupa 25 baris yang merepresentasikan seluruh sesi praktik dokter, di mana satu dokter bisa muncul lebih dari satu kali apabila memiliki beberapa jadwal praktik. Informasi ini sangat membantu dalam pelayanan informasi kepada pasien.

4.3.3. Query dengan Subquery dan CTE

Query 8: Pasien yang Pernah Berobat Lebih dari 1 kali (Subquery)

Studi Kasus: Tim kesehatan rumah sakit ingin mengidentifikasi pasien yang sering kembali berobat, karena pasien dengan kunjungan berulang biasanya memiliki kondisi kesehatan yang perlu pemantauan lebih intensif,

seperti penyakit kronis. Daftar pasien ini bisa digunakan sebagai dasar program follow-up atau edukasi kesehatan. Untuk mendapatkan daftar tersebut, diperlukan subquery karena kondisi filternya berbasis hasil agregasi.

```
SELECT nama_pasien
FROM pasien
WHERE id_pasien IN (
  SELECT id_pasien
  FROM pendaftaran
  GROUP BY id_pasien
  HAVING COUNT(*) > 1
);
```

Subquery terlebih dahulu mengelompokkan data tabel pendaftaran berdasarkan id_pasien, lalu menyaring hanya yang memiliki jumlah pendaftaran lebih dari 1 menggunakan HAVING. Hasil subquery berupa daftar id_pasien yang kemudian digunakan sebagai filter pada query luar melalui klausa IN. Query ini mengidentifikasi 1 pasien, yaitu Rina Wulandari (PA03) yang tercatat pada pendaftaran PE03 dan PE10.

Query 9: Jumlah Kunjungan Tiap Dokter (CTE)

Studi Kasus: Manajer operasional rumah sakit ingin mengevaluasi beban kerja masing-masing dokter dalam satu periode untuk keperluan penjadwalan ulang atau penambahan tenaga medis. Data ini diambil dari tabel rekam_medis, namun agar query lebih mudah dibaca dan dipelihara dibandingkan subquery bersarang, digunakanlah CTE untuk memisahkan proses agregasi dari proses pengambilan nama dokter.

```
WITH kunjungan_dokter AS (
  SELECT
    id_dokter,
    COUNT(*) AS total_kunjungan
  FROM rekam_medis
  GROUP BY id_dokter
)
SELECT
```

```
d.nama_dokter,  
k.total_kunjungan  
FROM kunjungan_dokter k  
JOIN dokter d  
ON k.id_dokter = d.id_dokter;
```

CTE bernama `kunjungan_dokter` menghasilkan tabel sementara yang berisi `id_dokter` dan jumlah rekam medis yang ditangani setiap dokter. Tabel sementara ini kemudian di-JOIN dengan tabel `dokter` untuk menampilkan nama dokter yang lebih informatif. Berdasarkan hasil query, dr. Hendra Prasetyo, Sp.PD menangani kunjungan terbanyak dengan 9 kunjungan, yang menunjukkan tingginya permintaan layanan poli penyakit dalam.

4.3.4. Query dengan Fungsi Agregat, GROUP BY, dan HAVING

Query agregat dengan GROUP BY dan HAVING digunakan untuk menghasilkan ringkasan statistik dari data yang ada. Klausula HAVING berfungsi sebagai filter pada hasil agregasi, berbeda dengan WHERE yang memfilter baris sebelum diagregasi.

Query 10: Dokter yang Menangani Lebih dari 3 Pasien

Studi Kasus: Manajemen rumah sakit ingin mengetahui dokter mana saja yang memiliki beban kerja tinggi, yaitu yang menangani lebih dari 3 pasien dalam periode yang diamati. Informasi ini berguna untuk menilai kinerja dokter, mengatur distribusi pasien yang lebih merata, atau mempertimbangkan penambahan dokter di poliklinik tertentu yang kelebihan beban.

```
SELECT  
d.nama_dokter,  
COUNT(*) AS jumlah_pasien  
FROM rekam_medis rm  
JOIN dokter d  
ON rm.id_dokter = d.id_dokter
```

```
GROUP BY d.nama_dokter  
HAVING COUNT(*) > 3;
```

Query menggabungkan tabel `rekam_medis` dan `dokter`, lalu mengelompokkan hasilnya berdasarkan nama dokter menggunakan `GROUP BY`. Fungsi `COUNT(*)` menghitung jumlah rekam medis per dokter, dan klausa `HAVING` menyaring hanya kelompok yang jumlahnya lebih dari 3. Hasilnya menunjukkan tiga dokter dengan beban kunjungan tertinggi, yaitu dr. Hendra Prasetyo, Sp.PD (9 kunjungan), dr. Andi Wirawan (5 kunjungan), dan dr. Lina Marlina, Sp.PD (4 kunjungan).

4.4. Implementasi View

View adalah tabel virtual yang dibentuk dari hasil query `SELECT`. View menyederhanakan akses data yang kompleks tanpa menyimpan data secara fisik, sehingga pengguna dapat mengakses informasi gabungan dengan perintah `SELECT` sederhana. Pada sistem ini dibuat dua buah view.

4.4.1. View `view_riwayat_pasien`

View ini menggabungkan 8 tabel untuk menyajikan riwayat lengkap setiap pasien dalam satu tampilan terpadu, mencakup data identitas pasien, rekam medis, diagnosa, resep obat, serta informasi dokter dan poli yang menangani.

```
CREATE VIEW view_riwayat_pasien AS  
SELECT  
    p.id_pasien,  
    p.nama_pasien,  
    p.jenis_kelamin,  
    p.gol_darah,  
    rm.id_rekam_medis,  
    rm.keluhan,  
    rm.hasil_pemeriksaan,
```

```

    rm.tgl_pemeriksaan,
    d.diagnosa,
    d.keterangan AS keterangan_diagnosa,
    o.nama_obat,
    r.dosis,
    r.jumlah AS jumlah_obat,
    r.aturan,
    dok.nama_dokter,
    dok.spesialis,
    po.nama_poli,
    po.lokasi AS lokasi_poli
FROM pasien p
JOIN pendaftaran pf ON p.id_pasien = pf.id_pasien
JOIN rekam_medis rm ON pf.id_pendaftaran = rm.id_pendaftaran
LEFT JOIN diagnosa d ON rm.id_rekam_medis = d.id_rekam_medis
LEFT JOIN resep r ON rm.id_rekam_medis = r.id_rekam_medis
LEFT JOIN obat o ON r.id_obat = o.id_obat
JOIN dokter dok ON rm.id_dokter = dok.id_dokter
JOIN poli po ON dok.id_poli = po.id_poli;

```

View ini menggunakan LEFT JOIN pada tabel diagnosa, resep, dan obat agar pasien yang belum memiliki diagnosa atau resep tetap muncul dalam hasil. View ini berguna bagi dokter dan tenaga medis untuk melihat riwayat lengkap pasien secara cepat.

Untuk menampilkan pengujian view_riwayat_pasien, sebagai contoh, gunakan sintaks:

```

SELECT * FROM view_riwayat_pasien
WHERE id_pasien = 'PA01';

```

4.4.2. View view_laporan_harian

View ini menghasilkan laporan harian operasional rumah sakit per dokter per tanggal, mencakup jumlah pasien, rekam medis, diagnosa, resep, jenis obat yang digunakan, total nilai resep, dan rata-rata berat badan pasien.

```
CREATE VIEW view_laporan_harian AS
SELECT
    pf.tgl_daftar AS tanggal,
    po.nama_poli,
    d.nama_dokter,
    COUNT(DISTINCT pf.id_pendaftaran) AS jumlah_pasien,
    COUNT(DISTINCT          rm.id_rekam_medis)          AS
jumlah_rekam_medis,
    COUNT(DISTINCT di.id_diagnosa) AS jumlah_diagnosa,
    COUNT(DISTINCT r.id_resep) AS jumlah_resep,
    COUNT(DISTINCT r.id_obat) AS jumlah_jenis_obat,
    ROUND(SUM(r.jumlah      *      o.harga_obat),      0)      AS
total_nilai_resep,
    ROUND(AVG(v.berat_badan),          1)          AS
rata_rata_berat_badan_pasien
FROM pendaftaran pf
JOIN jadwal j ON pf.id_jadwal = j.id_jadwal
JOIN dokter d ON j.id_dokter = d.id_dokter
JOIN poli po ON d.id_poli = po.id_poli
LEFT JOIN rekam_medis rm ON pf.id_pendaftaran =
rm.id_pendaftaran
LEFT JOIN diagnosa di ON rm.id_rekam_medis =
di.id_rekam_medis
LEFT JOIN resep r ON rm.id_rekam_medis = r.id_rekam_medis
LEFT JOIN obat o ON r.id_obat = o.id_obat
LEFT JOIN vital v ON rm.id_rekam_medis = v.id_rekam_medis
GROUP BY pf.tgl_daftar, po.nama_poli, d.nama_dokter
```

```
ORDER BY pf.tgl_daftar DESC, jumlah_pasien DESC;
```

View ini memanfaatkan COUNT(DISTINCT ...) untuk menghindari duplikasi akibat hasil JOIN banyak tabel. Fungsi SUM dan AVG digunakan untuk menghasilkan metrik operasional. View ini sangat berguna bagi manajemen rumah sakit untuk monitoring kinerja harian.

Untuk menampilkan pengujian view_laporan_harian, gunakan sintaks:

```
SELECT * FROM view_laporan_harian;
```

4.5. Implementasi Stored Procedure

Stored procedure adalah sekumpulan perintah SQL yang tersimpan di dalam basis data dan dapat dipanggil kapan saja dengan perintah CALL. Stored procedure mempermudah eksekusi operasi yang sering digunakan dan meningkatkan keamanan akses data.

4.5.1. Procedure riwayat_pasien

Stored procedure menerima satu parameter input berupa id_pasien (VARCHAR 10) dan mengembalikan riwayat medis lengkap pasien tersebut, mencakup tanggal pemeriksaan, keluhan, hasil pemeriksaan, dan diagnosa.

```
DELIMITER //
```

```
CREATE PROCEDURE riwayat_pasien(  
    IN p_id_pasien VARCHAR(10)  
)  
  
BEGIN  
    SELECT  
        p.nama_pasien,  
        rm.tgl_pemeriksaan,  
        rm.keluhan,  
        rm.hasil_pemeriksaan,  
        dg.diagnosa  
    FROM pasien p
```

```

JOIN pendaftaran pd
    ON p.id_pasien = pd.id_pasien
JOIN rekam_medis rm
    ON pd.id_pendaftaran = rm.id_pendaftaran
LEFT JOIN diagnosa dg
    ON rm.id_rekam_medis = dg.id_rekam_medis
WHERE p.id_pasien = p_id_pasien;
END //
DELIMITER ;

```

Contoh pemanggilan procedure untuk melihat riwayat pasien PA20 (Fajar Ramadhan):

```
CALL riwayat_pasien('PA20');
```

Procedure menggunakan LEFT JOIN pada tabel diagnosa agar rekam medis yang belum memiliki diagnosa tetap ditampilkan. Parameter p_id_pasien memungkinkan pengguna memanggil riwayat pasien manapun hanya dengan mengganti nilai id_pasien saat pemanggilan.

4.6. Implementasi Trigger

Trigger adalah prosedur yang dijalankan secara otomatis oleh DBMS ketika terjadi operasi tertentu (INSERT, UPDATE, atau DELETE) pada sebuah tabel. Trigger yang dibuat pada sistem ini berfungsi sebagai audit log untuk memantau seluruh perubahan data pada tabel pasien.

4.6.1. Trigger tr_audit_pasien_insert

Trigger ini aktif setelah terjadi operasi INSERT pada tabel pasien. Setiap data pasien baru yang ditambahkan akan dicatat ke tabel audit_log beserta nilai data baru, nama tabel, aksi, dan username yang melakukan operasi.

```
CREATE TRIGGER tr_audit_pasien_insert
```

```

AFTER INSERT ON pasien
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (nama_tabel, aksi, id_data,
data_baru, username)
    VALUES (
        'pasien',
        'INSERT',
        NEW.id_pasien,
        CONCAT('nama: ', NEW.nama_pasien,
            ', JK: ', NEW.jenis_kelamin,
            ', tgl_lahir: ', NEW.tgl_lahir,
            ', alamat: ', NEW.alamat,
            ', no_hp: ', NEW.no_hp_pasien,
            ', gol_darah: ', NEW.gol_darah),
        USER()
    );
END//
DELIMITER;

```

Contoh penggunaan trigger INSERT:

```

INSERT INTO pasien (id_pasien, nama_pasien, jenis_kelamin,
tgl_lahir, alamat, no_hp_pasien, gol_darah)
VALUES ('PA26', 'Conrad Fisher', 'L', '2000-01-01', 'Jl.
Gunung Muria', '0882222220', 'O');
SELECT * FROM audit_log;

```

4.6.2. Trigger tr_audit_pasien_update

Trigger ini aktif setelah operasi UPDATE pada tabel pasien. Trigger menyimpan nilai sebelum perubahan (data_lama menggunakan OLD) dan sesudah perubahan (data_baru menggunakan NEW) ke tabel audit_log, sehingga jejak perubahan data dapat ditelusuri secara lengkap.

```

CREATE TRIGGER tr_audit_pasien_update
AFTER UPDATE ON pasien
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (nama_tabel, aksi, id_data,
data_lama, data_baru, username)
VALUES (
    'pasien',
    'UPDATE',
    NEW.id_pasien,
    CONCAT('nama: ', OLD.nama_pasien,
        ', JK: ', OLD.jenis_kelamin,
        ', tgl_lahir: ', OLD.tgl_lahir,
        ', alamat: ', OLD.alamat,
        ', no_hp: ', OLD.no_hp_pasien,
        ', gol_darah: ', OLD.gol_darah),
    CONCAT('nama: ', NEW.nama_pasien,
        ', JK: ', NEW.jenis_kelamin,
        ', tgl_lahir: ', NEW.tgl_lahir,
        ', alamat: ', NEW.alamat,
        ', no_hp: ', NEW.no_hp_pasien,
        ', gol_darah: ', NEW.gol_darah),
    USER()
);
END//
DELIMITER //

```

Contoh penggunaan trigger UPDATE:

```

UPDATE pasien SET nama_pasien = 'Jeremiah Fisher' WHERE
id_pasien = 'PA26';

```

4.6.3. Trigger tr_audit_pasien_delete

Trigger ini aktif setelah operasi DELETE pada tabel pasien. Data yang dihapus dicatat ke kolom data_lama pada audit_log menggunakan OLD, sehingga data tersebut masih dapat ditelusuri meskipun sudah tidak ada di tabel pasien.

```
CREATE TRIGGER tr_audit_pasien_delete
AFTER DELETE ON pasien
FOR EACH ROW
BEGIN
    INSERT INTO audit_log (nama_tabel, aksi, id_data,
data_lama, username)
VALUES (
    'pasien',
    'DELETE',
    OLD.id_pasien,
    CONCAT('nama: ', OLD.nama_pasien,
        ', JK: ', OLD.jenis_kelamin,
        ', tgl_lahir: ', OLD.tgl_lahir,
        ', alamat: ', OLD.alamat,
        ', no_hp: ', OLD.no_hp_pasien,
        ', gol_darah: ', OLD.gol_darah),
    USER()
);
END//

DELIMITER ;
```

Ketiga trigger ini bersama-sama membentuk sistem audit log yang komprehensif. Setiap perubahan data pasien — baik penambahan, pembaruan, maupun penghapusan tercatat secara otomatis dengan timestamp, username, dan detail perubahan datanya.

Contoh penggunaan trigger DELETE:

```
DELETE FROM pasien WHERE id_pasien = 'PA26';
```

```
SELECT * FROM audit_log;
```

4.7. Implementasi Function

Function adalah objek basis data yang menerima parameter, melakukan pemrosesan, dan mengembalikan satu nilai. Function dapat dipanggil langsung di dalam perintah SELECT. Pada sistem ini dibuat dua buah function.

4.7.1. Function fn_hitung_usia

Function ini menghitung usia pasien dalam satuan tahun berdasarkan tanggal lahir yang diberikan sebagai parameter. Function menggunakan fungsi YEAR() dan DATE_ADD() untuk memastikan perhitungan usia akurat, usia baru bertambah setelah tanggal ulang tahun pada tahun berjalan terlewati.

```
DELIMITER //  
  
CREATE FUNCTION fn_hitung_usia(p_tgl_lahir DATE)  
RETURNS INT  
DETERMINISTIC  
READS SQL DATA  
BEGIN  
    DECLARE v_usia INT;  
    IF p_tgl_lahir IS NULL THEN  
        RETURN NULL;  
    END IF;  
    SET v_usia = YEAR(CURDATE()) - YEAR(p_tgl_lahir);  
    IF DATE_ADD(p_tgl_lahir, INTERVAL v_usia YEAR) >  
CURDATE() THEN  
        SET v_usia = v_usia - 1;  
    END IF;  
    RETURN v_usia;  
END //
```

```
DELIMITER ;
```

Contoh penggunaan function untuk menampilkan pasien berusia di atas 20 tahun:

```
SELECT
    nama_pasien,
    tgl_lahir,
    fn_hitung_usia(tgl_lahir) AS usia
FROM pasien
WHERE fn_hitung_usia(tgl_lahir) > 20;
```

Seluruh 24 pasien dalam basis data memenuhi syarat usia > 20 tahun.

4.7.2. Function **fn_total_biaya_obat**

Function ini menghitung total biaya obat dari seluruh resep yang terkait dengan satu rekam medis. Parameter input berupa `id_rekam_medis`, dan function menjumlahkan hasil perkalian jumlah obat dengan harga satuan dari tabel resep dan obat. Fungsi `COALESCE` digunakan untuk mengembalikan nilai 0 apabila rekam medis tidak memiliki resep.

```
DELIMITER //
CREATE FUNCTION fn_total_biaya_obat(p_id_rekam_medis
VARCHAR(10))
RETURNS DECIMAL(12,2)
DETERMINISTIC
READS SQL DATA
BEGIN
    DECLARE v_total DECIMAL(12,2);
    SELECT COALESCE(SUM(r.jumlah * o.harga_obat), 0)
    INTO v_total
    FROM resep r
    JOIN obat o ON r.id_obat = o.id_obat
```

```
WHERE r.id_rekam_medis = p_id_rekam_medis;

RETURN v_total;

END //

DELIMITER ;
```

Contoh penggunaan function untuk menampilkan total biaya obat seluruh rekam medis beserta nama pasien:

```
SELECT

    rm.id_rekam_medis,

    p.nama_pasien,

    fn_total_biaya_obat(rm.id_rekam_medis) AS
total_biaya_obat
FROM rekam_medis rm
JOIN pendaftaran pf ON rm.id_pendaftaran =
pf.id_pendaftaran
JOIN pasien p ON pf.id_pasien = p.id_pasien;
```

Function ini sangat berguna untuk fitur penghitungan tagihan pasien, di mana total biaya obat dapat dijumlahkan dengan biaya layanan lainnya untuk menghasilkan tagihan akhir.

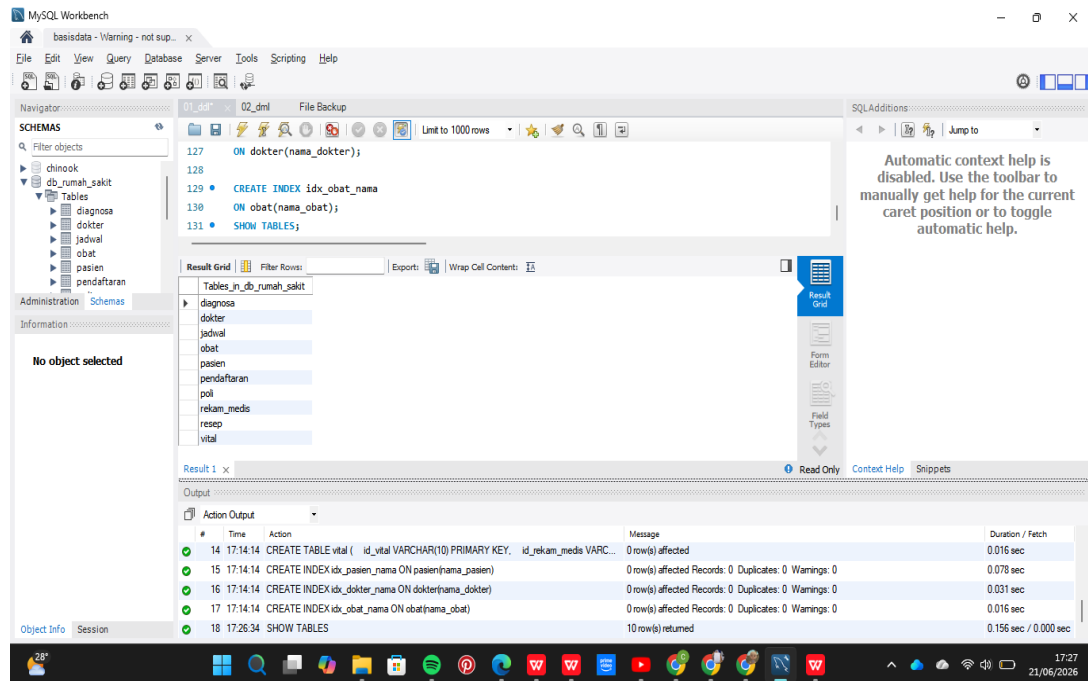
BAB V

PENGUJIAN QUERY

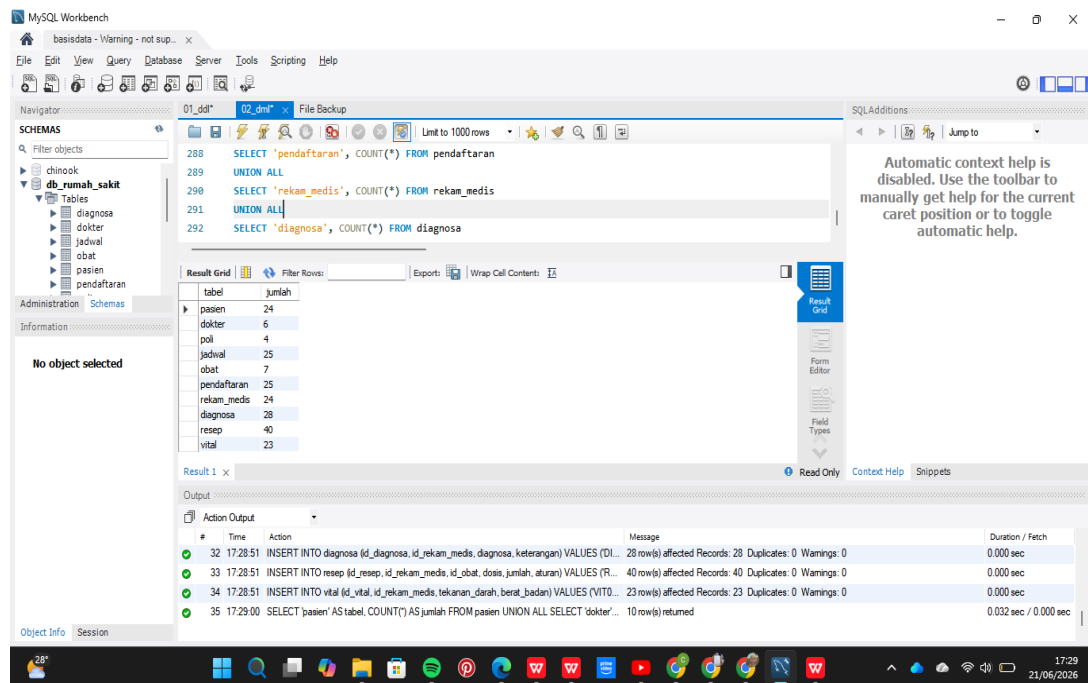
Bab ini membahas hasil pengujian terhadap query dan objek lanjutan yang telah diimplementasikan pada Sistem Informasi Rumah Sakit. Pengujian dilakukan untuk memastikan bahwa setiap query, view, stored procedure, trigger, dan function dapat berjalan dengan baik sesuai kebutuhan sistem. Hasil pengujian disajikan dalam bentuk output yang dihasilkan, serta analisis untuk menunjukkan bahwa objek basis data yang dibuat telah berfungsi sebagaimana mestinya.

5.1. Pengujian DDL dan DML

Pengujian DDL dan DML dilakukan dengan menjalankan seluruh script 01_ddl.sql dan 02_dml.sql, kemudian memverifikasi struktur tabel dan jumlah data menggunakan perintah SHOW TABLES, DESCRIBE, dan SELECT COUNT(*).



Gambar 5. 1 Tampilan DDL dan DML

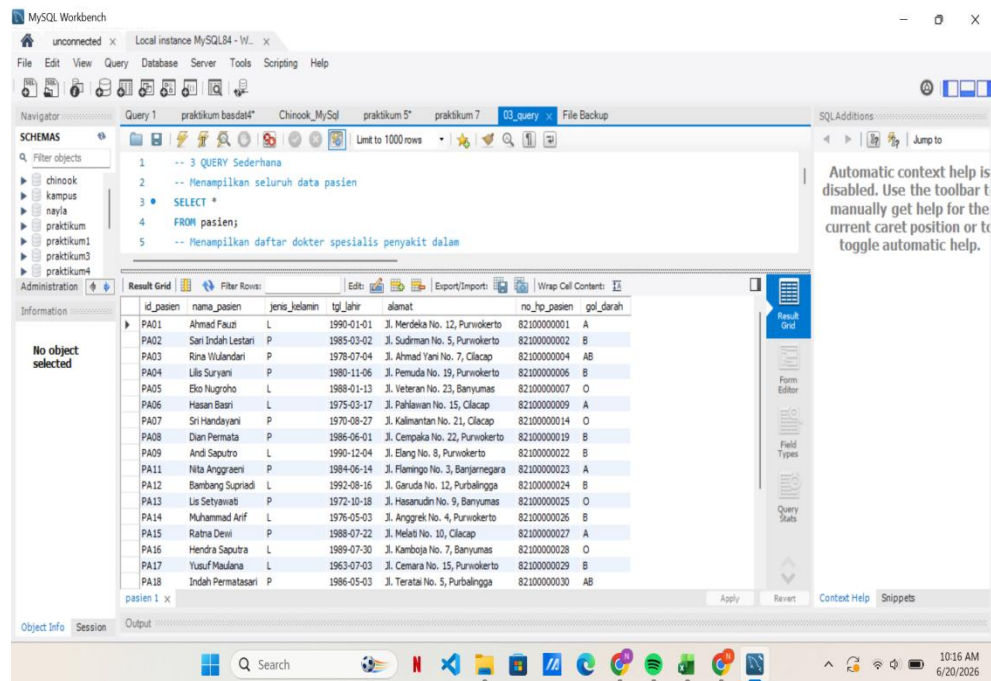


Gambar 5. 2 Tampilan DDL dan DML

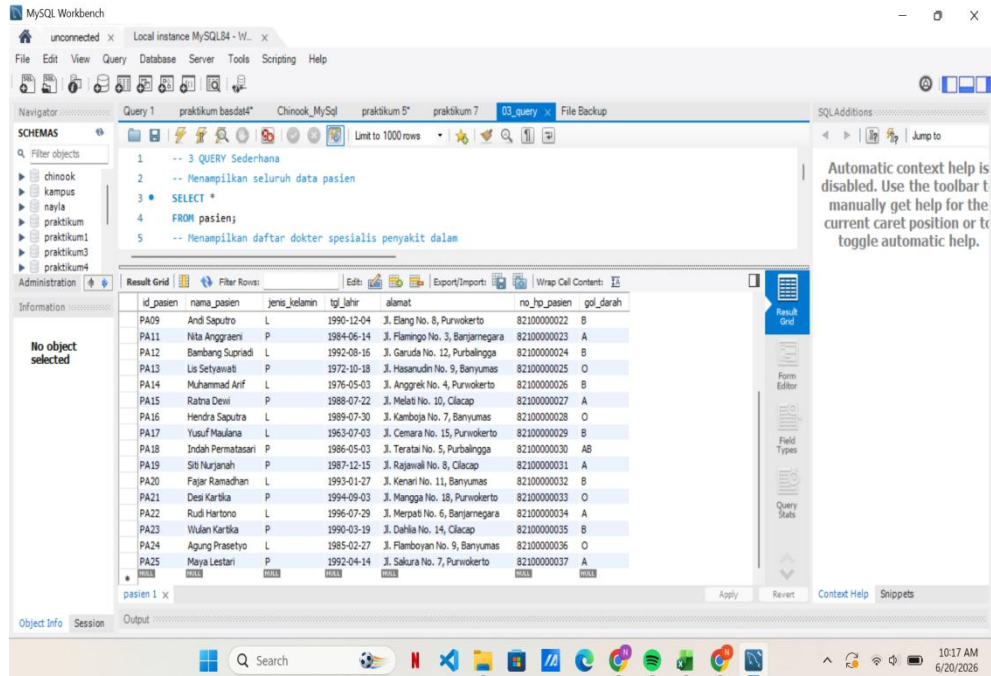
5.2. Pengujian Query Select

Pengujian query select dilakukan untuk memastikan bahwa data yang tersimpan dalam basis data dapat ditampilkan sesuai kebutuhan pengguna. Query yang diuji meliputi query sederhana hingga query yang melibatkan beberapa tabel dan fungsi agregasi.

5.2.1. Menampilkan Seluruh Data Pasien



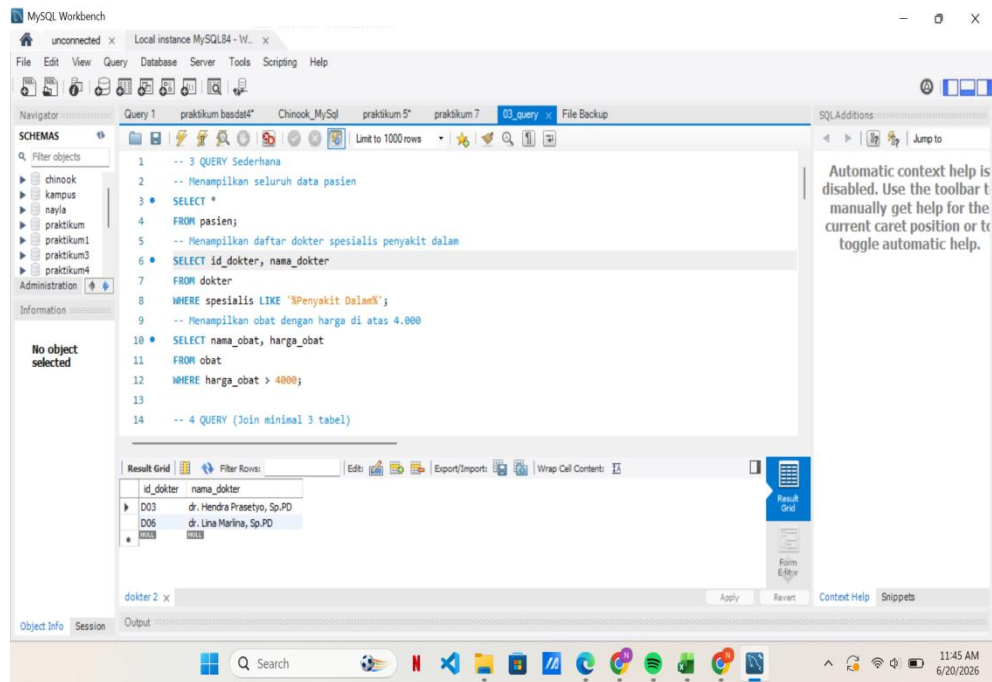
Gambar 5. 3 Tampilan Seluruh Data Pasien



Gambar 5. 4 Tampilan Seluruh Data Pasien

Query berhasil menampilkan 24 baris data dengan kolom id_pasien, nama_pasien, jenis_kelamin, tgl_lahir, alamat, no_hp_pasien, dan gol_darah.

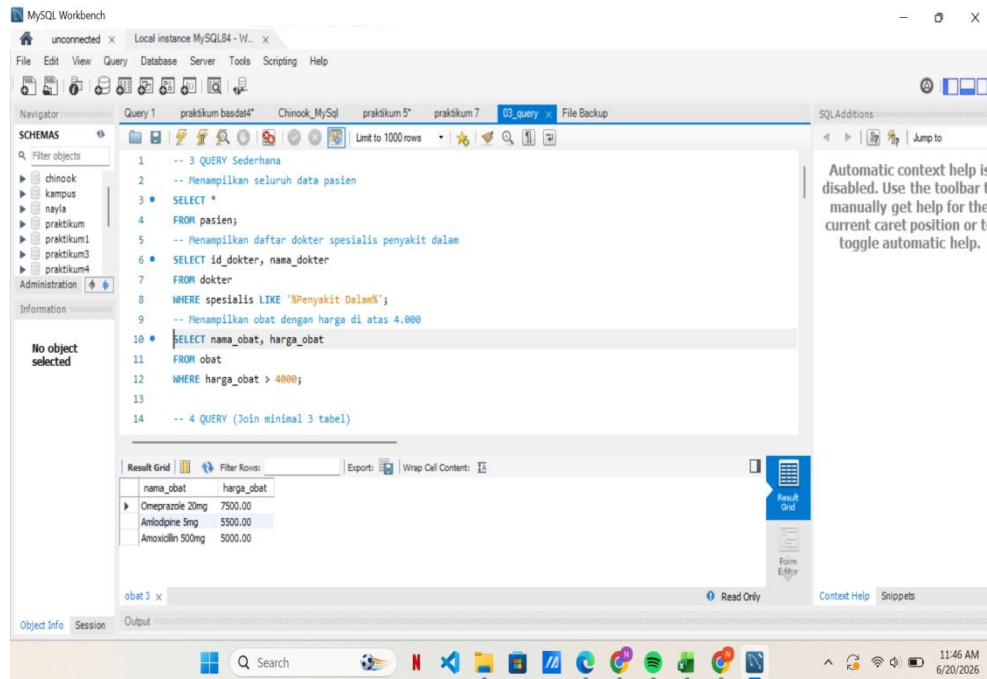
5.2.2. Menampilkan Daftar Dokter Spesialis Penyakit Dalam



Gambar 5. 5 Tampilan Daftar Dokter Spesialis Penyakit Dalam

Query berhasil menampilkan 2 dokter yang sesuai filter. Operator LIKE '%Penyakit Dalam%' berfungsi dengan benar menyaring berdasarkan substring.

5.2.3. Menampilkan Obat dengan Harga di Atas 4000



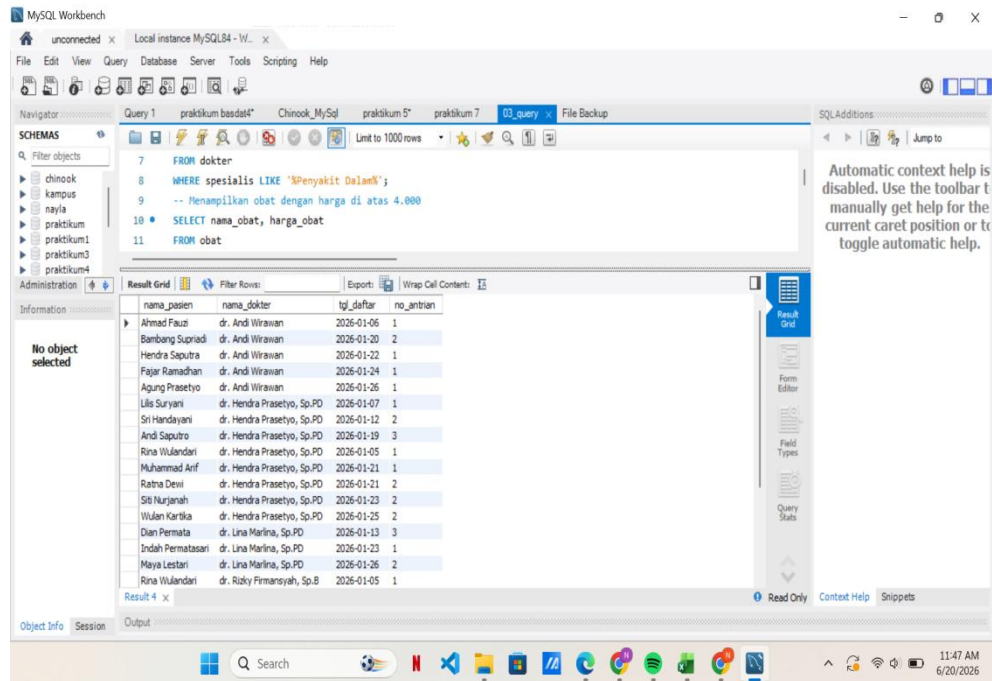
Gambar 5. 6 Tampilan Obat dengan Harga di Atas 4000

Query berhasil menampilkan 3 obat sesuai ekspektasi. Filter kondisi > 4000 berjalan tepat. Cetirizine 10mg seharga Rp4.000 tidak masuk dalam hasil.

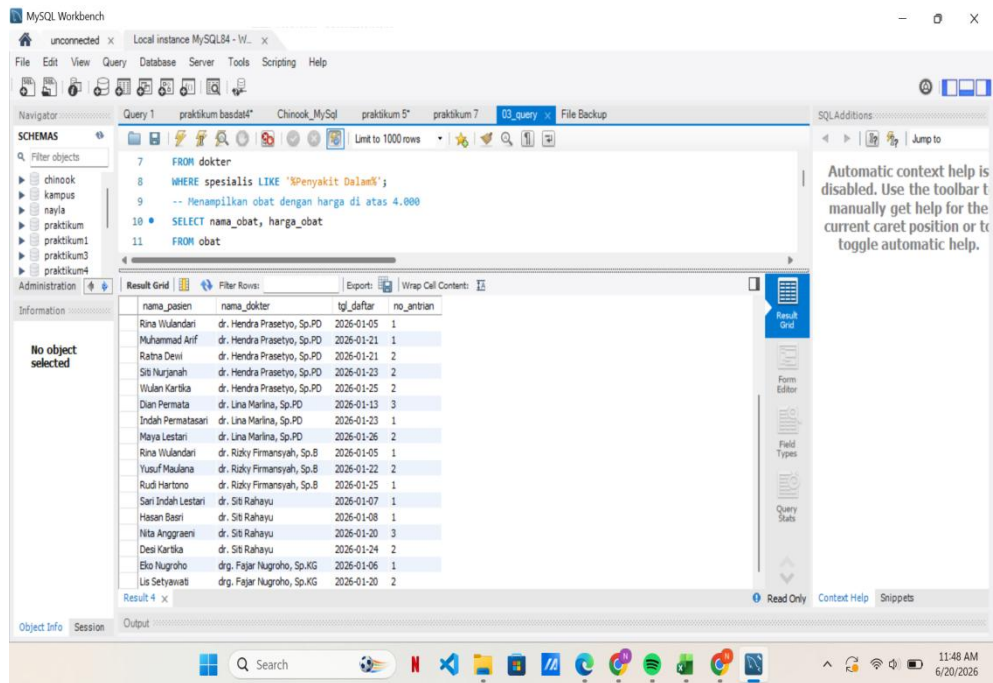
5.3. Pengujian Query dengan JOIN

Pengujian empat query JOIN dilakukan untuk memverifikasi bahwa penggabungan beberapa tabel menghasilkan data yang konsisten dan sesuai relasi yang telah didefinisikan.

5.3.1. Menampilkan Data Pendaftaran Pasien Beserta Dokter yang Menanganinya



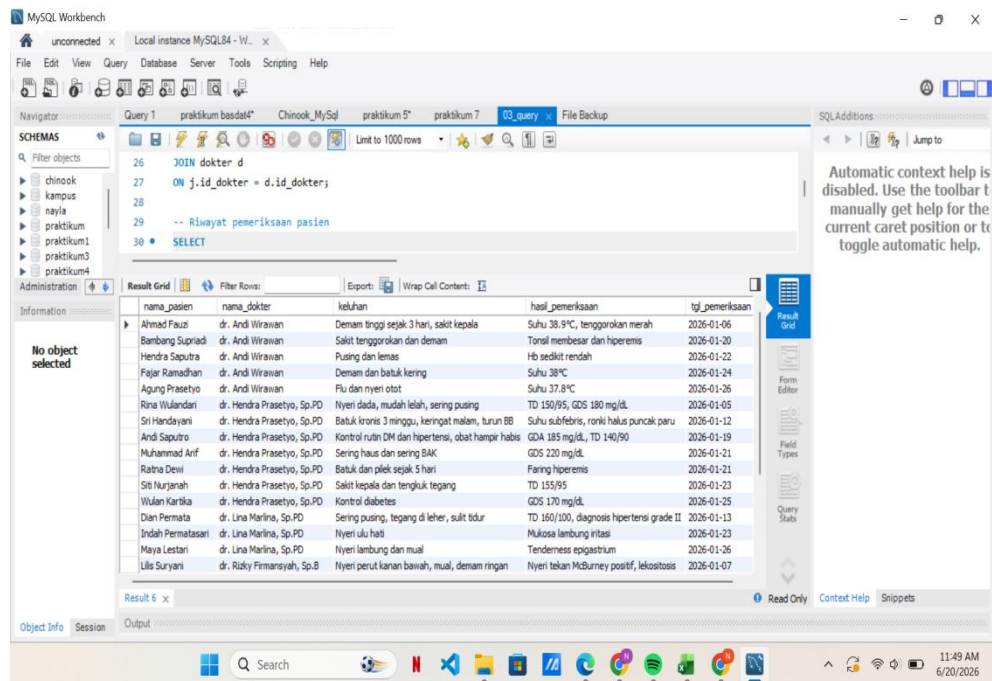
Gambar 5. 7 Tampilan Data Pendaftaran Pasien Beserta Dokter yang Menangani



Gambar 5. 8 Tampilan Data Pendaftaran Pasien Beserta Dokter yang Menangani

Query berhasil menampilkan 25 baris. Setiap baris menunjukkan pasien yang terhubung dengan dokter yang benar melalui relasi pendaftaran, jadwal, dan dokter.

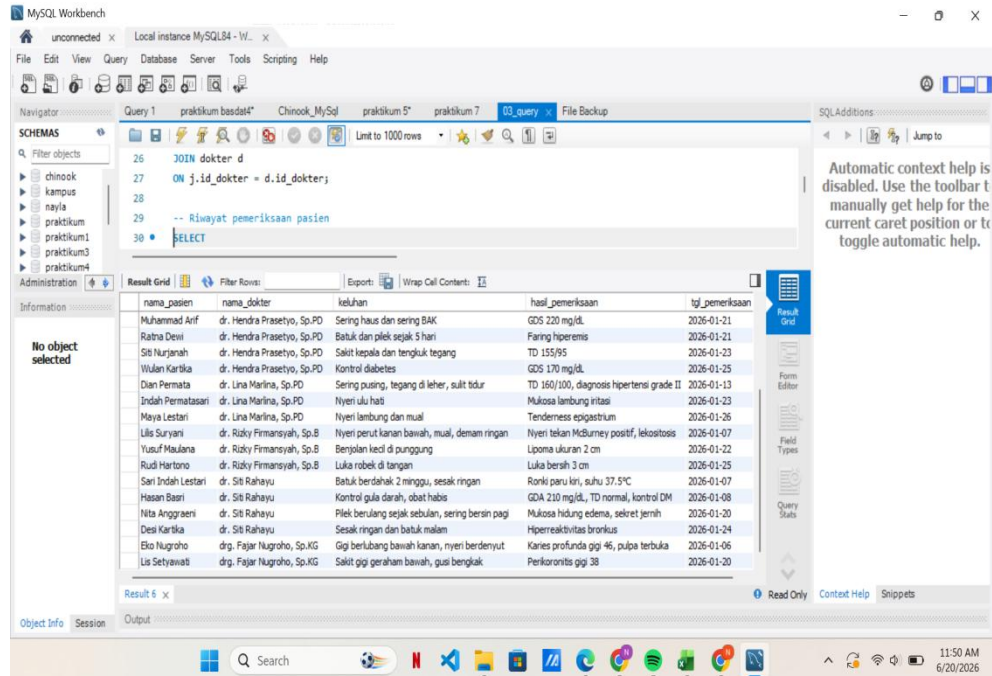
5.3.2. Menampilkan Riwayat Pemeriksaan Pasien



The screenshot shows the MySQL Workbench interface with a query executed. The query is a JOIN between a table of doctors and a table of patient examinations. The result grid displays the following data:

nama_pasien	nama_dokter	keluhan	hasil_pemeriksaan	tgl_pemeriksaan
Ahmad Fauzi	dr. Andi Wirawan	Demam tinggi sejak 3 hari, sakit kepala	Suhu 38.9°C, tenggorokan merah	2026-01-06
Bambang Supriadi	dr. Andi Wirawan	Sakit tenggorokan dan demam	Tonsil membesar dan hiperemis	2026-01-20
Hendra Saputra	dr. Andi Wirawan	Pusing dan lemas	Hb sedikit rendah	2026-01-22
Fajar Ramadhan	dr. Andi Wirawan	Demam dan batuk kering	Suhu 38°C	2026-01-24
Agung Prasetyo	dr. Andi Wirawan	Flu dan nyeri otot	Suhu 37.8°C	2026-01-26
Rina Wulandari	dr. Hendra Prasetyo, Sp.PD	Nyeri dada, mudah lelah, sering pusing	TD 150/95, GDS 180 mg/dL	2026-01-05
Sri Handayani	dr. Hendra Prasetyo, Sp.PD	Batuk kronis 3 minggu, keringat malam, turun BB	Suhu subfebris, ronki halus puncak paru	2026-01-12
Andi Saputro	dr. Hendra Prasetyo, Sp.PD	Kontrol rutin DM dan hipertensi, obat hampir habis	GDA 185 mg/dL, TD 140/90	2026-01-19
Muhammad Arif	dr. Hendra Prasetyo, Sp.PD	Sering haus dan sering BAK	GDS 220 mg/dL	2026-01-21
Ratna Dewi	dr. Hendra Prasetyo, Sp.PD	Batuk dan pilek sejak 5 hari	Faring hiperemis	2026-01-21
Siti Nurjanah	dr. Hendra Prasetyo, Sp.PD	Sakit kepala dan tengkuk tegang	TD 155/95	2026-01-23
Wulan Kartika	dr. Hendra Prasetyo, Sp.PD	Kontrol diabetes	GDS 170 mg/dL	2026-01-25
Dan Permata	dr. Lina Marlina, Sp.PD	Sering pusing, tegang di leher, sulit tidur	TD 160/100, diagnosis hipertensi grade II	2026-01-13
Indah Permatastari	dr. Lina Marlina, Sp.PD	Nyeri ulu hati	Mukosa lambung iritasi	2026-01-23
Maya Lestari	dr. Lina Marlina, Sp.PD	Nyeri lambung dan mual	Tenderness epigastrium	2026-01-26
Lila Suryani	dr. Rizky Firmansyah, Sp.B	Nyeri perut kanan bawah, mual, demam ringan	Nyeri tekan McBurney positif, leukositosis	2026-01-07

Gambar 5. 9 Tampilan Riwayat Pemeriksaan Pasien



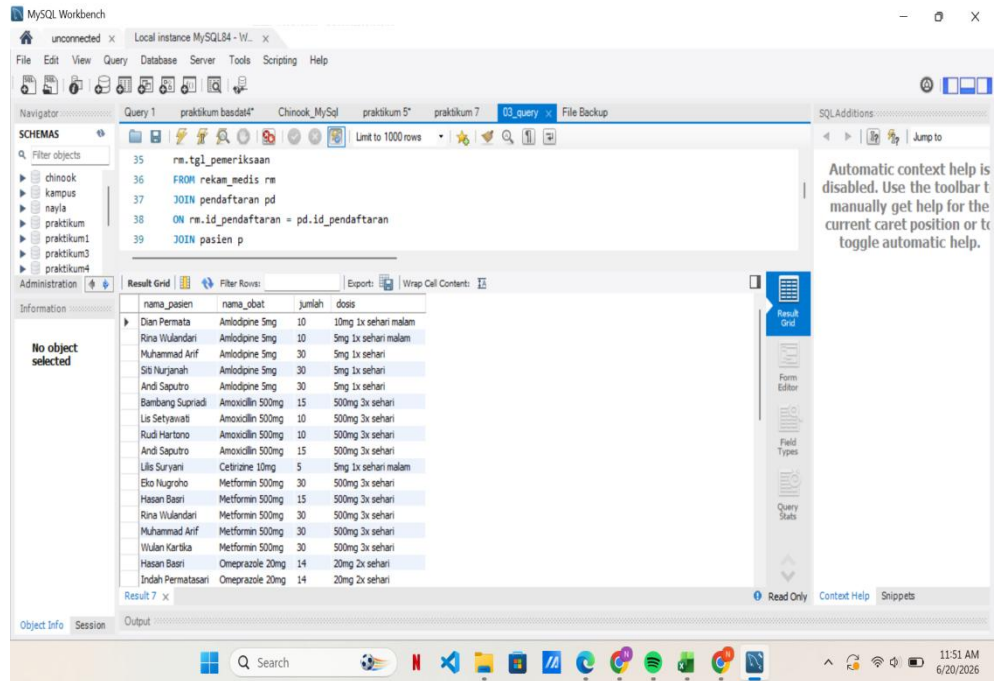
The screenshot shows the MySQL Workbench interface with a query executed. The query is a JOIN between a table of doctors and a table of patient examinations. The result grid displays the following data:

nama_pasien	nama_dokter	keluhan	hasil_pemeriksaan	tgl_pemeriksaan
Muhammad Arif	dr. Hendra Prasetyo, Sp.PD	Sering haus dan sering BAK	GDS 220 mg/dL	2026-01-21
Ratna Dewi	dr. Hendra Prasetyo, Sp.PD	Batuk dan pilek sejak 5 hari	Faring hiperemis	2026-01-21
Siti Nurjanah	dr. Hendra Prasetyo, Sp.PD	Sakit kepala dan tengkuk tegang	TD 155/95	2026-01-23
Wulan Kartika	dr. Hendra Prasetyo, Sp.PD	Kontrol diabetes	GDS 170 mg/dL	2026-01-25
Dan Permata	dr. Lina Marlina, Sp.PD	Sering pusing, tegang di leher, sulit tidur	TD 160/100, diagnosis hipertensi grade II	2026-01-13
Indah Permatastari	dr. Lina Marlina, Sp.PD	Nyeri ulu hati	Mukosa lambung iritasi	2026-01-23
Maya Lestari	dr. Lina Marlina, Sp.PD	Nyeri lambung dan mual	Tenderness epigastrium	2026-01-26
Lila Suryani	dr. Rizky Firmansyah, Sp.B	Nyeri perut kanan bawah, mual, demam ringan	Nyeri tekan McBurney positif, leukositosis	2026-01-07
Yusuf Maulana	dr. Rizky Firmansyah, Sp.B	Bengkak kaki di punggung	Lipoma ukuran 2 cm	2026-01-22
Rudi Hartono	dr. Rizky Firmansyah, Sp.B	Luka robek di tangan	Luka bersih 3 cm	2026-01-25
Seri Indah Lestari	dr. Siti Rahayu	Batuk berdahak 2 minggu, sesak ringan	Ronki paru kiri, suhu 37.5°C	2026-01-07
Hasan Baari	dr. Siti Rahayu	Kontrol gula darah, obat habis	GDA 210 mg/dL, TD normal, kontrol DM	2026-01-08
Nita Anggraeni	dr. Siti Rahayu	Pilek berulang sejak sebulan, sering bersin pagi	Mukosa hidung edema, sekret jernih	2026-01-20
Desi Kartika	dr. Siti Rahayu	Sesak ringan dan batuk malam	Hiperreaktivitas bronkus	2026-01-24
Eko Nugroho	dr. Fajar Nugroho, Sp.KG	Gigi berlubang bawah kanan, nyeri berdenyut	Karies profunda gigi 46, pulpa terbuka	2026-01-06
Lis Setyewati	dr. Fajar Nugroho, Sp.KG	Sakit gigi geraham bawah, gusi bengkak	Perikoronitis gigi 38	2026-01-20

Gambar 5. 10 Tampilan Riwayat Pemeriksaan Pasien

Query berhasil menampilkan 24 baris. Integritas data terjaga, nama pasien dan dokter pada setiap baris sesuai dengan relasi foreign key di tabel rekam_medis.

5.3.3. Menampilkan Resep Obat per Pasien



Gambar 5. 11 Tampilan Resep Obat per Pasien

MySQL Workbench

Query 1 praktikum dasar4* Chinook_MySql praktikum 5* praktikum 7 03_query File Backup

```

35  rm.tgl_pemeriksaan
36  FROM rekan_medis rm
37  JOIN pendaftaran pd
38  ON rm.id_pendaftaran = pd.id_pendaftaran
39  JOIN pasien p

```

Result Grid

nama_pasien	nama_obat	jumlah	dosis
Maya Lestari	Omeprazole 20mg	14	20mg 2x sehari
Ahmad Fauzi	Paracetamol 500...	10	500mg 3x sehari
Rina Wulandari	Paracetamol 500...	15	500mg 3x sehari
Rina Wulandari	Paracetamol 500...	5	5mg 1x sehari malam
Sri Handayani	Paracetamol 500...	10	500mg 3x sehari
Dani Permata	Paracetamol 500...	15	500mg 3x sehari
Bambang Supriadi	Paracetamol 500...	10	500mg 3x sehari
Lis Setyavati	Paracetamol 500...	10	500mg 3x sehari
Ratna Dewi	Paracetamol 500...	10	500mg 3x sehari
Yusuf Maulana	Paracetamol 500...	10	500mg 3x sehari
Indah Permata Sari	Paracetamol 500...	10	500mg 3x sehari
Siti Nurjanah	Paracetamol 500...	10	500mg 3x sehari
Fajar Ramadhan	Paracetamol 500...	10	500mg 3x sehari
Rudi Hartono	Paracetamol 500...	10	500mg 3x sehari
Agung Prasetyo	Paracetamol 500...	10	500mg 3x sehari
Maya Lestari	Paracetamol 500...	10	500mg 3x sehari
Sani Indah Lestari	Vitamin C 500mg	7	500mg 1x sehari

Gambar 5. 12 Tampilan Resep Obat per Pasien

MySQL Workbench

Query 1 praktikum dasar4* Chinook_MySql praktikum 5* praktikum 7 03_query File Backup

```

35  rm.tgl_pemeriksaan
36  FROM rekan_medis rm
37  JOIN pendaftaran pd
38  ON rm.id_pendaftaran = pd.id_pendaftaran
39  JOIN pasien p

```

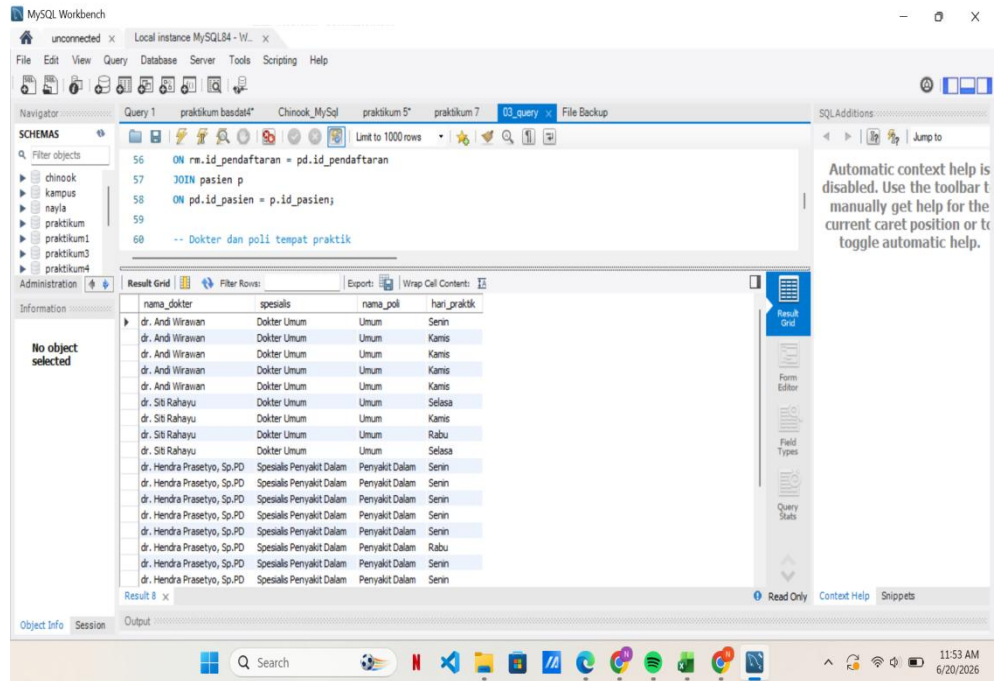
Result Grid

nama_pasien	nama_obat	jumlah	dosis
Bambang Supriadi	Paracetamol 500...	10	500mg 3x sehari
Lis Setyavati	Paracetamol 500...	10	500mg 3x sehari
Ratna Dewi	Paracetamol 500...	10	500mg 3x sehari
Yusuf Maulana	Paracetamol 500...	10	500mg 3x sehari
Indah Permata Sari	Paracetamol 500...	10	500mg 3x sehari
Siti Nurjanah	Paracetamol 500...	10	500mg 3x sehari
Fajar Ramadhan	Paracetamol 500...	10	500mg 3x sehari
Rudi Hartono	Paracetamol 500...	10	500mg 3x sehari
Agung Prasetyo	Paracetamol 500...	10	500mg 3x sehari
Maya Lestari	Paracetamol 500...	10	500mg 3x sehari
Sani Indah Lestari	Vitamin C 500mg	7	500mg 1x sehari
Nita Anggraeni	Vitamin C 500mg	10	500mg 1x sehari
Ratna Dewi	Vitamin C 500mg	10	500mg 1x sehari
Hendra Saputra	Vitamin C 500mg	14	500mg 1x sehari
Fajar Ramadhan	Vitamin C 500mg	10	500mg 1x sehari
Desi Kartika	Vitamin C 500mg	10	500mg 1x sehari
Agung Prasetyo	Vitamin C 500mg	10	500mg 1x sehari

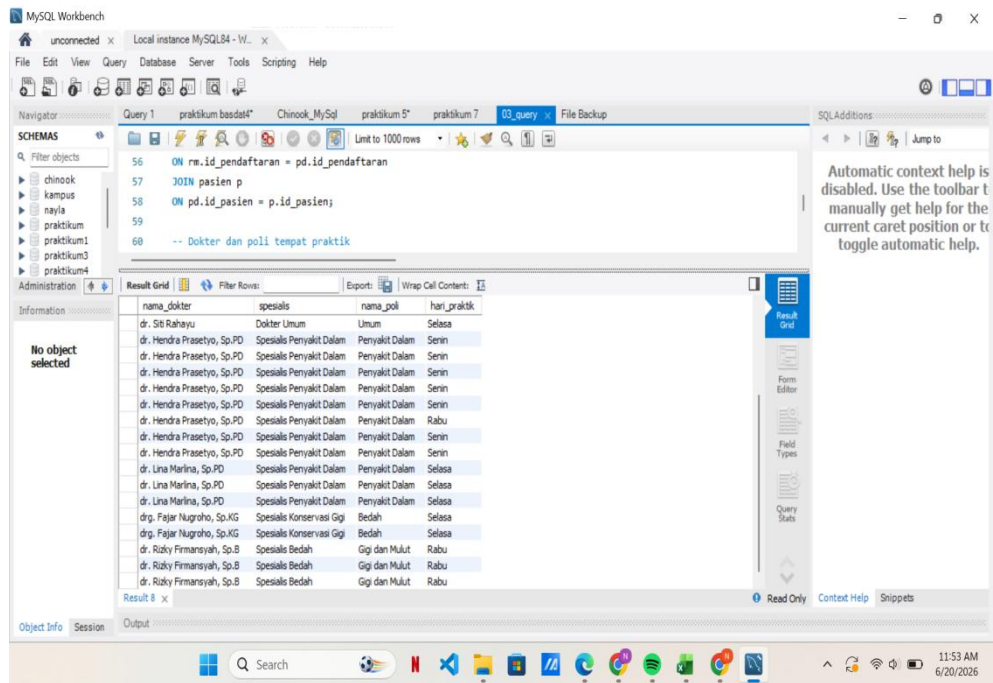
Gambar 5. 13 Tampilan Resep Obat per Pasien

Query berhasil menampilkan 40 baris yang merepresentasikan seluruh resep. Penelusuran dari tabel resep hingga nama pasien melalui 5 tabel berjalan dengan benar.

5.3.4. Menampilkan Dokter dan Jadwal Poli



Gambar 5. 14 Tampilan Dokter dan Jadwal Poli

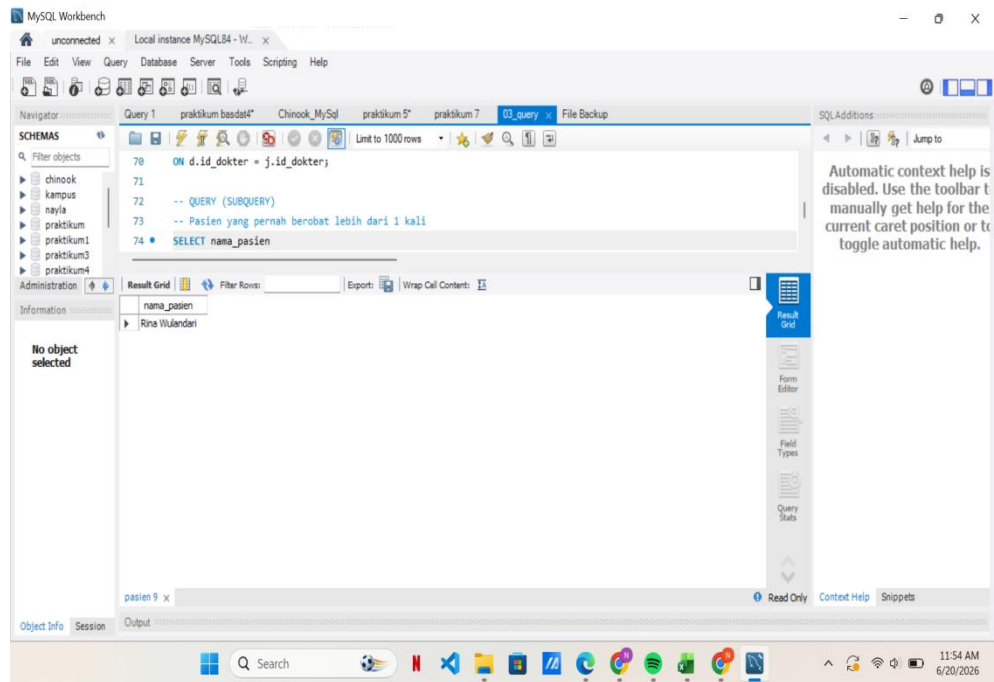


Gambar 5. 15 Tampilan Dokter dan Jadwal Poli

Query berhasil menampilkan 25 baris. Setiap dokter tampil dengan nama poli dan hari praktik yang sesuai relasi tabel dokter, poli, dan jadwal.

5.4. Pengujian Query Subquery dan CTE

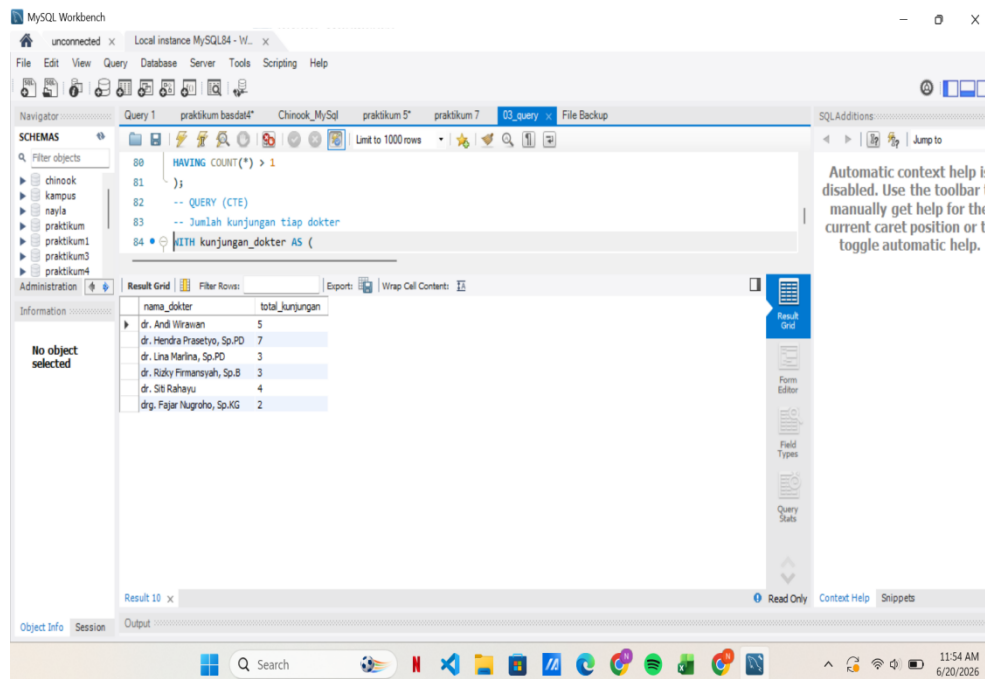
5.4.1. Menampilkan Pasien yang Pernah Periksa Lebih dari Satu Kali (Subquery)



Gambar 5. 16 Tampilan Pasien yang Pernah Periksa Lebih Dari Satu Kali

Query berhasil mengidentifikasi 1 pasien yang berobat lebih dari satu kali, yaitu Rina Wulandari (PA03). Subquery berfungsi benar dalam menyaring hasil agregat sebelum diteruskan ke query luar.

5.4.2. Menampilkan Jumlah Kunjungan Setiap Dokter (CTE)

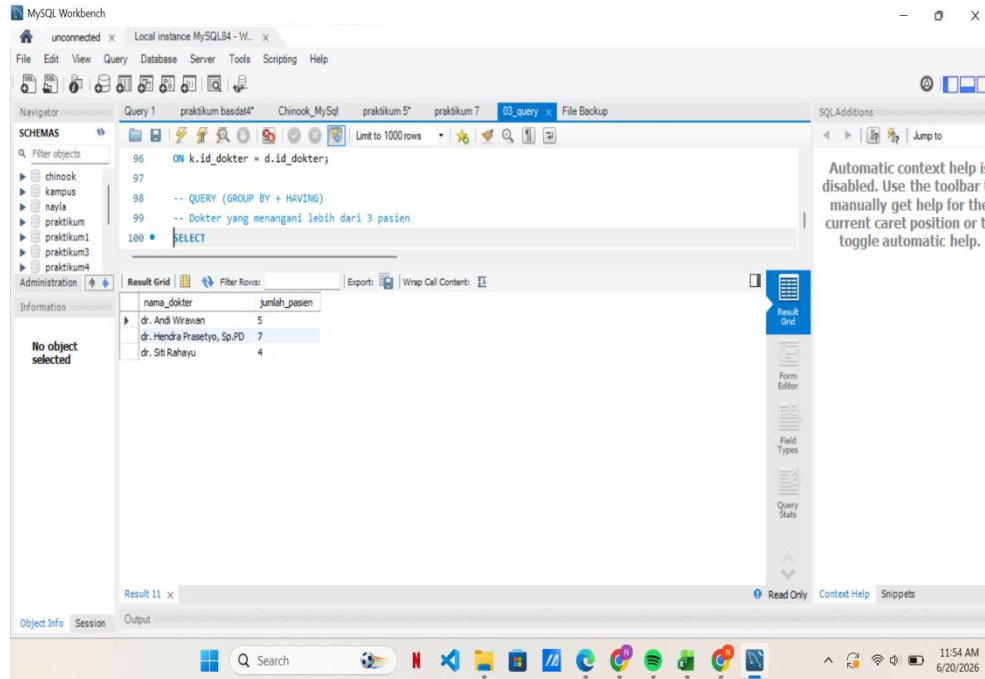


Gambar 5. 17 Tampilan Jumlah Kunjungan Setiap Dokter

CTE berhasil dieksekusi. Hasil menunjukkan 6 dokter dengan distribusi kunjungan: dr. Hendra Prasetyo, Sp.PD (9 kunjungan), dr. Andi Wirawan (5), dr. Lina Marlina, Sp.PD (4), dr. Siti Rahayu (4), dr. Rizky Firmansyah, Sp.B (3), dan drg. Fajar Nugroho, Sp.KG (2).

5.5. Pengujian Query GROUP BY dan HAVING

5.5.1. Menampilkan Dokter yang Menangani Pasien Lebih dari Tiga Pasien



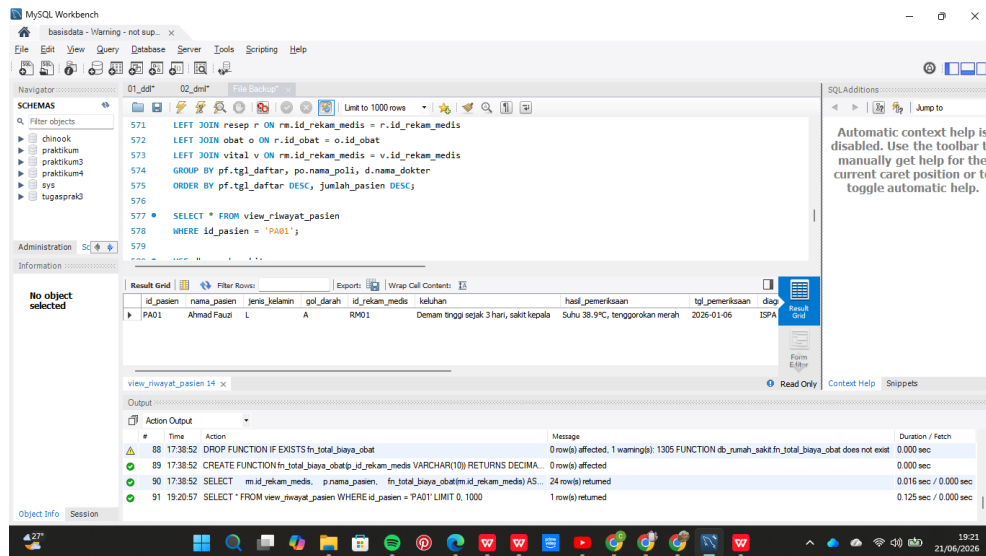
Gambar 5. 18 Tampilan Dokter yang Menangani Lebih dari Tiga Pasien

Query berhasil menyaring dokter sesuai kondisi HAVING. Dokter yang memenuhi syarat ditampilkan beserta jumlah pasien yang ditangani, sedangkan dokter dengan jumlah pasien ≤ 3 tidak muncul dalam hasil.

5.6. Pengujian View

Pengujian view dilakukan dengan memanggil view menggunakan SELECT dan memverifikasi konsistensi data dengan tabel-tabel sumbernya. Script DDL.

5.6.1. Menampilkan pengujian view_riwayat_pasien



Gambar 5. 19 Tampilan Riwayat Pasien

View berhasil menampilkan data lengkap Ahmad Fauzi (PA01) mencakup identitas pasien, rekam medis, diagnosa ISPA, resep Paracetamol 500mg, dokter dr. Andi Wirawan, dan poli Umum Gedung A Lantai 1.

5.6.2. Menampilkan pengujian view_laporan_harian

MySQL Workbench

basidata - Warning - not sup... x

File Edit View Query Database Server Tools Scripting Help

Navigator 01_dd* 02_dml* File Backup* x

Limit to 1000 rows

571 LEFT JOIN resep r ON r.id_rekam_medis = r.id_rekam_medis
572 LEFT JOIN obat o ON r.id_obat = o.id_obat
573 LEFT JOIN vital v ON r.id_rekam_medis = v.id_rekam_medis
574 GROUP BY pf.tgl_daftar, po.nama_poli, d.nama_dokter
575 ORDER BY pf.tgl_daftar DESC, jumlah_pasien DESC;

Result Grid

tanggal	nama_pol	nama_dokter	jumlah_pasien	jumlah_rekam_medis	jumlah_diagnosa	jumlah_resep	jumlah_jenis_obat	total_nilai_re
2026-01-26	Penyakit Dalam	dr. Lina Marlina, Sp.PD	1	1	1	2	2	125000
2026-01-26	Umum	dr. Andi Wirawan	1	1	1	2	2	30000
2026-01-25	Bedah	dr. Rizky Firmansyah, Sp.B	1	1	1	2	2	70000
2026-01-25	Penyakit Dalam	dr. Hendra Prasetyo, Sp.PD	1	1	1	1	1	90000
2026-01-24	Umum	dr. Andi Wirawan	1	1	1	2	2	30000
2026-01-24	Umum	dr. Siti Rahayu	1	1	1	1	1	10000
2026-01-23	Penyakit Dalam	dr. Hendra Prasetyo, Sp.PD	1	1	1	2	2	185000
2026-01-23	Penyakit Dalam	dr. Lina Marlina, Sp.PD	1	1	1	2	2	125000
2026-01-22	Bedah	dr. Rizky Firmansyah, Sp.B	1	1	1	1	1	20000
2026-01-22	Umum	dr. Andi Wirawan	1	1	1	1	1	14000
2026-01-21	Penyakit Dalam	dr. Hendra Prasetyo, Sp.PD	2	2	3	4	4	540000
2026-01-20	Gig dan Mulut	dr. Fajar Nugroho, Sp.KG	1	1	1	2	2	70000
2026-01-20	Umum	dr. Andi Wirawan	1	1	1	2	2	95000
2026-01-20	Umum	dr. Siti Rahayu	1	1	1	1	1	10000
2026-01-19	Penyakit Dalam	dr. Hendra Prasetyo, Sp.PD	1	1	2	2	2	480000

view_laporan_harian 15 x

Output

Action Output

Object Info Session

Time Action Message Trow(s) returned Duration / Fetch

91 19:20:57 SELECT * FROM view_laporan_harian WHERE id_pasien = 'PA01' LIMIT 0, 1000 1 row(s) returned 0.125 sec / 0.000 sec

Gambar 5. 20 Tampilan Laporan Harian

MySQL Workbench

basidata - Warning - not sup... x

File Edit View Query Database Server Tools Scripting Help

Navigator 01_dd* 02_dml* File Backup* x

Limit to 1000 rows

571 LEFT JOIN resep r ON r.id_rekam_medis = r.id_rekam_medis
572 LEFT JOIN obat o ON r.id_obat = o.id_obat
573 LEFT JOIN vital v ON r.id_rekam_medis = v.id_rekam_medis
574 GROUP BY pf.tgl_daftar, po.nama_poli, d.nama_dokter
575 ORDER BY pf.tgl_daftar DESC, jumlah_pasien DESC;

Result Grid

tanggal	nama_pol	nama_dokter	jumlah_pasien	jumlah_rekam_medis	jumlah_diagnosa	jumlah_resep	jumlah_jenis_obat	total_nilai_re
2026-01-22	Umum	dr. Andi Wirawan	1	1	1	1	1	14000
2026-01-21	Penyakit Dalam	dr. Hendra Prasetyo, Sp.PD	2	2	3	4	4	540000
2026-01-20	Gig dan Mulut	dr. Fajar Nugroho, Sp.KG	1	1	1	2	2	70000
2026-01-20	Umum	dr. Andi Wirawan	1	1	1	2	2	95000
2026-01-20	Umum	dr. Siti Rahayu	1	1	1	1	1	10000
2026-01-19	Penyakit Dalam	dr. Hendra Prasetyo, Sp.PD	1	1	2	2	2	480000
2026-01-13	Penyakit Dalam	dr. Lina Marlina, Sp.PD	1	1	1	2	2	85000
2026-01-12	Penyakit Dalam	dr. Hendra Prasetyo, Sp.PD	1	1	1	1	1	20000
2026-01-08	Umum	dr. Siti Rahayu	1	1	1	2	2	130000
2026-01-07	Penyakit Dalam	dr. Hendra Prasetyo, Sp.PD	1	1	1	1	1	20000
2026-01-07	Umum	dr. Siti Rahayu	1	1	1	1	1	7000
2026-01-06	Gig dan Mulut	dr. Fajar Nugroho, Sp.KG	1	1	1	1	1	90000
2026-01-06	Umum	dr. Andi Wirawan	1	1	1	1	1	20000
2026-01-05	Bedah	dr. Rizky Firmansyah, Sp.B	1	1	3	4	3	555000
2026-01-05	Penyakit Dalam	dr. Hendra Prasetyo, Sp.PD	1	0	0	0	0	0

view_laporan_harian 15 x

Output

Action Output

Object Info Session

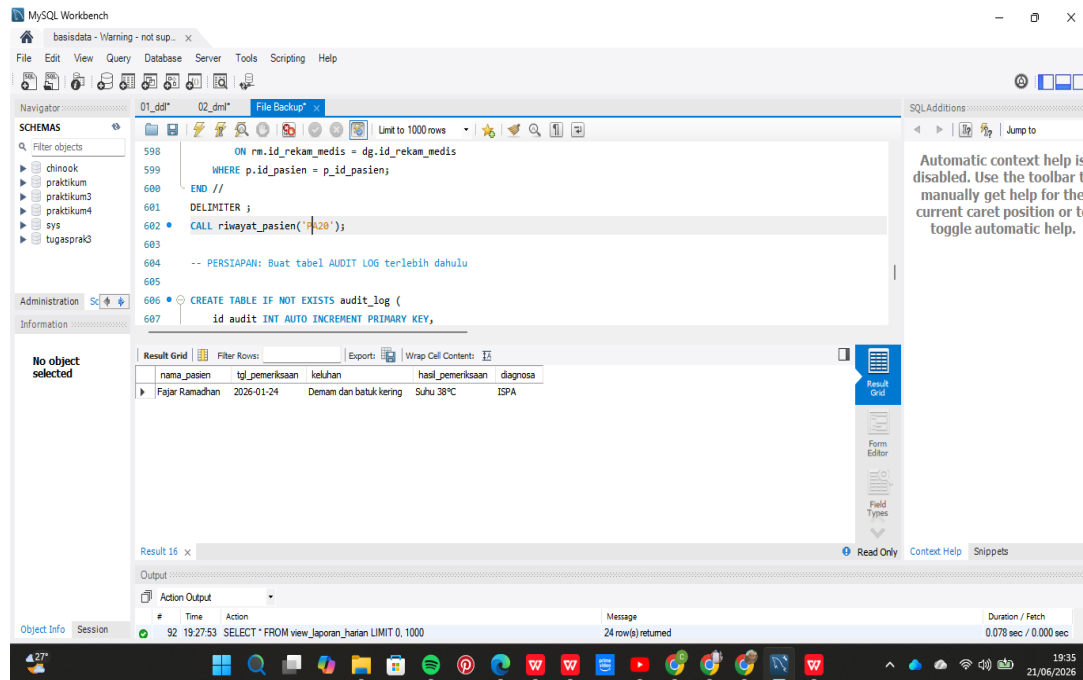
Time Action Message Trow(s) returned Duration / Fetch

91 19:20:57 SELECT * FROM view_laporan_harian WHERE id_pasien = 'PA01' LIMIT 0, 1000 1 row(s) returned 0.125 sec / 0.000 sec

Gambar 5. 21 Tampilan Laporan Harian

View berhasil menampilkan laporan harian dengan data yang teragregasi secara akurat. Kolom total_nilai_resep dan rata_rata_berat_badan_pasien terisi sesuai data dari tabel resep, obat, dan vital. Pengurutan berdasarkan tgl_daftar DESC berjalan benar.

5.7. Pengujian Stored Procedure

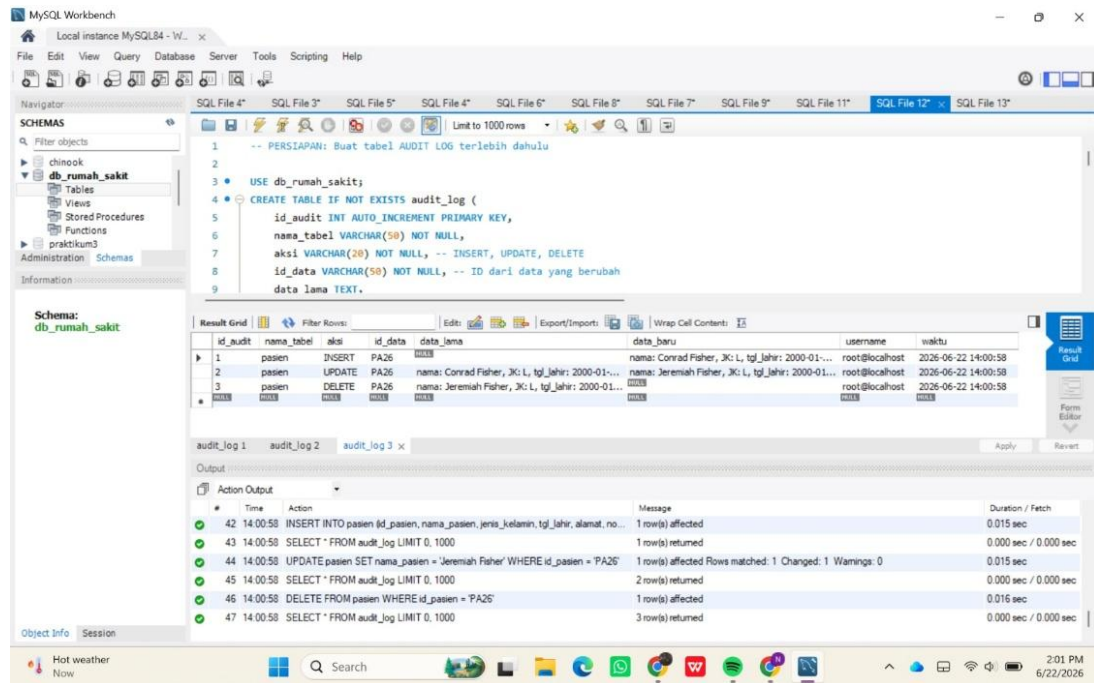


Gambar 5. 22 Tampilan Procedure

Procedure berhasil menampilkan riwayat Fajar Ramadhan sesuai data pada tabel rekam_medis dan diagnosa. Output konsisten dengan hasil query langsung ke tabel.

5.8. Pengujian Trigger

Pengujian trigger dilakukan dengan melakukan operasi INSERT, UPDATE, dan DELETE pada tabel pasien secara berurutan, kemudian memverifikasi isi tabel audit_log setelah setiap operasi.

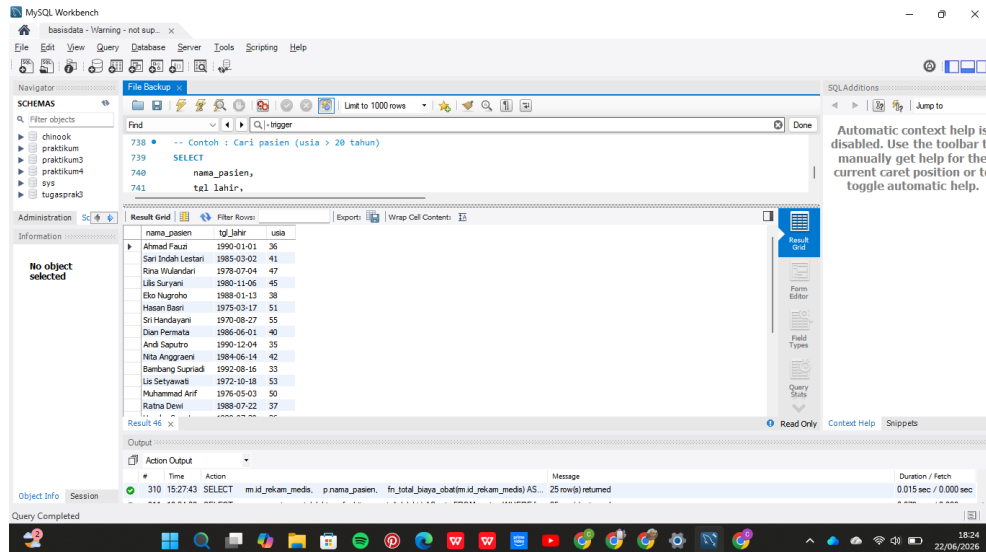


Gambar 5. 23 Tampilan Trigger

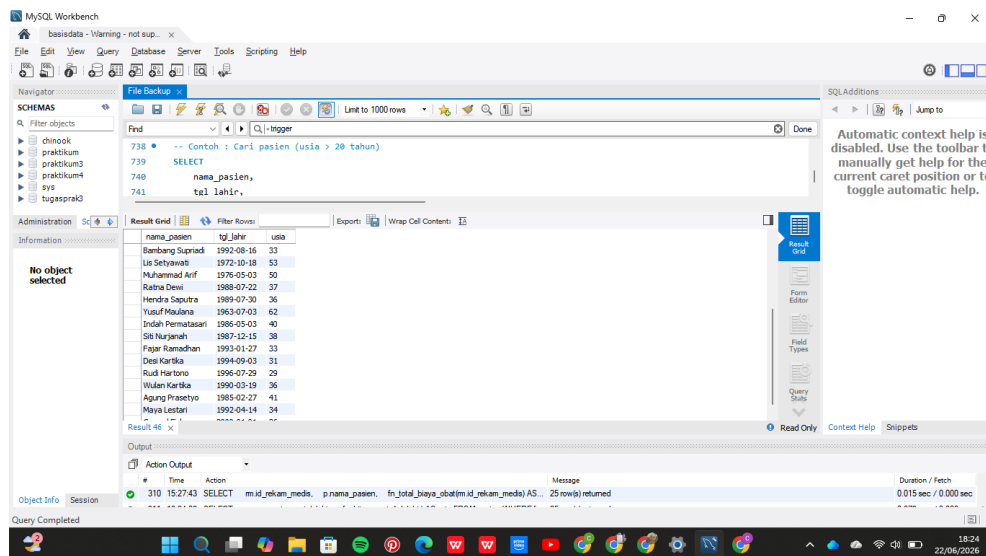
Berdasarkan hasil pengujian trigger menggunakan operasi INSERT, UPDATE, dan DELETE pada tabel pasien, dapat dilihat bahwa setiap perubahan data berhasil tercatat secara otomatis ke dalam tabel audit_log. Saat dilakukan INSERT, trigger mencatat nama tabel (pasien), jenis aksi (INSERT), ID data (PA26), data baru yang berisi seluruh informasi pasien yang ditambahkan, username akun MySQL yang aktif, serta timestamp pencatatan secara otomatis. Pada proses UPDATE, trigger berhasil menyimpan jejak perubahan dengan mencatat nilai sebelum perubahan pada kolom data_lama menggunakan OLD dan nilai setelah perubahan pada kolom data_baru menggunakan NEW, sehingga riwayat perubahan data dapat ditelusuri secara lengkap. Selanjutnya, saat dilakukan DELETE, trigger tetap menyimpan seluruh informasi data pasien yang dihapus menggunakan OLD, sementara kolom data_baru bernilai NULL sesuai dengan rancangan trigger. Dengan demikian, seluruh aktivitas penambahan, perubahan, dan penghapusan data pada tabel pasien berhasil terdokumentasi secara lengkap dalam tabel audit_log.

5.9. Pengujian Function

5.9.1. Menampilkan pengujian fn_hitung_usia



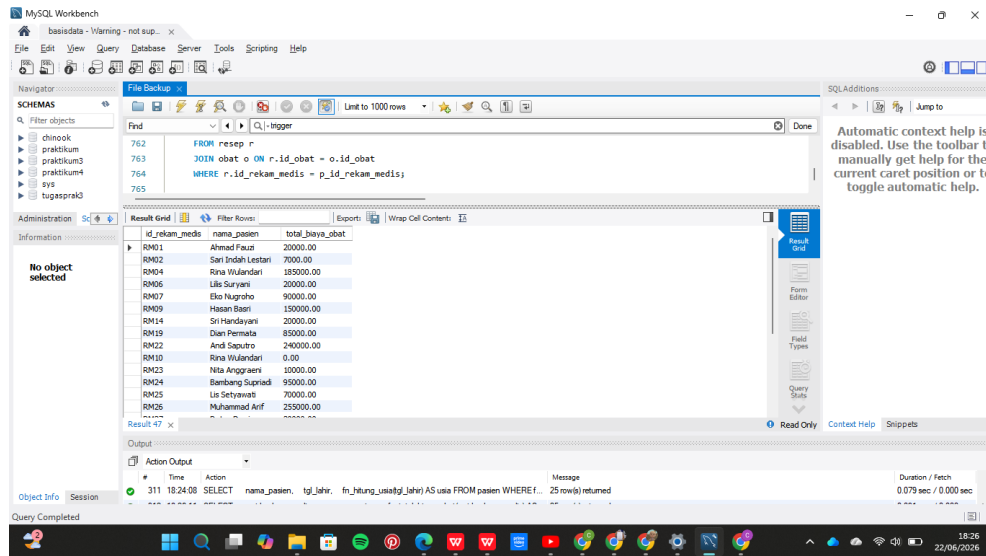
Gambar 5. 24 Tampilan Function Menghitung Usia



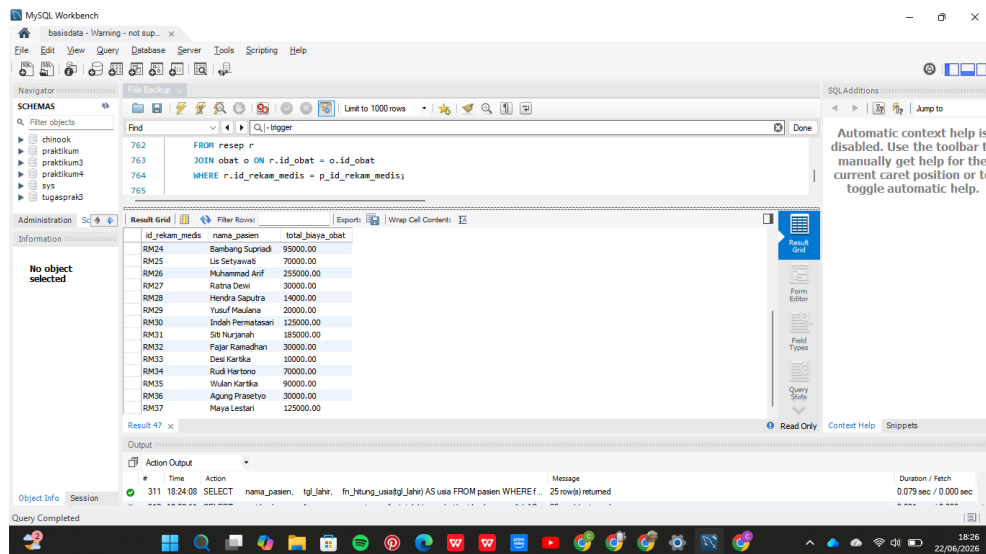
Gambar 5. 25 Tampilan Function Menghitung Usia

Function berhasil menghitung usia seluruh 24 pasien dengan akurat. Seperti, Ahmad Fauzi lahir 1990-01-01, pada tahun pengujian 2026 berusia 36 tahun, sesuai output function. Function juga menangani kondisi NULL dengan mengembalikan NULL sesuai logika IF di awal function.

5.9.2. Menampilkan pengujian fn_total_biaya_obat



Gambar 5. 26 Tampilan Function Total Biaya Obat



Gambar 5. 27 Tampilan Function Total Biaya Obat

Function berhasil menghitung total biaya obat dengan benar. Seperti RM01 (Ahmad Fauzi) memiliki 1 resep Paracetamol 500mg sebanyak 10 buah dengan harga Rp2.000, sehingga total biaya = $10 \times \text{Rp}2.000 = \text{Rp}20.000$.

BAB VI

PENUTUP

6.1 Kesimpulan

Bedasarkan hasil perancangan dan implementasi basis data Sistem Informasi Rumah Sakit (SIRS), dapat disimpulkan bahwa:

1. Basis data relasional berhasil dirancang untuk mengintegrasikan berbagai data operasional rumah sakit yang meliputi data pasien, dokter, poli, jadwal praktik, pendaftaran, rekam medis, vital, diagnosa, obat, dan resep ke dalam satu sistem yang saling terhubung. Struktur basis data yang dirancang mampu mendukung pengelolaan data secara lebih terorganisasi, konsisten, dan efisien.
2. Hubungan antar entitas dalam Sistem Informasi Rumah Sakit berhasil dimodelkan menggunakan Entity Relationship Diagram (ERD) sehingga relasi antar data dapat digambarkan secara jelas sesuai dengan proses bisnis pelayanan rumah sakit. ERD yang dibuat mencerminkan keterkaitan antar entitas dan menjadi dasar dalam pembentukan skema relasional.
3. Proses normalisasi telah diterapkan mulai dari Unnormalized Form (UNF) hingga Third Normal Form (3NF). Hasil normalisasi menghasilkan struktur tabel yang lebih efisien, mengurangi redundansi data, menghindari anomali data, serta menjaga integritas dan konsistensi informasi yang tersimpan dalam basis data.
4. Rancangan basis data berhasil diimplementasikan menggunakan SQL melalui pembuatan database, tabel, relasi, constraint, indeks, serta pengisian data (DDL dan DML). Implementasi tersebut menunjukkan bahwa struktur basis data dapat digunakan untuk mendukung kebutuhan operasional Sistem Informasi Rumah Sakit.
5. Pengembangan query, view, function, stored procedure, dan trigger berhasil dilakukan untuk mendukung pengelolaan data serta kebutuhan operasional rumah sakit. Objek-objek basis data tersebut membantu proses pencarian informasi, penyajian laporan, otomatisasi pengolahan data, serta pengawasan

perubahan data melalui audit log sehingga meningkatkan efektivitas pengelolaan basis data.

6.2 Saran

Bedasarkan hasil proyek yang telah dilakukan, beberapa saran untuk pengembangan lebih lanjut adalah sebagai berikut:

1. Sistem dapat dikembangkan dengan menambahkan modul pelayanan rumah sakit yang lebih lengkap, seperti rawat inap, pembayaran, tagihan, laboratorium, rekam medis elektronik yang lebih detail, serta manajemen persediaan obat.
2. Perlu ditambahkan mekanisme keamanan yang lebih baik, seperti pengaturan hak akses pengguna (role-based access control), enkripsi data sensitif pasien, serta sistem autentikasi untuk menjaga kerahasiaan dan keamanan data.
3. Implementasi trigger dan prosedur tersimpan dapat diperluas untuk mendukung validasi data yang lebih kompleks, pengelolaan stok obat secara otomatis, serta pembuatan laporan operasional secara real-time.
4. Sistem dapat diintegrasikan dengan aplikasi berbasis web atau mobile sehingga proses pengelolaan data dan pelayanan kesehatan dapat dilakukan secara lebih mudah dan fleksibel.
5. Pengujian performa dan optimasi basis data perlu dilakukan pada skala data yang lebih besar agar sistem tetap mampu memberikan kinerja yang baik ketika digunakan dalam lingkungan rumah sakit yang sebenarnya.

LAMPIRAN

Lampiran 1. Tautan Repository GitHub Publik Kelompok

[Link Github](#)

Lampiran 2. Tautan Video Demo Aplikasi

[Link Video Demo Aplikasi](#)

Lampiran 3. Tautan Berkas SQL Mandiri

[Link Berkas SQL](#)

Lampiran 4. Tautan Artificial Intelligence

[Link GPT](#)

[Link GPT](#)